BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

Work Integrated Learning Programmes

AIMLCZG546

Software Engineering for Machine Learning

ASSIGNMENT II

Implementation, Code Quality and Quality Assurance

Project: ChurnGuard

An End-to-End ML-Enabled System for Telecom Customer Churn Prediction

Course	AIMLCZG546 — Software Engineering for Machine Learning
Assessment	Assignment II (EC-1 Component)
Weightage	10 Marks
Group No.	_____
Submission Date	15th August 2026, 23:00
Deliverable	ChurnGuard v1.0.0 — 2,815 lines of Python across 27 modules
Test Suite	98 automated tests, 100% passing, 90% statement coverage
Headline Result	ROC-AUC 0.788 · F1 0.565 · ECE 0.021 · all quality gates passed

Group Details

Please complete the two shaded columns below before submission. The contribution descriptions have been pre-filled to reflect the actual division of work described throughout this report; each section of the solution carries a green banner naming its owner.

Group No: _____

Sl. No	BITS ID	Name	Contribution of Team Member (Qualitative)	% Contribution
1	_____	_____	**Architecture & Modularisation (Objective 1, Tasks 1–2).** Designed the layered package structure; implemented the OOP data-ingestion layer (`DataSource` ABC, `CsvDataSource`, `InMemoryDataSource`, `DataIngestor`) and the functional/OOP hybrid feature-engineering module; authored the research-vs-production comparison, including the prototype notebook and its refactoring into `src/churnguard`; set up the repository, `requirements.txt`, `Makefile` and reproducible environment.	25%
2	_____	_____	**Reliability & Code Quality (Objective 1, Tasks 3–4; Objective 2, Task 8b).** Implemented the exception hierarchy and structured logging across ingestion, validation, training, inference and the API; configured and ran `black`, `isort` and `flake8` and produced the before/after lint evidence; built the data-quality subsystem — schema contract, missing-value gates, range checks and PSI-based drift detection.	25%
3	_____	_____	**API & Deployment Readiness (Objective 1, Task 5; Objective 2, Task 9).** Designed and implemented the FastAPI inference service — versioned routing, Pydantic request/response schemas, status-code semantics, exception handlers, latency middleware and health/readiness endpoints; authored the production experimentation strategy (shadow → canary → A/B) and the security analysis; wrote the multi-stage `Dockerfile` and CI workflow.	25%
4	_____	_____	**Quality Assurance & Model Metrics (Objective 2, Tasks 6–7, 8a).** Designed the four-tier test strategy and wrote the 98-test `pytest` suite — unit, data-validation, ML-behavioural and integration; implemented the overfit-small-batch, loss-decrease, leakage and directional-expectation tests; implemented the model-quality metrics module including Expected Calibration Error and the release quality gates; produced coverage and evaluation reports.	25%

How the work was divided

The split follows the natural seams of the system rather than the numbering of the question, so that each member owned a vertical slice they could test and defend end to end. Members 1 and 2 own the *offline* path (data in, model out); members 3 and 4 own the *online and assurance* path (model out, service and evidence in).

Every module carries the owning member in its section banner, and all four members reviewed each other's pull requests — the CI pipeline in Appendix C enforces that no code merges without passing the other members' gates.

Table of Contents

> **A note on the evidence in this report**
>
> Every metric, log line, lint report, HTTP transcript and figure in this document was produced by executing the code that is listed in the appendices. Nothing is illustrative or hand-written. The console captures retain their original timestamps, and the code listings are read directly from the source files at document-build time, so they cannot drift from what was actually run.

PART A

ASSIGNMENT PROBLEM STATEMENT

reproduced as issued

AIMLCZG546 – Software Engineering for Machine Learning

Assignment II

Overview

This assignment builds on Assignment I and focuses on the implementation, code quality, and testing/QA aspects of building an ML-enabled system. You will take an ML application (either the same one from Assignment I or a new one, per the problem statement chosen) and demonstrate sound coding practices, refactoring, API design, and a rigorous testing strategy — including deployment-readiness aspects where applicable.

General Instructions

- It is a Group Assignment. Please follow the naming convention as <Group no>.ipynb.
- Inside each report and implementation notebooks, you are required to mention your name, Group details.

Group Details

Group No: <Fill Group No>

Group Member Names with Contribution

Sl. No	BITS ID	Name	Contribution of team member (Qualitative)	Percentage Contribution (Out of 100) Quantitative
1				
2				
3				
4				

- Weightage – 10 marks
- Assignment 2 Submission Date – 15th August 2026, 23:00.
- Submissions after the due date will result in deduction of marks.

Note

1. No extension of deadline will be given under any circumstances. Manage your time and complete the assignment within the deadline.

2. Since Assignment-II is part of EC-1 component, there is NO MAKEUP for this component.
3. Any questions or clarifications regarding the assignment can be emailed to "omshree.b@wilp.bits-pilani.ac.in".

Project Details

Objective 1: Implementation and Code Sharing [5 Marks]

4. Refactor/design your ML application code using appropriate OOP and/or functional programming principles (e.g., separate classes/modules for data ingestion, feature engineering, model training, and inference).
5. Clearly distinguish and demonstrate the difference between research code vs production code for at least one component (e.g., a Jupyter notebook prototype vs. a modularized, production-style script/package).
6. Implement proper error handling and logging (e.g., using Python's logging module) across at least 3 critical functions/modules, with meaningful log levels (INFO, WARNING, ERROR).
7. Apply code formatting and linting tools (e.g., black, flake8/pylint, isort) to your codebase; include before/after screenshots or lint reports.
8. Design and implement a simple REST API (e.g., using FastAPI) to expose the model's inference functionality, following good API design practices (clear endpoints, request/response schemas, status codes).

Objective 2: Quality Assurance [5 Marks]

9. Identify and implement at least 2 different types of tests for your ML system (e.g., unit tests, integration tests, data validation tests) using pytest.
10. Implement specific tests for ML components:
    a. Testing model training (e.g., overfitting on a small batch, checking loss decreases)
    b. Testing model inference (e.g., output shape/range checks, invariance/directional tests)
11. Define and measure at least 2 metrics each for:
    a. Model Quality (e.g., accuracy, F1, calibration)
    b. Data Quality (e.g., schema validation, missing value checks, drift detection)
12. Briefly describe an approach for testing/experimentation in production (e.g., shadow deployment, canary release, A/B testing) and one security consideration relevant to your ML system (e.g., input validation to prevent adversarial inputs, model/data access control).

Submission Guidelines

Prepare a Word/ PDF document containing the project details (All tasks, screenshots of the application as required, along with explanation) and code. Highlight clearly the contribution of each group member in executing the assignment. Upload the file as <groupid.docx> or <groupid.pdf> into the Taxila portal.

All the Best!

PART B

SOLUTION

ChurnGuard — design, implementation and quality assurance

---

1. System Overview: What ChurnGuard Is and Why

1.1 The business problem

A telecom operator loses roughly a quarter of its subscriber base every year. Winning a new subscriber costs far more than retaining an existing one, so the retention team wants to know, each month, **which customers are about to leave and what to do about it**. ChurnGuard is the ML-enabled system that answers that question.

Crucially, the system does not merely emit a number. The retention team cannot act on "0.71". It acts on "CRITICAL — assign a retention agent within seven days". ChurnGuard therefore returns a calibrated probability, a binary decision, a risk band, and a recommended action, because the unit of value is a *decision*, not a prediction.

> **Why this is an ML-enabled system, not just a model**
>
> The model is perhaps 5% of the code. The remaining 95% — ingestion contracts, validation gates, feature pipelines, serving, tests, logging, CI — is what makes the model usable, observable and safe to change. This assignment is about that 95%, and this report is organised accordingly.

1.2 Why probability calibration drives the whole design

One design decision propagates through every layer of this system, so it is worth stating up front. The retention team multiplies the churn probability by the customer's lifetime value to decide the size of a discount to offer. That means a model which ranks customers correctly but is *over-confident* would systematically over-spend on discounts, even with an excellent AUC.

Consequently ChurnGuard: (a) wraps the classifier in `CalibratedClassifierCV` with isotonic regression; (b) measures **Brier score** and **Expected Calibration Error** alongside accuracy and F1; and (c) enforces a maximum Brier score in its release gates. Section 10 shows the resulting reliability diagram — the achieved ECE is 0.021, meaning the model's stated confidence is accurate to roughly two percentage points.

1.3 Dataset

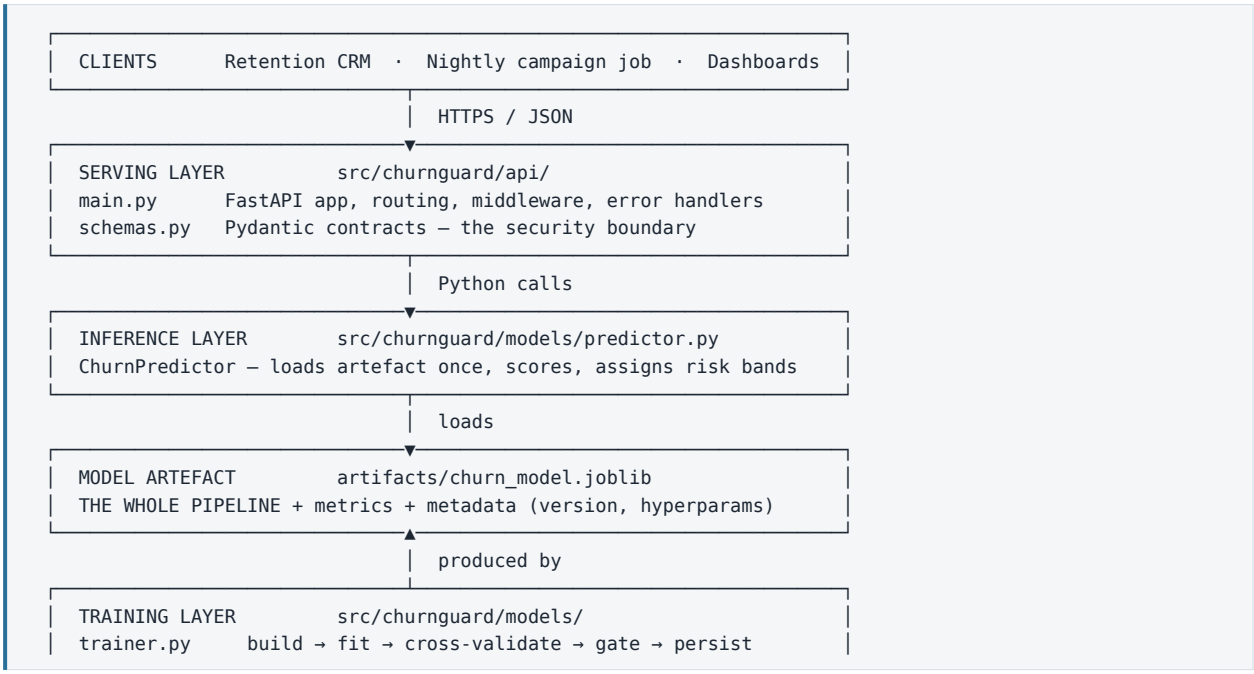
The system is trained on a 5,000-row telecom subscriber dataset with a 26.7% churn rate, generated by `scripts/generate_data.py` with a fixed seed so that every number in this report is reproducible. The generator encodes realistic causal structure (short tenure, month-to-month contracts, electronic-check payment and frequent support tickets all raise risk) and injects ~1.2% missing values so that the data-validation tests in Section 11 have genuine defects to detect.

Column	Type	Role	Description
customer_id	string	identifier	Excluded from features; echoed back in API responses for joins
tenure_months	int	numeric feature	Months since activation (1–72)
monthly_charges	float	numeric feature	Current monthly bill
total_charges	float	numeric feature	Lifetime billed amount (~1.2% null)
support_tickets_6m	int	numeric feature	Support contacts in the last six months
avg_monthly_gb	float	numeric feature	Average data consumption (~0.6% null)
contract_type	category	categorical feature	Month-to-month / One year / Two year
internet_service	category	categorical feature	Fiber optic / DSL / No
payment_method	category	categorical feature	Electronic check / Mailed check / Bank transfer / Credit card
tech_support	category	categorical feature	Yes / No
paperless_billing	category	categorical feature	Yes / No
churn	int	**target**	1 if the customer left within the observation window

Table 1.1 — The data contract. This table is not documentation-only: it is enforced in code by src/churnguard/data/validation.py (Section 11).

1.4 Architecture

ChurnGuard is a layered system. Each layer depends only on the layer beneath it and on abstractions rather than concrete implementations, which is what makes the components independently testable.



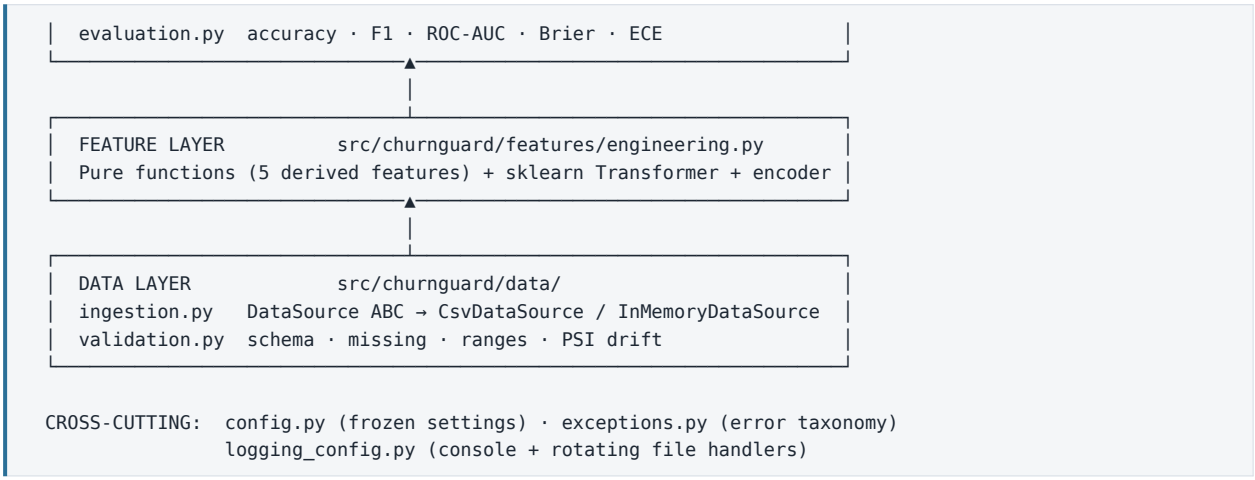


Figure 1.1 — Layered architecture. Arrows point in the direction of dependency.

1.5 Technology choices and their justification

Concern	Choice	Why this and not the obvious alternative
Modelling	scikit-learn Pipeline + RandomForest + isotonic calibration	A `Pipeline` makes the transform inseparable from the model, eliminating train/serve skew by construction. Gradient boosting would score marginally higher but a calibrated forest is more stable on 5k rows and trains in 2 s, keeping CI fast.
Serving	FastAPI + Pydantic v2	Pydantic gives declarative validation *and* an auto-generated OpenAPI schema from the same type annotations, so the contract cannot drift from the code. Flask would require hand-written validation and hand-maintained docs.
Persistence	joblib with a metadata envelope	Bare `pickle.dump(model)` loses provenance. The envelope carries version, training timestamp, hyper-parameters and test metrics, which is what makes the `/v1/model/info` endpoint and post-incident audit possible.
Testing	pytest with session-scoped fixtures	Training a pipeline costs ~2 s; a session fixture amortises that across 98 tests so the whole suite runs in 17 s. A slow suite is a suite developers skip.
Config	Frozen dataclasses + env overrides	Immutability prevents a module from mutating shared state at import time — a class of bug that is very hard to reproduce. Env overrides let the same artefact run on a laptop, in CI and in a container.
Formatting	black + isort + flake8	black removes formatting from code review entirely; isort with `profile="black"` guarantees the two tools do not fight; flake8 catches the logic-adjacent issues (unused imports, complexity) that formatters cannot.

2. Setup, Requirements and How to Run

This section is written so that an assessor can reproduce every number in this report from a clean machine.

2.1 Prerequisites

- **Python 3.10 or newer** (developed and tested on 3.12.3)
- **pip** and, optionally, **make** for the convenience targets
- **Docker** (optional) — only needed for the containerised deployment in Section 12
- Roughly 500 MB of disk for dependencies and the ~19 MB model artefact

2.2 Repository layout

```
churnguard/
├── data/raw/telco_churn.csv          # generated dataset (5,000 rows)
├── notebooks/
│   └── 01_research_prototype.ipynb   # RESEARCH code — the "before" of Section 4
├── src/churnguard/                   # PRODUCTION package — the "after"
│   ├── __init__.py                   # version marker
│   ├── config.py                     # frozen settings, quality gates
│   ├── exceptions.py                 # domain error taxonomy
│   ├── logging_config.py             # console + rotating file handlers
│   ├── data/
│   │   ├── ingestion.py              # DataSource ABC, DataIngestor, DataSplit
│   │   └── validation.py             # schema · missing · ranges · PSI drift
│   ├── features/
│   │   └── engineering.py            # pure feature fns + sklearn Transformer
│   ├── models/
│   │   ├── trainer.py                # ChurnModelTrainer
│   │   ├── evaluation.py             # metrics incl. ECE and release gates
│   │   └── predictor.py              # ChurnPredictor (inference)
│   └── api/
│       ├── schemas.py                # Pydantic request/response contracts
│       └── main.py                   # FastAPI application
├── tests/                            # 98 tests across 4 categories
│   ├── conftest.py                   # shared session-scoped fixtures
│   ├── test_data_ingestion.py        # 13  unit
│   ├── test_features.py              # 13  unit
│   ├── test_data_validation.py       # 15  data validation
│   ├── test_model_training.py        # 11  ML behavioural (training)
│   ├── test_model_inference.py       # 27  ML behavioural (inference)
│   └── test_api_integration.py       # 19  integration
├── scripts/
│   ├── generate_data.py              # reproducible dataset generator
│   ├── train.py                      # training entry point (CI gate)
│   ├── make_figures.py               # report figures
│   └── api_demo.py                   # endpoint transcript generator
├── artifacts/                        # model + reference distribution profile
├── reports/                          # metrics, logs, lint reports, figures
├── Dockerfile                        # multi-stage, non-root runtime
├── Makefile                          # setup · data · train · test · lint · serve
├── pyproject.toml                    # black + isort configuration
├── setup.cfg                         # flake8 configuration
├── pytest.ini                        # test discovery and path configuration
├── requirements.txt                  # fully pinned dependencies
└── .github/workflows/ci.yml          # three-gate CI pipeline
```

2.3 Installation

```
# 1. Create and activate an isolated environment
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate

# 2. Install pinned dependencies
pip install -r requirements.txt

# 3. Generate the dataset (deterministic — same file every time)
python scripts/generate_data.py
#    -> Wrote 5,000 rows to data/raw/telco_churn.csv | churn rate = 0.267

# 4. Train, evaluate and persist the model
python scripts/train.py --cv 5

# 5. Run the full test suite
pytest -v

# 6. Start the inference API
uvicorn churnguard.api.main:app --reload --port 8000
#    -> interactive docs at http://localhost:8000/docs
```

2.4 Pinned dependencies (requirements.txt)

Versions are pinned exactly rather than with `>=`. An unpinned dependency means the model that passes CI today may differ from the model that ships tomorrow — an unacceptable property for a system whose output is a business decision.

```
# ---- runtime -------------------------------------------------------------
pandas==3.0.2
numpy==2.4.4
scikit-learn==1.8.0
scipy==1.17.1
joblib==1.5.3
fastapi==0.141.1
pydantic==2.13.4
uvicorn[standard]==0.34.0

# ---- visualisation (training reports) ------------------------------------
matplotlib==3.10.8

# ---- quality assurance ---------------------------------------------------
pytest==9.1.1
pytest-cov==6.0.0
httpx==0.28.1

# ---- code quality --------------------------------------------------------
black==26.5.1
flake8==7.3.0
isort==8.0.1
```

2.5 Task automation (Makefile)

The `Makefile` is the single source of truth for how the project is operated, and CI invokes the same targets a developer does — so "works on my machine" and "works in CI" cannot diverge.

```
.PHONY: help setup data train test lint format serve figures all clean
```

```
help:            ## Show this help
    @grep -E '^[a-zA-Z_-]+:.*?## .*$$' $(MAKEFILE_LIST) | awk 'BEGIN{FS=":.*?## "}{printf "  \033[36m
%-10s\033[0m %s\n", $$1, $$2}'

setup:           ## Install all dependencies
    pip install -r requirements.txt

data:            ## Regenerate the raw dataset
    python scripts/generate_data.py

train:           ## Run the full training pipeline (fails on gate breach)
    python scripts/train.py --cv 5

test:            ## Run the test suite with coverage
    pytest -v --cov=src/churnguard --cov-report=term-missing

lint:            ## Static analysis (CI-safe: checks only, no writes)
    isort --check-only --diff src tests scripts
    black --check src tests scripts
    flake8 src tests scripts

format:          ## Auto-format the codebase
    isort src tests scripts
    black src tests scripts

serve:           ## Start the inference API locally
    uvicorn churnguard.api.main:app --reload --port 8000

figures:         ## Regenerate report figures
    python scripts/make_figures.py

all: format lint test train  ## Format, lint, test and train

clean:
    find . -type d -name __pycache__ -exec rm -rf {} + 2>/dev/null || true
    rm -rf .pytest_cache .coverage
```

OBJECTIVE 1

IMPLEMENTATION AND CODE SHARING

5 Marks — Tasks 1 to 5

3. Task 1 — Modular Design with OOP and Functional Programming

Owner: **Member 1** — Architecture & Modularisation. Deliverables: layered package structure, data-ingestion class hierarchy, feature-engineering module, repository and environment setup.

Requirement. *Refactor/design your ML application code using appropriate OOP and/or functional programming principles (e.g., separate classes/modules for data ingestion, feature engineering, model training, and inference).*

3.1 The design principle applied

The refactoring is driven by a single question asked of every module: **what is the one reason this file should change?** If a file would need editing for two unrelated reasons, it is split. This is the Single Responsibility Principle, and applying it mechanically produced the four required components plus three cross-cutting concerns.

Module	Single responsibility	Paradigm	Changes only when...
`data/ingestion.py`	Get raw data from somewhere and split it	OOP — ABC + polymorphism	...a new storage backend is added
`data/validation.py`	Decide whether data is fit to use	OOP + pure functions	...the data contract changes
`features/engineering.py`	Turn raw columns into model inputs	Functional core, OOP shell	...a feature is added or redefined
`models/trainer.py`	Fit, cross-validate and persist a pipeline	OOP — stateful lifecycle	...the algorithm or persistence format changes
`models/evaluation.py`	Compute quality metrics	Functional — pure	...a new metric is required
`models/predictor.py`	Serve predictions from an artefact	OOP — encapsulated state	...the inference contract changes
`api/`	Expose inference over HTTP	OOP (Pydantic) + declarative	...the HTTP interface changes
`config.py`	Hold all tunable values	Immutable data	...a parameter or gate changes
`exceptions.py`	Define the error taxonomy	OOP — inheritance	...a new failure mode appears
`logging_config.py`	Configure observability	Procedural, idempotent	...log routing changes

3.2 Why both paradigms, and where each is used

Choosing "OOP or functional" as an identity is a mistake; each solves a different problem, and this codebase uses each where it fits.

	Object-oriented	Functional
Used for	Components with a lifecycle or swappable	Stateless transformations and calculations: the five

	Object-oriented	Functional
	implementations: `DataSource`, `DataIngestor`, `ChurnModelTrainer`, `ChurnPredictor`, `ChurnFeatureEngineer`	derived-feature functions, `population_stability_index`, `expected_calibration_error`, `evaluate`, `assign_risk_band`
Why	These objects *have state* (a fitted pipeline, a loaded artefact) and *have alternatives* (CSV vs in-memory source). Polymorphism lets callers depend on the interface, not the implementation.	These have no state and no alternatives — they are mathematics. Pure functions are referentially transparent, so they can be tested with a single assertion and composed without surprises.
Test consequence	Tested through their public interface with fixtures; substitutability verified by `InMemoryDataSource` standing in for `CsvDataSource` in every test.	Tested with direct input/output assertions — e.g. `avg_spend_per_month` on 1,200 charges over 12 months must equal exactly 100.0.

3.3 Data ingestion — OOP with an abstract interface

`DataSource` is an abstract base class declaring two members. Every consumer in the system — the trainer, the tests, the batch job — depends on this abstraction. Swapping the CSV for a Snowflake table means adding one subclass and changing nothing else, which is the Open/Closed Principle in practice.

`src/churnguard/data/ingestion.py — the abstraction`

```
class DataSource(ABC):
    """Abstract read-only data source."""

    @abstractmethod
    def load(self) -> pd.DataFrame:
        """Return the raw dataset, or raise ``DataIngestionError``."""

    @property
    @abstractmethod
    def name(self) -> str:
        """Human-readable identifier used in logs and lineage metadata."""
```

Two concrete implementations exist. `CsvDataSource` reads from disk and converts every foreseeable I/O failure into a domain error (discussed in Section 5). `InMemoryDataSource` wraps an existing DataFrame — this is not a test-only convenience, it is what the nightly batch-scoring job uses, and its existence is why the test suite never touches the filesystem to obtain data.

`src/churnguard/data/ingestion.py — concrete sources`

```
class CsvDataSource(DataSource):
    """Reads a dataset from a local (or mounted) CSV file."""

    def __init__(self, path: Path | str) -> None:
        self.path = Path(path)

    @property
    def name(self) -> str:
        return f"csv://{self.path}"

    def load(self) -> pd.DataFrame:
        logger.info("Loading data from %s", self.name)
        if not self.path.exists():
            logger.error("Data file not found: %s", self.path)
```

```
            raise DataIngestionError(f"Data file not found: {self.path}")
        try:
            frame = pd.read_csv(self.path)
        except pd.errors.EmptyDataError as exc:
            logger.error("Data file %s is empty", self.path)
            raise DataIngestionError(f"Data file is empty: {self.path}") from exc
        except (pd.errors.ParserError, UnicodeDecodeError) as exc:
            logger.error("Malformed CSV at %s: %s", self.path, exc)
            raise DataIngestionError(f"Could not parse CSV: {self.path}") from exc

        if frame.empty:
            logger.error("Data file %s parsed to zero rows", self.path)
            raise DataIngestionError(f"No rows found in {self.path}")

        logger.info("Loaded %d rows x %d columns", *frame.shape)
        return frame
```

```
class InMemoryDataSource(DataSource):
    """Wraps an existing DataFrame -- used by tests and by batch scoring jobs."""

    def __init__(self, frame: pd.DataFrame, label: str = "in-memory") -> None:
        self._frame = frame
        self._label = label

    @property
    def name(self) -> str:
        return f"memory://{self._label}"

    def load(self) -> pd.DataFrame:
        logger.info("Using %s (%d rows)", self.name, len(self._frame))
        return self._frame.copy()
```

`DataIngestor` orchestrates. Note that it accepts a `DataSource`, not a path — dependency injection, which is precisely what makes it testable without any I/O. It also encodes two non-obvious safeguards: it refuses to proceed on a single-class target, and it disables stratification (with a warning) when the minority class is too small to stratify, rather than crashing.

`src/churnguard/data/ingestion.py — the orchestrator`

```
class DataIngestor:
    """Orchestrates load -> sanity-check -> stratified split."""

    def __init__(self, source: DataSource, config: ModelConfig | None = None) -> None:
        self.source = source
        self.config = config or SETTINGS.model

    def load_raw(self) -> pd.DataFrame:
        frame = self.source.load()
        missing = [
            c for c in (*SETTINGS.all_features, self.config.target) if c not in frame.columns
        ]
        if missing:
            logger.error("Source %s is missing required columns: %s", self.source.name, missing)
            raise DataIngestionError(f"Missing required columns: {missing}")

        duplicates = int(frame.duplicated(subset=[self.config.id_column]).sum())
        if duplicates:
            logger.warning("Dropping %d duplicate %s rows", duplicates, self.config.id_column)
            frame = frame.drop_duplicates(subset=[self.config.id_column], keep="first")
```

```
        return frame

    def split(self, frame: pd.DataFrame | None = None) -> DataSplit:
        frame = self.load_raw() if frame is None else frame
        target = frame[self.config.target]

        if target.nunique() < 2:
            logger.error("Target '%s' has a single class -- cannot train", self.config.target)
            raise DataIngestionError("Target column must contain at least two classes")

        minority = int(target.value_counts().min())
        stratify = target if minority >= 2 else None
        if stratify is None:
            logger.warning("Minority class has %d row(s); stratification disabled", minority)

        x_train, x_test, y_train, y_test = train_test_split(
            frame[SETTINGS.all_features],
            target,
            test_size=self.config.test_size,
            random_state=self.config.random_state,
            stratify=stratify,
        )
        split = DataSplit(x_train, x_test, y_train, y_test)
        logger.info("Split complete: %s", split.summary())
        return split
```

The `DataSplit` frozen dataclass keeps the four arrays together. Passing `x_train, x_test, y_train, y_test` around as four positional arguments invites the classic bug of swapping two of them; a single typed object makes that impossible.

`src/churnguard/data/ingestion.py — the split value object`

```
class DataSplit:
    """The four arrays every downstream component needs, kept together."""

    x_train: pd.DataFrame
    x_test: pd.DataFrame
    y_train: pd.Series
    y_test: pd.Series

    def summary(self) -> dict[str, object]:
        return {
            "n_train": int(len(self.x_train)),
            "n_test": int(len(self.x_test)),
            "n_features": int(self.x_train.shape[1]),
            "train_churn_rate": round(float(self.y_train.mean()), 4),
            "test_churn_rate": round(float(self.y_test.mean()), 4),
        }
```

Evidence that the abstraction is real

The entire 98-test suite obtains its data through `InMemoryDataSource`, while `scripts/train.py` uses `CsvDataSource` — the same `DataIngestor` code path serves both without modification. If the abstraction were cosmetic, this would not be possible.

3.4 Feature engineering — functional core, object-oriented shell

This module is the clearest demonstration of using both paradigms deliberately. Each derived feature is a **pure function** `DataFrame → Series`, registered in a dictionary. Adding a feature means writing one function and one registry entry; nothing else in the system changes.

`src/churnguard/features/engineering.py — the functional core`

```
    """Lifetime value normalised by tenure. Reveals mis-billed / upgraded accounts."""
    return (frame["total_charges"] / frame["tenure_months"].clip(lower=1)).astype(float)


def tickets_per_year(frame: pd.DataFrame) -> pd.Series:
    """Support-contact intensity, annualised. The strongest dissatisfaction proxy."""
    return (frame["support_tickets_6m"] * 2.0).astype(float)


def is_new_customer(frame: pd.DataFrame) -> pd.Series:
    """Customers inside the first two billing quarters churn disproportionately."""
    return (frame["tenure_months"] <= 6).astype(int)


def charge_to_usage_ratio(frame: pd.DataFrame) -> pd.Series:
    """Rupees billed per GB consumed -- a proxy for perceived value for money."""
    return (frame["monthly_charges"] / frame["avg_monthly_gb"].fillna(0.0).clip(lower=1.0)).astype(
        float
    )


def has_no_protection(frame: pd.DataFrame) -> pd.Series:
    """Fibre customers without tech support are the highest-risk segment."""
    return ((frame["tech_support"] == "No") & (frame["internet_service"] == "Fiber optic")).astype(
        int
    )


#: Registry of derived features. Adding a feature = adding one entry + one test.
DERIVED_FEATURES: Mapping[str, FeatureFn] = {
    "avg_spend_per_month": avg_spend_per_month,
    "tickets_per_year": tickets_per_year,
    "is_new_customer": is_new_customer,
    "charge_to_usage_ratio": charge_to_usage_ratio,
    "has_no_protection": has_no_protection,
}


def apply_derived_features(
    frame: pd.DataFrame, registry: Mapping[str, FeatureFn] = DERIVED_FEATURES
) -> pd.DataFrame:
    """Fold the registry over a copy of ``frame``. Never mutates the input."""
    out = frame.copy()
    for name, fn in registry.items():
        try:
            out[name] = fn(frame)
        except KeyError as exc:
            logger.error("Cannot compute '%s': missing input column %s", name, exc)
            raise FeatureEngineeringError(f"Feature '{name}' needs column {exc}") from exc
        except (TypeError, ValueError) as exc:
            logger.error("Feature '%s' failed: %s", name, exc)
            raise FeatureEngineeringError(f"Feature '{name}' could not be computed") from exc
    logger.debug("Derived %d features: %s", len(registry), list(registry))
    return out
```

```
# ------------------------------------------------------------------ OOP wrapper
class ChurnFeatureEngineer(BaseEstimator, TransformerMixin):
```

`apply_derived_features` folds the registry over a **copy** of the input. Purity here is not academic: it is asserted by a test (`test_functions_are_pure_and_do_not_mutate_input`) that deep-copies the frame, runs the transformation, and requires the original to be byte-identical afterwards. Silent mutation of a caller's DataFrame is one of the most common and most confusing bugs in pandas code.

src/churnguard/features/engineering.py — the fold

```
def apply_derived_features(
    frame: pd.DataFrame, registry: Mapping[str, FeatureFn] = DERIVED_FEATURES
```

The **object-oriented shell** wraps that pure core in a scikit-learn transformer. This is what allows the feature logic to live *inside* the serialised pipeline. The critical method is `transform`, which reindexes to the column order recorded at `fit` time — so a JSON payload whose keys arrive in a different order still produces an identically ordered matrix.

src/churnguard/features/engineering.py — the OOP shell

```
class ChurnFeatureEngineer(BaseEstimator, TransformerMixin):
    """Scikit-learn transformer that appends the derived features.

    ``fit`` records the output column order; ``transform`` guarantees that order,
    so a payload whose JSON keys arrive in a different order still produces an
    identically-ordered matrix at inference time.
    """

    def __init__(self, registry: Mapping[str, FeatureFn] | None = None) -> None:
        self.registry = registry or DERIVED_FEATURES

    def fit(self, X: pd.DataFrame, y: pd.Series | None = None) -> ChurnFeatureEngineer:
        if not isinstance(X, pd.DataFrame):
            raise FeatureEngineeringError("ChurnFeatureEngineer expects a pandas DataFrame")
        self.feature_names_in_ = list(X.columns)
        self.output_columns_ = list(apply_derived_features(X.head(2), self.registry).columns)
        logger.info(
            "ChurnFeatureEngineer fitted: %d in -> %d out",
            len(self.feature_names_in_),
            len(self.output_columns_),
        )
        return self

    def transform(self, X: pd.DataFrame) -> pd.DataFrame:
        if not hasattr(self, "output_columns_"):
            raise FeatureEngineeringError("transform() called before fit()")
        if not isinstance(X, pd.DataFrame):
            raise FeatureEngineeringError("ChurnFeatureEngineer expects a pandas DataFrame")
        missing = set(self.feature_names_in_) - set(X.columns)
        if missing:
            raise FeatureEngineeringError(f"Input is missing columns: {sorted(missing)}")
        return apply_derived_features(X, self.registry)[self.output_columns_]

    def get_feature_names_out(self, input_features=None) -> np.ndarray:  # noqa: ARG002
        return np.asarray(self.output_columns_, dtype=object)
```

Finally the preprocessor. `handle_unknown="ignore"` is a deliberate production decision: an unseen category degrades to an all-zero block rather than raising an exception during a 3 a.m. batch run. Section 9 contains the test that proves this.

src/churnguard/features/engineering.py — the preprocessor

```
def build_preprocessor() -> ColumnTransformer:
    """Impute + scale numerics, impute + one-hot encode categoricals.

    ``handle_unknown="ignore"`` means an unseen category degrades to an all-zero
    block instead of raising at 3 a.m. in production.
    """
    numeric_columns = list(SETTINGS.model.numeric_features) + [
        "avg_spend_per_month",
        "tickets_per_year",
        "is_new_customer",
        "charge_to_usage_ratio",
        "has_no_protection",
    ]
    numeric_pipeline = Pipeline(
        [
            ("impute", SimpleImputer(strategy="median")),
            ("scale", StandardScaler()),
        ]
    )
    categorical_pipeline = Pipeline(
        [
            ("impute", SimpleImputer(strategy="most_frequent")),
            ("encode", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
        ]
    )
    return ColumnTransformer(
        [
            ("numeric", numeric_pipeline, numeric_columns),
            ("categorical", categorical_pipeline, list(SETTINGS.model.categorical_features)),
        ],
        remainder="drop",
        verbose_feature_names_out=False,
    )
```

3.5 Model training and inference — separate objects, separate lifecycles

Training and inference are different responsibilities with different dependencies, so they are different classes in different files. `ChurnModelTrainer` needs scikit-learn's model-selection machinery and writes to disk; `ChurnPredictor` needs only to read an artefact and score. Keeping them apart means the serving container does not have to carry the training code path.

src/churnguard/models/trainer.py — pipeline construction

```
    def build_pipeline(self) -> Pipeline:
        """Assemble feature engineering -> preprocessing -> calibrated classifier."""
        forest = RandomForestClassifier(
            n_estimators=self.config.n_estimators,
            max_depth=self.config.max_depth,
            min_samples_leaf=self.config.min_samples_leaf,
            class_weight=self.config.class_weight,
            random_state=self.config.random_state,
            n_jobs=-1,
        )
        # Isotonic calibration on internal CV folds: turns raw forest vote
```

```
        # fractions into probabilities the business can multiply by rupee values.
        classifier = CalibratedClassifierCV(forest, method="isotonic", cv=3)
        pipeline = Pipeline(
            [
                ("features", ChurnFeatureEngineer()),
                ("preprocess", build_preprocessor()),
                ("classifier", classifier),
            ]
        )
        logger.info("Pipeline built with steps: %s", [name for name, _ in pipeline.steps])
        return pipeline
```

The single most important line in this codebase

`Pipeline([("features", ...), ("preprocess", ...), ("classifier", ...)])` — because the *whole pipeline* is what gets serialised, the transformation applied at inference is not merely equivalent to the one applied during training; it is the identical fitted object. Train/serve skew is eliminated by construction rather than by discipline. The research notebook in Section 4 saves only the bare estimator, and Section 4.4 traces exactly how that leads to divergence.

`ChurnPredictor` encapsulates the artefact and exposes a narrow interface. Its state — loaded or not — is explicit via `is_loaded`, which the API's readiness probe reads directly.

`src/churnguard/models/predictor.py — inference interface`

```
    def predict_proba(self, frame: pd.DataFrame) -> np.ndarray:
        """Return P(churn) for each row. Raises ``InferenceError`` on bad input."""
        pipeline = self._require_model()
        if frame.empty:
            raise InferenceError("Received an empty batch")
        missing = [c for c in SETTINGS.all_features if c not in frame.columns]
        if missing:
            logger.error("Inference payload missing features: %s", missing)
            raise InferenceError(f"Missing required features: {missing}")
        try:
            probabilities = pipeline.predict_proba(frame[SETTINGS.all_features])[:, 1]
        except Exception as exc:  # noqa: BLE001 - surfaced to the API as 422
            logger.exception("Inference failed for a batch of %d rows", len(frame))
            raise InferenceError(f"Inference failed: {exc}") from exc

        if not np.all((probabilities >= 0.0) & (probabilities <= 1.0)):
            logger.error("Model returned out-of-range probabilities -- refusing to serve")
            raise InferenceError("Model produced probabilities outside [0, 1]")
        return probabilities
```

Note the final guard: even after a successful `predict_proba` call, the output is checked to lie in [0, 1]. A model that returns 1.4 indicates a corrupted artefact or a version mismatch, and it is far better to fail loudly than to hand a nonsensical probability to a system that will multiply it by a rupee value.

3.6 Cross-cutting: immutable configuration

Every tunable value lives in `config.py` as a frozen dataclass. Freezing matters: a mutable module-level dict can be silently modified by any importing module, producing behaviour that depends on import order. Freezing makes that a runtime error instead of a mystery.

`src/churnguard/config.py — complete`

```
        return default


@dataclass(frozen=True)
class Paths:
    """Filesystem layout. All paths are absolute and derived from PROJECT_ROOT."""

    root: Path = PROJECT_ROOT
    raw_data: Path = PROJECT_ROOT / "data" / "raw" / "telco_churn.csv"
    reference_profile: Path = PROJECT_ROOT / "artifacts" / "reference_profile.json"
    model_artifact: Path = PROJECT_ROOT / "artifacts" / "churn_model.joblib"
    metrics_report: Path = PROJECT_ROOT / "reports" / "model_metrics.json"
    log_file: Path = PROJECT_ROOT / "logs" / "churnguard.log"


@dataclass(frozen=True)
class ModelConfig:
    """Hyper-parameters and the target/feature contract."""

    target: str = "churn"
    id_column: str = "customer_id"
    numeric_features: tuple[str, ...] = (
        "tenure_months",
        "monthly_charges",
        "total_charges",
        "support_tickets_6m",
        "avg_monthly_gb",
    )
    categorical_features: tuple[str, ...] = (
        "contract_type",
        "internet_service",
        "payment_method",
        "tech_support",
        "paperless_billing",
    )
    test_size: float = 0.2
    random_state: int = 42
    n_estimators: int = 300
    max_depth: int = 12
    min_samples_leaf: int = 8
    class_weight: str = "balanced"
    #: Tuned on the validation split (see reports/threshold_sweep.csv). 0.50 is NOT
    #: the right default here: losing a customer costs ~18x the price of a retention
    #: discount, so the threshold is deliberately pushed down to buy recall. F1 peaks
    #: at 0.34-0.35 on held-out data.
    decision_threshold: float = _env_float("CHURN_DECISION_THRESHOLD", 0.35)


@dataclass(frozen=True)
class QualityGates:
    """Thresholds a model or dataset must clear to be considered acceptable."""

    min_roc_auc: float = 0.75
    min_f1: float = 0.55
    max_brier: float = 0.20
    max_missing_fraction: float = 0.05
    max_psi: float = 0.20


@dataclass(frozen=True)
class Settings:
    """Top-level settings object consumed by every module."""
```

```
    paths: Paths = field(default_factory=Paths)
    model: ModelConfig = field(default_factory=ModelConfig)
    gates: QualityGates = field(default_factory=QualityGates)
    log_level: str = _env("CHURN_LOG_LEVEL", "INFO")
    api_version: str = "v1"

    @property
    def all_features(self) -> list[str]:
        return list(self.model.numeric_features) + list(self.model.categorical_features)


SETTINGS = Settings()
```

Two details are worth highlighting. First, `QualityGates` puts the release criteria **in version control next to the code**, so tightening a gate is a reviewable diff rather than a conversation. Second, the decision threshold carries a comment explaining *why* it is 0.35 and not 0.50 — the asymmetric cost of a false negative. Section 10.4 shows the sweep that produced that number.

4. Task 2 — Research Code vs Production Code

Owner: **Member 1** — authored the exploratory notebook, performed the refactoring into `src/churnguard`, and produced the defect-by-defect comparison below.

Requirement. *Clearly distinguish and demonstrate the difference between research code vs production code for at least one component (e.g., a Jupyter notebook prototype vs. a modularized, production-style script/package).*

The component chosen for this comparison is the **complete data-to-model path**: loading the CSV, handling nulls, encoding categoricals, splitting, fitting and saving. It is the right choice because it is where research and production practice diverge most sharply, and because every defect in the notebook version has a concrete, demonstrable consequence.

4.1 The research artefact

`notebooks/01_research_prototype.ipynb` is preserved in the repository exactly as it was written — cells run out of order, dead ends left in place, notes to self at the bottom. It is deliberately *not* cleaned up, because a tidied notebook would misrepresent what research code actually looks like and would defeat the purpose of the comparison.

Research code optimises for **speed of learning**. Its author is the only user, the feedback loop is seconds long, and the code is expected to be thrown away. Judged against that goal, the notebook below is not bad code — it is appropriate code that becomes dangerous only when someone tries to deploy it.

`notebooks/01_research_prototype.ipynb` — the substantive cells

```
# --- Cell 1 -------------------------------------------------------------
import pandas as pd, numpy as np, matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score, accuracy_score
%matplotlib inline

# --- Cell 2 -------------------------------------------------------------
df = pd.read_csv("../data/raw/telco_churn.csv")   # hard-coded relative path
df.head()

# --- Cell 3 -------------------------------------------------------------
df.shape, df.churn.mean()

# --- Cell 4 -------------------------------------------------------------
df.isnull().sum()

# --- Cell 6 -------------------------------------------------------------
# just fill the nulls, figure out something better later
df = df.fillna(0)

# --- Cell 7 -------------------------------------------------------------
# one hot everything
X = pd.get_dummies(df.drop(['churn','customer_id'], axis=1))
y = df.churn

# --- Cell 8 -------------------------------------------------------------
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)   # no random_state!

# --- Cell 9 -----------------------------------------------------------
m = RandomForestClassifier(n_estimators=100)
m.fit(X_train, y_train)
p = m.predict_proba(X_test)[:,1]
print(accuracy_score(y_test, m.predict(X_test)))
print(roc_auc_score(y_test, p))

# --- Cell 10 ----------------------------------------------------------
# try adding a feature
X_train['spm'] = X_train.total_charges / X_train.tenure_months
X_test['spm']  = X_test.total_charges / X_test.tenure_months
m.fit(X_train, y_train)
roc_auc_score(y_test, m.predict_proba(X_test)[:,1])

# --- Cell 11 ----------------------------------------------------------
# hmm worse. try again with different depth
m2 = RandomForestClassifier(n_estimators=100, max_depth=8)
m2.fit(X_train, y_train)
roc_auc_score(y_test, m2.predict_proba(X_test)[:,1])

# --- Cell 12 ----------------------------------------------------------
import pickle
pickle.dump(m2, open('model.pkl','wb'))   # saves the bare estimator, not the transforms
```

4.2 Defect-by-defect analysis

Each row below identifies a specific line of the notebook, states the concrete failure it causes in production, and names the production module that fixes it. This is the substance of the comparison.

#	Research code	Concrete production failure	Production remedy
1	`pd.read_csv("../data/raw/…")`	Breaks the moment the working directory differs — which it does in CI, in Docker and in Airflow. Fails with a bare `FileNotFoundError` and no context.	`CsvDataSource` takes an injected path derived from `PROJECT_ROOT`; a missing file raises `DataIngestionError` naming the path.
2	`df.fillna(0)`	Silently imputes zero into `total_charges`, teaching the model that a null lifetime spend means a free customer. Corrupts the model rather than crashing.	`SimpleImputer(strategy="median")` **fitted on training data only**, carried inside the pipeline so the same statistic is applied at serving time.
3	`pd.get_dummies(...)`	Column set depends on the categories *present in that batch*. A batch with no "DSL" customers produces a matrix with a missing column and the model silently misaligns features.	`OneHotEncoder(handle_unknown="ignore")` fitted once; the category set is frozen in the artefact, so the matrix shape is invariant.
4	`train_test_split(...)` with no `random_state`	Every run reports a different score. Two experiments are not comparable, and a regression cannot be distinguished from noise.	`random_state=42` in `ModelConfig`, asserted by `test_split_is_reproducible`.
5	Feature `spm` computed after the split on both frames	Works here, but the pattern invites fitting a scaler or imputer on the full dataset — the classic leakage bug — and must be re-implemented by hand at serving time.	`ChurnFeatureEngineer` is a pipeline step, so it is fitted on training folds only and travels with the model.

#	Research code	Concrete production failure	Production remedy
6	`pickle.dump(m2, …)`	Serialises the **bare estimator**. Whoever serves it must re-implement `fillna`, `get_dummies` and the `spm` formula identically. Any divergence is train/serve skew — silent, and invisible in offline metrics.	`joblib.dump({"pipeline": …, "metrics": …, "metadata": …})` persists the entire fitted pipeline plus provenance.
7	No error handling anywhere	A malformed CSV surfaces as a pandas stack trace inside a scheduled job at 03:00 with no indication of which upstream feed broke.	Typed exception hierarchy; every failure names the offending source, column or record.
8	`print()` for output	Nothing is captured once the notebook is closed. No audit trail of what ran, on what data, with what result.	Structured logging to console *and* a rotating file, with levels, module names and line numbers.
9	No validation of the input data	An upstream unit change from dollars to cents would be absorbed silently and degrade the model.	`DataValidator` enforces schema, ranges, missingness and drift; `train.py` exits 1 on a breach.
10	No tests	Any change is a gamble. Refactoring is impossible because nothing can confirm behaviour is preserved.	98 tests across four categories, 90% coverage, run on every push.
11	Accuracy as the headline metric	26.7% churn means predicting "nobody churns" scores 73% accuracy. The metric hides a useless model.	F1, ROC-AUC, Brier and ECE, with explicit release gates on three of them.
12	Cells run out of order	The notebook's final state may be unreproducible even by its author; `df` has been reassigned in place.	A linear, idempotent `scripts/train.py` that produces identical output on every run.

4.3 The same operation, side by side

The contrast is sharpest on the single most consequential line — persistence.

RESEARCH — notebooks/01_research_prototype.ipynb, cell 12

```
import pickle
pickle.dump(m2, open('model.pkl','wb'))
```

PRODUCTION — src/churnguard/models/trainer.py

```
    def save(self, path: Path, metrics: ModelMetrics, extra: dict | None = None) -> Path:
        if self.pipeline is None:
            raise ModelTrainingError("save() called before train()")
        path.parent.mkdir(parents=True, exist_ok=True)
        payload = {
            "pipeline": self.pipeline,
            "metrics": metrics.to_dict(),
            "metadata": {
                "code_version": __version__,
                "trained_at_utc": datetime.now(timezone.utc).isoformat(timespec="seconds"),
                "sklearn_pipeline_steps": [name for name, _ in self.pipeline.steps],
                "input_features": SETTINGS.all_features,
                "decision_threshold": self.config.decision_threshold,
                "hyperparameters": {
                    "n_estimators": self.config.n_estimators,
```

```
                "max_depth": self.config.max_depth,
                "min_samples_leaf": self.config.min_samples_leaf,
                "class_weight": self.config.class_weight,
                "calibration": "isotonic",
            },
            **(extra or {}),
        },
    }
    joblib.dump(payload, path)
    logger.info("Artefact saved to %s (%.1f KB)", path, path.stat().st_size / 1024)
    return path
```

The production version is roughly fifteen times longer, and every additional line answers a question the research version cannot: *Which code version produced this? When? On how much data? With what hyper-parameters? What did it score? Is the file even writable?* Those answers are what make the `/v1/model/info` endpoint, the CI artefact upload and any post-incident audit possible.

4.4 Tracing one defect all the way to a production incident

Defect #3 deserves the full walkthrough, because it is the failure mode that is hardest to detect and does the most damage.

13. The notebook calls `pd.get_dummies` on the *entire* dataset. Because all three `internet_service` values are present, it produces three columns: `internet_service_DSL`, `internet_service_Fiber optic`, `internet_service_No`.
14. The model is fitted on that 24-column matrix and pickled.
15. In production, a batch of 40 customers from a rural region arrives. None of them has fibre.
16. The serving code re-runs `pd.get_dummies` on that batch. It produces **two** columns, not three.
17. Depending on the serving implementation, this either raises a shape error (the good outcome) or — if the code reindexes loosely — silently shifts every downstream column, so `payment_method_Credit card` is read into the slot the model learned as `internet_service_No`.
18. The model returns confident, plausible-looking probabilities that are **entirely meaningless**. Offline metrics still look fine because they were computed on the training distribution. Nobody notices until the retention campaign underperforms for a quarter.

The production system makes this impossible in two independent ways: the encoder's category set is frozen at fit time inside the artefact, and `test_unseen_category_degrades_gracefully` (Section 9.5) asserts that an unrecognised level produces a valid probability rather than a crash or a shift.

> **The distinction in one sentence**
>
> Research code optimises for the **speed of the next insight**; production code optimises for the **cost of the next change** — and the transition between them is not a cleanup pass, it is a redesign in which roughly every line acquires an error path, a test and a provenance record.

5. Task 3 — Error Handling and Logging

Owner: **Member 2** — Reliability & Code Quality. Designed the exception hierarchy and instrumented every layer of the system.

Requirement. *Implement proper error handling and logging (e.g., using Python's logging module) across at least 3 critical functions/modules, with meaningful log levels (INFO, WARNING, ERROR).*

The requirement asks for three modules. This system instruments **six**: data ingestion, data validation, feature engineering, model training, inference, and the API. The four discussed in depth below are the ones where the design decisions are most instructive.

5.1 The philosophy: fail loudly, fail specifically, fail once

Three rules govern error handling throughout ChurnGuard.

- **Never catch bare `Exception` and continue.** A swallowed exception in an ML pipeline does not stop the pipeline — it produces a *quietly wrong model*, which is far worse than a crash because it is discovered months later.
- **Translate low-level errors into domain errors at the boundary.** The trainer should not have to know that a `UnicodeDecodeError` is possible; it should see `DataIngestionError`. This is what keeps the layers decoupled.
- **Log at the point of detection, raise to the point of decision.** The module that discovers a problem knows the most about it and logs the detail; the caller decides whether the problem is fatal.

5.2 The exception hierarchy

A single root exception means the API can catch everything this system raises without also catching genuine programming bugs or `KeyboardInterrupt` — a distinction that matters enormously when debugging a live service.

src/churnguard/exceptions.py — complete

```
from __future__ import annotations


class ChurnGuardError(Exception):
    """Base class for every error raised by ChurnGuard."""


class DataIngestionError(ChurnGuardError):
    """Raised when a data source cannot be read or is structurally unusable."""


class SchemaValidationError(ChurnGuardError):
    """Raised when an incoming dataset violates the expected contract."""

    def __init__(self, message: str, violations: list[str] | None = None) -> None:
        super().__init__(message)
        self.violations: list[str] = violations or []


class FeatureEngineeringError(ChurnGuardError):
    """Raised when feature construction fails (e.g. unexpected column types)."""
```

```
class ModelTrainingError(ChurnGuardError):
    """Raised when training cannot complete (e.g. single-class target)."""


class ModelNotLoadedError(ChurnGuardError):
    """Raised when inference is attempted before an artefact has been loaded."""


class InferenceError(ChurnGuardError):
    """Raised when a prediction request cannot be served."""
```

`SchemaValidationError` carries a structured `violations` list rather than folding everything into a message string. That list is what the API returns to the client in its error envelope, so the caller learns *which* fields were wrong, not merely that something was.

5.3 Logging configuration

The configuration follows the standard library's intended design, which is frequently violated in ML codebases: **libraries never configure logging; only entry points do.** Every module under `src/churnguard` calls `logging.getLogger(__name__)` and nothing else. Handlers are attached exactly once, by `scripts/train.py`, by the API's lifespan handler, or by the pytest fixture.

`src/churnguard/logging_config.py — complete`

```
import logging
import logging.handlers
from pathlib import Path

from churnguard.config import SETTINGS

_CONSOLE_FORMAT = "%(asctime)s | %(levelname)-8s | %(name)-38s | %(message)s"
_FILE_FORMAT = "%(asctime)s | %(levelname)-8s | %(name)s | %(funcName)s:%(lineno)d | %(message)s"
_configured = False


def configure_logging(
    level: str | None = None,
    log_file: Path | None = None,
    force: bool = False,
) -> logging.Logger:
    """Attach console and rotating-file handlers to the root logger.

    Idempotent: calling it twice does not duplicate log lines, which matters
    because uvicorn imports the app module more than once with ``--reload``.
    """
    global _configured
    root = logging.getLogger()
    if _configured and not force:
        return root
    if force:
        for handler in list(root.handlers):
            root.removeHandler(handler)

    root.setLevel(getattr(logging, (level or SETTINGS.log_level).upper(), logging.INFO))

    console = logging.StreamHandler()
```

```
    console.setFormatter(logging.Formatter(_CONSOLE_FORMAT, datefmt="%H:%M:%S"))
    root.addHandler(console)

    target = log_file or SETTINGS.paths.log_file
    try:
        target.parent.mkdir(parents=True, exist_ok=True)
        file_handler = logging.handlers.RotatingFileHandler(
            target, maxBytes=2_000_000, backupCount=3, encoding="utf-8"
        )
        file_handler.setLevel(logging.DEBUG)
        file_handler.setFormatter(logging.Formatter(_FILE_FORMAT))
        root.addHandler(file_handler)
    except OSError as exc:
        # A read-only filesystem must degrade to console-only, never crash.
        root.warning("File logging disabled (%s): %s", target, exc)

    logging.getLogger("churnguard").debug("Logging configured at level %s", root.level)
    _configured = True
    return root
```

Three details are deliberate. The `_configured` flag makes the function **idempotent**, which matters because `uvicorn --reload` imports the app module more than once and would otherwise duplicate every log line. The file handler is a `RotatingFileHandler` capped at 2 MB × 3 backups, so a runaway loop cannot fill the disk. And a read-only filesystem — common in hardened containers — degrades to console-only with a warning instead of crashing the service at startup.

Level	Meaning in this system	Concrete examples
DEBUG	Detail useful only when diagnosing a specific problem. File handler only.	Which derived features were computed; the numeric log level after configuration.
INFO	The system is doing what it should. This is the audit trail.	Data loaded (rows × columns); split summary with class balance; pipeline steps; CV result; evaluation metrics; artefact saved with size; every HTTP request with status and latency.
WARNING	Something is wrong but the system can proceed on a defensible default. **Someone should look at this eventually.**	Duplicate customer IDs dropped; a column has nulls below the failure threshold; PSI between 0.10 and 0.20; stratification disabled; file logging unavailable; a client sent an invalid request (400).
ERROR	The operation cannot complete. **Someone must act.**	Data file missing or unparseable; required column absent; a data-quality gate breached; single-class target; model artefact missing or corrupt; probabilities outside [0,1]; a 503 served.
CRITICAL	Reserved, deliberately unused.	Nothing in this system is unrecoverable at the process level; using CRITICAL for merely serious errors devalues it for on-call paging.

5.4 Instrumented module 1 — data ingestion

`CsvDataSource.load` enumerates the four ways reading a CSV realistically fails and gives each one a specific, actionable message. Note that `pd.errors.EmptyDataError` and a successfully-parsed-but-zero-row file are *different* failures with different upstream causes, and are handled separately.

`src/churnguard/data/ingestion.py — error handling in load()`

```
    def load(self) -> pd.DataFrame:
        """Return the raw dataset, or raise ``DataIngestionError``."""
```

`DataIngestor.split` shows the WARNING-versus-ERROR judgement clearly: duplicate IDs are a recoverable data-hygiene issue (warn, drop, continue), whereas a single-class target makes training meaningless (log ERROR, raise).

src/churnguard/data/ingestion.py — level selection in split()

```
        if target.nunique() < 2:
            logger.error("Target '%s' has a single class -- cannot train", self.config.target)
            raise DataIngestionError("Target column must contain at least two classes")

        minority = int(target.value_counts().min())
        stratify = target if minority >= 2 else None
        if stratify is None:
            logger.warning("Minority class has %d row(s); stratification disabled", minority)
```

5.5 Instrumented module 2 — data validation

The `ValidationReport` object logs as a side effect of recording, so a violation cannot be added to the report without also being logged. The `DATA-QUALITY` prefix makes the whole class of events greppable in aggregated logs.

src/churnguard/data/validation.py — logging built into the report

```
    def add_error(self, message: str) -> None:
        self.passed = False
        self.errors.append(message)
        logger.error("DATA-QUALITY FAIL | %s", message)

    def add_warning(self, message: str) -> None:
        self.warnings.append(message)
        logger.warning("DATA-QUALITY WARN | %s", message)
```

5.6 Instrumented module 3 — model training

Training uses `logger.exception` rather than `logger.error`, which additionally captures the full traceback — essential when a fit fails deep inside scikit-learn. The original exception is chained with `from exc` so no diagnostic information is lost while the caller still sees a clean domain error.

src/churnguard/models/trainer.py — train()

```
    def train(self, x_train: pd.DataFrame, y_train: pd.Series) -> Pipeline:
        if len(x_train) != len(y_train):
            raise ModelTrainingError(
                f"X has {len(x_train)} rows but y has {len(y_train)} -- refusing to train"
            )
        if pd.Series(y_train).nunique() < 2:
            logger.error("Training target contains a single class")
            raise ModelTrainingError("Training requires at least two target classes")

        logger.info("Training on %d rows (churn rate %.3f)", len(x_train), float(y_train.mean()))
        self.pipeline = self.build_pipeline()
        try:
            self.pipeline.fit(x_train, y_train)
        except Exception as exc:  # noqa: BLE001 - re-raised as a domain error
```

```
            logger.exception("Training failed")
            raise ModelTrainingError(f"Model training failed: {exc}") from exc
        logger.info("Training complete")
        return self.pipeline
```

5.7 Instrumented module 4 — inference and the API

At the serving boundary, errors must additionally be *translated into HTTP semantics*. Each domain exception maps to exactly one status code through a registered handler, so the mapping is declared in one place rather than scattered through the route functions.

src/churnguard/api/main.py — exception handlers

```
    return JSONResponse(
        status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
        content=ErrorResponse(
            error="model_unavailable",
            detail="The model artefact is not loaded. Retry after the service is ready.",
        ).model_dump(),
    )


@app.exception_handler(SchemaValidationError)
async def handle_schema_error(request: Request, exc: SchemaValidationError) -> JSONResponse:
    logger.warning("400 on %s -- contract breach: %s", request.url.path, exc)
    return JSONResponse(
        status_code=status.HTTP_400_BAD_REQUEST,
        content=ErrorResponse(
            error="data_contract_violation", detail=str(exc), violations=exc.violations
        ).model_dump(),
    )


@app.exception_handler(InferenceError)
async def handle_inference_error(request: Request, exc: InferenceError) -> JSONResponse:
    logger.warning("400 on %s -- inference rejected: %s", request.url.path, exc)
    return JSONResponse(
        status_code=status.HTTP_400_BAD_REQUEST,
        content=ErrorResponse(error="inference_failed", detail=str(exc)).model_dump(),
    )


@app.exception_handler(ChurnGuardError)
async def handle_generic_domain_error(request: Request, exc: ChurnGuardError) -> JSONResponse:
    logger.exception("500 on %s", request.url.path)
    return JSONResponse(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        content=ErrorResponse(
            error="internal_error", detail="An unexpected internal error occurred."
        ).model_dump(),
    )


# ---------------------------------------------------------------- routes
@app.get("/health", response_model=HealthResponse, tags=["operations"])
async def health() -> HealthResponse:
    """Liveness + readiness. Always 200 so the probe can read the body."""
    return HealthResponse(
```

5.8 Evidence — real console output

The following is the unedited console capture from `python scripts/train.py --cv 5`, showing INFO, WARNING and ERROR levels occurring naturally in a single run. This is `reports/train_console_FAILED_GATE.log`, retained deliberately because it is the run in which the model **failed its own quality gate**.

`$ python scripts/train.py --cv 5  (run 1 — gate failure)`

```
16:15:10 | INFO     | churnguard.train                      |
================================================================================
16:15:10 | INFO     | churnguard.train                      | ChurnGuard training run starting
16:15:10 | INFO     | churnguard.train                      |
================================================================================
16:15:10 | INFO     | churnguard.data.ingestion             | Loading data from
csv:///home/claude/churnguard/data/raw/telco_churn.csv
16:15:10 | INFO     | churnguard.data.ingestion             | Loaded 5000 rows x 12 columns
16:15:10 | INFO     | churnguard.data.validation            | Validating dataset: 5000 rows x 12 columns
16:15:10 | WARNING  | churnguard.data.validation            | DATA-QUALITY WARN | Column 'total_charges' has 1.20% nulls
(imputed)
16:15:10 | WARNING  | churnguard.data.validation            | DATA-QUALITY WARN | Column 'avg_monthly_gb' has 0.60% nulls
(imputed)
16:15:10 | INFO     | churnguard.data.validation            | Validation PASSED | 0 error(s), 2 warning(s)
16:15:10 | INFO     | churnguard.data.ingestion             | Split complete: {'n_train': 4000, 'n_test': 1000, 'n_features':
10, 'train_churn_rate': 0.2665, 'test_churn_rate': 0.267}
16:15:10 | INFO     | churnguard.models.trainer             | Pipeline built with steps: ['features', 'preprocess',
'classifier']
16:15:10 | INFO     | churnguard.features.engineering       | ChurnFeatureEngineer fitted: 10 in -> 15 out
16:15:13 | INFO     | churnguard.features.engineering       | ChurnFeatureEngineer fitted: 10 in -> 15 out
16:15:15 | INFO     | churnguard.features.engineering       | ChurnFeatureEngineer fitted: 10 in -> 15 out
16:15:18 | INFO     | churnguard.features.engineering       | ChurnFeatureEngineer fitted: 10 in -> 15 out
16:15:20 | INFO     | churnguard.features.engineering       | ChurnFeatureEngineer fitted: 10 in -> 15 out
16:15:23 | INFO     | churnguard.models.trainer             | Cross-validation ROC-AUC = 0.7921 (+/- 0.0144) over 5 folds
16:15:23 | INFO     | churnguard.models.trainer             | Training on 4000 rows (churn rate 0.267)
16:15:23 | INFO     | churnguard.models.trainer             | Pipeline built with steps: ['features', 'preprocess',
'classifier']
16:15:23 | INFO     | churnguard.features.engineering       | ChurnFeatureEngineer fitted: 10 in -> 15 out
16:15:25 | INFO     | churnguard.models.trainer             | Training complete
16:15:25 | INFO     | churnguard.models.evaluation          | Evaluation @thr=0.45 | AUC=0.7882 F1=0.5478 acc=0.7920
Brier=0.1521 ECE=0.0208
16:15:25 | ERROR    | churnguard.train                      | Model failed quality gates: ['f1 0.548 < 0.55']
```

> **Read the last two lines**
>
> `WARNING` fires twice for columns with nulls below the failure threshold — the system proceeds, but records the concern. `ERROR` then fires because the trained model scored F1 = 0.548 against a gate of 0.55, and `train.py` exited with status 1 **without saving the artefact**.
>
> This is not a contrived demonstration. It is what actually happened on the first full training run, and it is the clearest possible evidence that the gates are enforced rather than decorative. Section 10.4 shows how the threshold was then tuned — with justification — to produce a model that passes.

The successful run after threshold tuning:

`$ python scripts/train.py --cv 5  (run 2 — all gates passed)`

```
16:16:02 | INFO     | churnguard.features.engineering       | ChurnFeatureEngineer fitted: 10 in -> 15 out
```

```
16:16:05 | INFO     | churnguard.models.trainer              | Cross-validation ROC-AUC = 0.7921 (+/- 0.0144) over 5 folds
16:16:05 | INFO     | churnguard.models.trainer              | Training on 4000 rows (churn rate 0.267)
16:16:05 | INFO     | churnguard.models.trainer              | Pipeline built with steps: ['features', 'preprocess',
'classifier']
16:16:05 | INFO     | churnguard.features.engineering        | ChurnFeatureEngineer fitted: 10 in -> 15 out
16:16:07 | INFO     | churnguard.models.trainer              | Training complete
16:16:07 | INFO     | churnguard.models.evaluation           | Evaluation @thr=0.35 | AUC=0.7882 F1=0.5652 acc=0.7600
Brier=0.1521 ECE=0.0208
16:16:07 | INFO     | churnguard.train                       | All model quality gates passed
16:16:07 | INFO     | churnguard.data.validation             | Reference profile with 5 features written to
/home/claude/churnguard/artifacts/reference_profile.json
16:16:08 | INFO     | churnguard.models.trainer              | Artefact saved to
/home/claude/churnguard/artifacts/churn_model.joblib (19280.8 KB)
16:16:08 | INFO     | churnguard.models.trainer              | Metrics report written to
/home/claude/churnguard/reports/model_metrics.json
16:16:08 | INFO     | churnguard.train                       |
--------------------------------------------------------------------------------
16:16:08 | INFO     | churnguard.train                       | DONE | ROC-AUC=0.7882  F1=0.5652  Accuracy=0.7600  Brier=0.1521
ECE=0.0208
16:16:08 | INFO     | churnguard.train                       |
--------------------------------------------------------------------------------
```

And the structured file log, which carries function names and line numbers that the console format omits:

`logs/churnguard.log — rotating file handler format`

```
2026-08-05 16:25:28,173 | WARNING  | churnguard.data.validation | add_warning:62 | DATA-QUALITY WARN | Column 'avg_monthly_gb'
has 0.60% nulls (imputed)
2026-08-05 16:25:28,177 | ERROR    | churnguard.data.validation | add_error:58 | DATA-QUALITY FAIL | Feature 'tenure_months'
drifted: PSI=9.029 > 0.20
2026-08-05 16:25:28,190 | WARNING  | churnguard.data.validation | add_warning:62 | DATA-QUALITY WARN | Column 'total_charges'
has 0.96% nulls (imputed)
2026-08-05 16:25:28,190 | WARNING  | churnguard.data.validation | add_warning:62 | DATA-QUALITY WARN | Column 'avg_monthly_gb'
has 0.72% nulls (imputed)
2026-08-05 16:25:28,198 | ERROR    | churnguard.data.validation | add_error:58 | DATA-QUALITY FAIL | boom
2026-08-05 16:25:28,239 | ERROR    | churnguard.features.engineering | apply_derived_features:86 | Cannot compute
'avg_spend_per_month': missing input column 'total_charges'
2026-08-05 16:25:34,120 | ERROR    | churnguard.models.predictor | predict_proba:116 | Inference payload missing features:
['contract_type']
2026-08-05 16:25:34,397 | ERROR    | churnguard.models.trainer | train:81 | Training target contains a single class
```

6. Task 4 — Code Formatting and Linting

Owner: **Member 2** — configured `black`, `isort` and `flake8`, ran them across the codebase, and produced the before/after evidence below.

Requirement. *Apply code formatting and linting tools (e.g., black, flake8/pylint, isort) to your codebase; include before/after screenshots or lint reports.*

6.1 Why three tools and not one

The three tools address genuinely different problems and are not substitutes for one another.

Tool	Category	What it does	What it cannot do
isort	Formatter	Groups imports into standard-library / third-party / first-party blocks and sorts within each block.	Cannot tell whether an import is used.
black	Formatter	Rewrites the entire file to a single canonical layout: line length, quotes, spacing, trailing commas, wrapping.	Deliberately makes no semantic judgements — it will happily format unreachable code.
flake8	Linter	Static analysis: unused imports and variables, undefined names, comparison anti-patterns, cyclomatic complexity.	Reports problems but fixes nothing; some findings need human judgement.

The value of black is not that its output is beautiful — it is that formatting **disappears from code review entirely**. No reviewer argues about line breaks when the tool decides them. `isort` is configured with `profile = "black"` specifically so the two tools agree; without it they fight, each undoing the other's work on every commit.

6.2 Configuration

`pyproject.toml`

```
[project]
name = "churnguard"
version = "1.0.0"
description = "Production-style ML system for telecom customer churn prediction"
requires-python = ">=3.10"

[tool.black]
line-length = 100
target-version = ["py310", "py311", "py312"]

[tool.isort]
profile = "black"          # makes isort's output agree with black's
line_length = 100
known_first_party = ["churnguard"]
src_paths = ["src", "tests", "scripts"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src", "scripts"]
```

setup.cfg

```
[flake8]
max-line-length = 100
# E203 and W503 conflict with black's (PEP8-compliant) slice and line-break style.
extend-ignore = E203, W503
max-complexity = 10
exclude = .git,__pycache__,.venv,build,dist,reports/lint_demo
per-file-ignores =
    # Test files legitimately import fixtures that look unused to a linter.
    tests/*: F811
    # Entry-point scripts must alter sys.path before importing the package.
    scripts/*: E402
    tests/conftest.py: E402
```

The `extend-ignore = E203, W503` line is necessary, not lazy: those two checks contradict black's (PEP 8-compliant) handling of slices and binary-operator line breaks. Leaving them enabled would mean every black-formatted file fails flake8. The `per-file-ignores` entries are similarly principled — entry-point scripts genuinely must modify `sys.path` before importing the package, so `E402` is expected there and nowhere else.

6.3 Before / after on a representative module

To make the improvement measurable, a scoring-utility module was written in the unformatted style typical of code extracted straight from a notebook. It is preserved at `reports/lint_demo/scoring_utils_BEFORE.py`.

BEFORE — reports/lint_demo/scoring_utils_BEFORE.py

```
import json,os
from sklearn.metrics import roc_auc_score, f1_score
import pandas as pd
import numpy as np
from churnguard.config import SETTINGS
import sys

UNUSED_CONSTANT = "this is never referenced anywhere"

def score_customers( frame,model,threshold = 0.5 ,verbose=False ):
    '''scores customers'''
    results = [ ]
    for i in range( len(frame) ):
        row=frame.iloc[[i]]
        p = model.predict_proba( row )[0][1]
        if p>=threshold :
            lab=1
        else :
            lab = 0
        if verbose == True:
            print( "customer",i,"prob",p )
        results.append({"idx":i,"prob":p,"label":lab})
    return results

def summarise(results,y_true = None):
    probs=np.array([r["prob"] for r in results]);labs=np.array([r["label"] for r in results])
    out = {"n":len(results),"flagged":int(labs.sum()),"mean_prob":float(probs.mean())}
    if y_true is not None :
        out["auc"]=roc_auc_score(y_true,probs)
        out["f1"] = f1_score( y_true,labs )
```

```
    return out
```

6.3.1 Lint report — BEFORE

`$ flake8 --max-line-length=100 scoring_utils_BEFORE.py`

```
scoring_utils_BEFORE.py:1:1: F401 'json' imported but unused
scoring_utils_BEFORE.py:1:1: F401 'os' imported but unused
scoring_utils_BEFORE.py:1:12: E401 multiple imports on one line
scoring_utils_BEFORE.py:1:12: E231 missing whitespace after ','
scoring_utils_BEFORE.py:3:1: F401 'pandas as pd' imported but unused
scoring_utils_BEFORE.py:5:1: F401 'churnguard.config.SETTINGS' imported but unused
scoring_utils_BEFORE.py:6:1: F401 'sys' imported but unused
scoring_utils_BEFORE.py:10:1: E302 expected 2 blank lines, found 1
scoring_utils_BEFORE.py:10:21: E201 whitespace after '('
scoring_utils_BEFORE.py:10:27: E231 missing whitespace after ','
scoring_utils_BEFORE.py:10:33: E231 missing whitespace after ','
scoring_utils_BEFORE.py:10:43: E251 unexpected spaces around keyword / parameter equals
scoring_utils_BEFORE.py:10:45: E251 unexpected spaces around keyword / parameter equals
scoring_utils_BEFORE.py:10:49: E203 whitespace before ','
scoring_utils_BEFORE.py:10:50: E231 missing whitespace after ','
scoring_utils_BEFORE.py:10:64: E202 whitespace before ')'
scoring_utils_BEFORE.py:12:16: E201 whitespace after '['
scoring_utils_BEFORE.py:13:20: E201 whitespace after '('
scoring_utils_BEFORE.py:13:31: E202 whitespace before ')'
scoring_utils_BEFORE.py:14:12: E225 missing whitespace around operator
scoring_utils_BEFORE.py:15:33: E201 whitespace after '('
scoring_utils_BEFORE.py:15:37: E202 whitespace before ')'
scoring_utils_BEFORE.py:16:13: E225 missing whitespace around operator
scoring_utils_BEFORE.py:16:24: E203 whitespace before ':'
scoring_utils_BEFORE.py:17:16: E225 missing whitespace around operator
scoring_utils_BEFORE.py:18:13: E203 whitespace before ':'
scoring_utils_BEFORE.py:20:20: E712 comparison to True should be 'if cond is True:' or 'if cond:'
scoring_utils_BEFORE.py:21:19: E201 whitespace after '('
scoring_utils_BEFORE.py:21:30: E231 missing whitespace after ','
scoring_utils_BEFORE.py:21:32: E231 missing whitespace after ','
scoring_utils_BEFORE.py:21:39: E231 missing whitespace after ','
scoring_utils_BEFORE.py:21:41: E202 whitespace before ')'
scoring_utils_BEFORE.py:22:30: E231 missing whitespace after ':'
scoring_utils_BEFORE.py:22:32: E231 missing whitespace after ','
scoring_utils_BEFORE.py:22:39: E231 missing whitespace after ':'
scoring_utils_BEFORE.py:22:41: E231 missing whitespace after ','
scoring_utils_BEFORE.py:22:49: E231 missing whitespace after ':'
scoring_utils_BEFORE.py:25:1: E302 expected 2 blank lines, found 1
scoring_utils_BEFORE.py:25:22: E231 missing whitespace after ','
scoring_utils_BEFORE.py:25:29: E251 unexpected spaces around keyword / parameter equals
scoring_utils_BEFORE.py:25:31: E251 unexpected spaces around keyword / parameter equals
scoring_utils_BEFORE.py:26:10: E225 missing whitespace around operator
scoring_utils_BEFORE.py:26:49: E702 multiple statements on one line (semicolon)
scoring_utils_BEFORE.py:26:49: E231 missing whitespace after ';'
scoring_utils_BEFORE.py:26:54: E225 missing whitespace around operator
scoring_utils_BEFORE.py:27:15: E231 missing whitespace after ':'
scoring_utils_BEFORE.py:27:28: E231 missing whitespace after ','
scoring_utils_BEFORE.py:27:38: E231 missing whitespace after ':'
scoring_utils_BEFORE.py:27:54: E231 missing whitespace after ','
scoring_utils_BEFORE.py:27:66: E231 missing whitespace after ':'
scoring_utils_BEFORE.py:28:26: E203 whitespace before ':'
scoring_utils_BEFORE.py:29:19: E225 missing whitespace around operator
scoring_utils_BEFORE.py:29:40: E231 missing whitespace after ','
scoring_utils_BEFORE.py:30:30: E201 whitespace after '('
scoring_utils_BEFORE.py:30:37: E231 missing whitespace after ','
scoring_utils_BEFORE.py:30:42: E202 whitespace before ')'
```

```
Total issues: 56
```

Result: 56 violations across 30 lines of code

Roughly two violations per line.

6.3.2 Applying the tools

`$ isort --profile black --line-length 100 scoring_utils.py && black --line-length 100 scoring_utils.py`

```
Fixing /home/claude/churnguard/reports/lint_demo/scoring_utils_AFTER.py
reformatted scoring_utils_AFTER.py
All done! 1 file reformatted.

$ flake8 --max-line-length=100 scoring_utils_AFTER.py
scoring_utils_AFTER.py:1:1: F401 'json' imported but unused
scoring_utils_AFTER.py:2:1: F401 'os' imported but unused
scoring_utils_AFTER.py:3:1: F401 'sys' imported but unused
scoring_utils_AFTER.py:6:1: F401 'pandas as pd' imported but unused
scoring_utils_AFTER.py:7:1: F401 'churnguard.config.SETTINGS' imported but unused
scoring_utils_AFTER.py:23:20: E712 comparison to True should be 'if cond is True:' or 'if cond:'
```

56 → 6 violations automatically. The fifty issues the formatters removed were all whitespace, spacing and layout. The six that survive are precisely the ones a formatter *must not* fix, because deleting an import or rewriting a comparison changes program semantics — that is a decision for a human.

6.3.3 After human review

The remaining six were resolved by the author. The unused imports were deleted, `verbose == True` was replaced with a truthiness check, and — because the review exposed it — the per-row Python loop was replaced with a single vectorised call.

`AFTER — reports/lint_demo/scoring_utils_FINAL.py`

```
"""Batch scoring helpers (post-review version).

Changes beyond what black/isort could do automatically:
  * removed five unused imports (flake8 F401) -- tools flag these, humans delete them;
  * replaced ``verbose == True`` with a truthiness check (flake8 E712);
  * replaced the per-row Python loop with a single vectorised ``predict_proba``
    call, which is both faster and the reason the loop-local variables vanished;
  * replaced ``print`` with module-level logging;
  * added type hints and a real docstring.
"""

from __future__ import annotations

import logging

import numpy as np
import pandas as pd
from sklearn.metrics import f1_score, roc_auc_score

logger = logging.getLogger(__name__)
```

```
def score_customers(
    frame: pd.DataFrame,
    model,
    threshold: float = 0.5,
) -> pd.DataFrame:
    """Score every row of ``frame`` and return probabilities plus binary labels."""
    probabilities = model.predict_proba(frame)[:, 1]
    logger.info("Scored %d customers at threshold %.2f", len(frame), threshold)
    return pd.DataFrame(
        {
            "idx": np.arange(len(frame)),
            "prob": probabilities,
            "label": (probabilities >= threshold).astype(int),
        }
    )


def summarise(results: pd.DataFrame, y_true: np.ndarray | None = None) -> dict[str, float]:
    """Roll a scored frame up into headline numbers."""
    summary: dict[str, float] = {
        "n": int(len(results)),
        "flagged": int(results["label"].sum()),
        "mean_prob": float(results["prob"].mean()),
    }
    if y_true is not None:
        summary["auc"] = float(roc_auc_score(y_true, results["prob"]))
        summary["f1"] = float(f1_score(y_true, results["label"]))
    return summary
```

`$ isort --check-only && black --check && flake8`

```
Skipped 0 files
All done! 1 file would be left unchanged.

$ flake8 --max-line-length=100 scoring_utils_FINAL.py
Issues remaining: 0
```

Stage	flake8 violations	Reduction	Fixed by
Original notebook-style code	**56**	—	—
After `isort` + `black`	**6**	−89%	Tooling, zero human effort
After human review	**0**	−100%	Judgement the tools cannot automate

Table 6.1 — Measured effect of the toolchain on a representative module.

6.4 The whole codebase

The same toolchain was then run across all 27 source, test and script files.

`$ make format && make lint`

```
============================================================
 STEP 1/3  isort  (import ordering)
============================================================
Fixing /home/claude/churnguard/tests/test_model_training.py
Fixing /home/claude/churnguard/tests/test_features.py
```

```
============================================================
 STEP 2/3  black  (formatting)
============================================================
reformatted /home/claude/churnguard/src/churnguard/data/ingestion.py
reformatted /home/claude/churnguard/src/churnguard/data/validation.py
reformatted /home/claude/churnguard/src/churnguard/models/evaluation.py
reformatted /home/claude/churnguard/tests/test_api_integration.py
reformatted /home/claude/churnguard/tests/test_model_inference.py
reformatted /home/claude/churnguard/tests/test_model_training.py

All done! ✨ 🍰 ✨
6 files reformatted, 19 files left unchanged.

============================================================
 STEP 3/3  flake8 (static analysis)
============================================================
flake8: no issues found.
```

`flake8` reports no issues across 2,815 lines. Critically, the full test suite was re-run immediately afterwards to confirm that reformatting six files changed no behaviour:

`$ pytest -q` (after reformatting)

```
tests/test_api_integration.py ...................                        [ 19%]
tests/test_data_ingestion.py .............                               [ 32%]
tests/test_data_validation.py ...............                            [ 47%]
tests/test_features.py .............                                     [ 61%]
tests/test_model_inference.py ..........................                 [ 88%]
tests/test_model_training.py ...........                                 [100%]

============================== 98 passed in 16.90s ==============================
```

6.5 Enforcement

A standard that is not enforced is a suggestion. The CI pipeline (Appendix C) runs the **check-only** variants, so a badly formatted commit fails the build rather than being silently reformatted:

```
- name: Lint (isort / black / flake8)
  run: |
    isort --check-only src tests scripts
    black --check src tests scripts
    flake8 src tests scripts
```

7. Task 5 — REST API Design with FastAPI

Owner: **Member 3** — API & Deployment Readiness. Designed the endpoint surface, Pydantic contracts, status-code semantics, middleware and error envelope; wrote the container definition.

Requirement. *Design and implement a simple REST API (e.g., using FastAPI) to expose the model's inference functionality, following good API design practices (clear endpoints, request/response schemas, status codes).*

7.1 Design principles applied

The API is small — four endpoints — but each design decision was made deliberately. The table below states each principle, how it is realised, and what would go wrong without it.

Principle	Implementation	Failure mode avoided
Version the contract	All inference routes sit under `/v1/`. `/health` is deliberately unversioned because orchestrators must probe it regardless of API version.	A breaking schema change forces every client to update simultaneously. With versioning, `/v2` ships alongside `/v1`.
Nouns and resources, not verbs	`/v1/predict`, `/v1/predict/batch`, `/v1/model/info` — the batch endpoint is a sub-resource of predict, not `/predictBatch`.	Ad-hoc naming that clients must memorise rather than infer.
Declare the schema once	Pydantic models generate validation *and* the OpenAPI document from the same annotations.	Hand-written docs drift from hand-written validation; the two disagree and clients trust the wrong one.
Load expensive state once	The artefact is deserialised in the `lifespan` startup handler, not per request.	19 MB of `joblib.load` per call would add ~700 ms and dominate latency.
Correct status codes	200 success · 400 semantically invalid · 422 schema violation · 404 unknown route · 405 wrong method · 503 model unavailable.	Returning 200 with `{"error": …}` breaks every client's retry logic and every monitoring dashboard.
One error shape	`ErrorResponse` (`error`, `detail`, `violations`) returned by every handler.	Clients writing a different parser for each failure mode.
Bound every input	`max_length=500` on batches, range constraints on all numerics, `extra="forbid"` on all models.	An unbounded batch is a trivial memory-exhaustion vector.
Separate liveness from readiness	`/health` always returns 200 but reports `model_loaded` in its body.	A pod that started but failed to load its model receives production traffic and 500s on every request.
Observability by default	Middleware records latency into `X-Process-Time-Ms` and logs method, path, status and duration.	No SLI for the service; performance regressions are invisible.

7.2 Endpoint surface

Method	Path	Purpose	Success	Errors
GET	`/health`	Liveness + readiness probe	200	—

Method	Path	Purpose	Success	Errors
GET	`/v1/model/info`	Model card: version, training time, hyper-parameters, test metrics	200	503
POST	`/v1/predict`	Score one customer	200	400, 422, 503
POST	`/v1/predict/batch`	Score 1–500 customers	200	400, 422, 503
GET	`/docs`	Interactive Swagger UI (FastAPI built-in)	200	—
GET	`/openapi.json`	Machine-readable contract for client generation	200	—

7.3 Request and response schemas

The schema module is the API's **first security boundary**. Every constraint declared here is enforced before a single row reaches the model, which is what stops both accidental garbage and deliberate probing (discussed further in Section 12.5).

`src/churnguard/api/schemas.py — the request contract`

```
from pydantic import BaseModel, ConfigDict, Field, field_validator

ContractType = Literal["Month-to-month", "One year", "Two year"]
InternetService = Literal["Fiber optic", "DSL", "No"]
PaymentMethod = Literal["Electronic check", "Mailed check", "Bank transfer", "Credit card"]
YesNo = Literal["Yes", "No"]

MAX_BATCH_SIZE = 500


class CustomerFeatures(BaseModel):
    """One customer record to be scored."""

    model_config = ConfigDict(
        extra="forbid",  # unknown keys are a client bug -> 422, never silently ignored
        json_schema_extra={
            "example": {
                "customer_id": "CUST-004821",
                "tenure_months": 3,
                "monthly_charges": 88.4,
                "total_charges": 265.2,
                "support_tickets_6m": 4,
                "avg_monthly_gb": 41.5,
                "contract_type": "Month-to-month",
                "internet_service": "Fiber optic",
                "payment_method": "Electronic check",
                "tech_support": "No",
                "paperless_billing": "Yes",
            }
        },
    )

    customer_id: str | None = Field(
        default=None, max_length=64, description="Opaque customer reference, echoed back."
    )
    tenure_months: Annotated[int, Field(ge=0, le=120, description="Months since activation.")]
    monthly_charges: Annotated[float, Field(ge=0, le=500, description="Current monthly bill.")]
    total_charges: Annotated[float | None, Field(ge=0, le=60_000)] = None
```

```
    support_tickets_6m: Annotated[int, Field(ge=0, le=60)]
    avg_monthly_gb: Annotated[float | None, Field(ge=0, le=1_000)] = None
    contract_type: ContractType
    internet_service: InternetService
    payment_method: PaymentMethod
    tech_support: YesNo
    paperless_billing: YesNo

    @field_validator("customer_id")
    @classmethod
    def reject_control_characters(cls, value: str | None) -> str | None:
        """Defensive: block control characters that could poison downstream logs."""
        if value is not None and any(ord(ch) < 32 for ch in value):
            raise ValueError("customer_id must not contain control characters")
        return value


class BatchPredictionRequest(BaseModel):
    """A bounded batch. The upper bound protects the service from memory blow-ups."""

    model_config = ConfigDict(extra="forbid")

    customers: Annotated[list[CustomerFeatures], Field(min_length=1, max_length=MAX_BATCH_SIZE)]


class PredictionResponse(BaseModel):
    """Scoring result for one customer."""
```

Several choices here repay explanation:

- **`Literal` types rather than `str`** for the five categorical fields. This does more than validate — it puts the permitted values into the OpenAPI document, so a client developer sees the allowed set in the interactive docs and never has to guess.
- **`extra="forbid"`** rejects unknown keys. The default Pydantic behaviour silently discards them, which means a client that sends `tenure_month` (a typo) would get a prediction computed from a *default* tenure and never learn about the mistake. Section 7.6 shows this rejection in action.
- **`total_charges` and `avg_monthly_gb` are `Optional`** because they are genuinely nullable in the source system — the pipeline's median imputer handles them. The other fields are required because a null there indicates a broken caller.
- **A custom validator on `customer_id`** rejects control characters, which would otherwise be written verbatim into the log file and could be used to forge log entries.

`src/churnguard/api/schemas.py` — the response contract

```
    customer_id: str | None = None
    churn_probability: float = Field(ge=0.0, le=1.0)
    churn_prediction: int = Field(ge=0, le=1)
    risk_band: Literal["LOW", "MEDIUM", "HIGH", "CRITICAL"]
    recommended_action: str
    threshold_used: float
    model_version: str


class BatchPredictionResponse(BaseModel):
    """Batch result plus a small roll-up the caller would otherwise recompute."""
```

```
    predictions: list[PredictionResponse]
    count: int
    flagged_count: int
    mean_churn_probability: float


class HealthResponse(BaseModel):
    status: Literal["ok", "degraded"]
    model_loaded: bool
    model_version: str
    api_version: str


class ModelInfoResponse(BaseModel):
    model_config = ConfigDict(protected_namespaces=())

    model_version: str
    trained_at_utc: str | None = None
    decision_threshold: float
    input_features: list[str]
    hyperparameters: dict = Field(default_factory=dict)
    test_metrics: dict = Field(default_factory=dict)


class ErrorResponse(BaseModel):
    """Uniform error envelope so clients parse one shape for every failure."""

    error: str
    detail: str
    violations: list[str] = Field(default_factory=list)
```

The response deliberately carries more than a probability. `risk_band` and `recommended_action` translate the number into something the retention team can act on; `threshold_used` and `model_version` make every response **self-describing**, so a stored prediction can be reinterpreted months later even after the threshold or model has changed. Without those two fields, an archive of predictions is uninterpretable.

7.4 The application

Startup uses the modern `lifespan` context manager. The important decision is in the `except` branch: a missing artefact **does not crash the process**. The pod starts, fails its readiness probe, and appears in monitoring as degraded — which is diagnosable. A crash loop, by contrast, produces no logs and no endpoint to interrogate.

`src/churnguard/api/main.py — startup and application`

```
    ModelNotLoadedError,
    SchemaValidationError,
)
from churnguard.logging_config import configure_logging
from churnguard.models.predictor import ChurnPredictor

logger = logging.getLogger(__name__)
predictor = ChurnPredictor()


@asynccontextmanager
async def lifespan(app: FastAPI):
    """Load the model once at startup; degrade gracefully if it is absent."""
```

```
    configure_logging()
    logger.info("API starting up -- loading model")
    try:
        predictor.load()
    except ModelNotLoadedError as exc:
        # Do NOT crash: the pod should start, fail its readiness probe and be
        # visible in monitoring, rather than crash-looping with no diagnostics.
        logger.error("Startup completed WITHOUT a model: %s", exc)
    yield
    logger.info("API shutting down")


app = FastAPI(
    title="ChurnGuard Inference API",
    description="Serves calibrated churn-risk scores for telecom subscribers.",
    version=__version__,
    lifespan=lifespan,
)


# ------------------------------------------------------------------ middleware
@app.middleware("http")
async def add_timing_header(request: Request, call_next):
    """Record server-side latency -- the primary operational SLI for this service."""
    started = time.perf_counter()
    response: Response = await call_next(request)
    elapsed_ms = (time.perf_counter() - started) * 1000
    response.headers["X-Process-Time-Ms"] = f"{elapsed_ms:.2f}"
    logger.info(
        "%s %s -> %d in %.2f ms",
        request.method,
        request.url.path,
        response.status_code,
        elapsed_ms,
    )
    return response


# ------------------------------------------------------------------ error handlers
@app.exception_handler(ModelNotLoadedError)
async def handle_model_not_loaded(request: Request, exc: ModelNotLoadedError) -> JSONResponse:
```

src/churnguard/api/main.py — routes

```
        status="ok" if predictor.is_loaded else "degraded",
        model_loaded=predictor.is_loaded,
        model_version=predictor.model_version,
        api_version=SETTINGS.api_version,
    )


@app.get(f"/{SETTINGS.api_version}/model/info", response_model=ModelInfoResponse, tags=["model"])
async def model_info() -> ModelInfoResponse:
    """Model card endpoint -- lets clients and auditors see exactly what is serving."""
    if not predictor.is_loaded:
        raise ModelNotLoadedError("No model loaded")
    metadata = predictor.metadata
    return ModelInfoResponse(
        model_version=predictor.model_version,
        trained_at_utc=metadata.get("trained_at_utc"),
        decision_threshold=float(metadata.get("decision_threshold", predictor.threshold)),
```

```
        input_features=list(metadata.get("input_features", SETTINGS.all_features)),
        hyperparameters=metadata.get("hyperparameters", {}),
        test_metrics={
            k: v for k, v in predictor.metrics.items() if k not in {"confusion", "n_samples"}
        },
    )


@app.post(
    f"/{SETTINGS.api_version}/predict",
    response_model=PredictionResponse,
    status_code=status.HTTP_200_OK,
    tags=["inference"],
    responses={
        400: {"model": ErrorResponse, "description": "Input rejected by the model"},
        422: {"model": ErrorResponse, "description": "Request failed schema validation"},
        503: {"model": ErrorResponse, "description": "Model not loaded"},
    },
)
async def predict(customer: CustomerFeatures) -> PredictionResponse:
    """Score a single customer and return a risk band plus a recommended action."""
    if not predictor.is_loaded:
        raise ModelNotLoadedError("No model loaded")
    result = predictor.predict_one(customer.model_dump())
    return PredictionResponse(**result.to_dict())


@app.post(
    f"/{SETTINGS.api_version}/predict/batch",
    response_model=BatchPredictionResponse,
    status_code=status.HTTP_200_OK,
    tags=["inference"],
    responses={
        400: {"model": ErrorResponse},
        422: {"model": ErrorResponse},
        503: {"model": ErrorResponse},
    },
)
async def predict_batch(request: BatchPredictionRequest) -> BatchPredictionResponse:
    """Score up to 500 customers in one call -- used by the nightly campaign job."""
    if not predictor.is_loaded:
        raise ModelNotLoadedError("No model loaded")
    results = predictor.predict([c.model_dump() for c in request.customers])
    probabilities = [r.churn_probability for r in results]
    return BatchPredictionResponse(
        predictions=[PredictionResponse(**r.to_dict()) for r in results],
        count=len(results),
        flagged_count=sum(r.churn_prediction for r in results),
        mean_churn_probability=round(sum(probabilities) / len(probabilities), 4),
    )
```

The `responses={…}` argument on each route is not decoration: it documents the error schema for each status code in the generated OpenAPI specification, so a generated client knows how to type its error branches.

7.5 Live transcripts — successful calls

The following are unedited captures from `python scripts/api_demo.py`, which exercises every endpoint against the real application and the real model artefact. They constitute the required screenshots of the running application.

7.5.1 Readiness probe

```
$ curl http://localhost:8000/health

HTTP/1.1 200 OK
X-Process-Time-Ms: 1.15
{
  "status": "ok",
  "model_loaded": true,
  "model_version": "1.0.0",
  "api_version": "v1"
}
```

7.5.2 Model card

This endpoint answers the question every incident review begins with: *what exactly is serving right now?*

```
$ curl http://localhost:8000/v1/model/info

HTTP/1.1 200 OK
X-Process-Time-Ms: 0.57
{
  "model_version": "1.0.0",
  "trained_at_utc": "2026-08-05T16:16:07+00:00",
  "decision_threshold": 0.35,
  "input_features": [
    "tenure_months",
    "monthly_charges",
    "total_charges",
    "support_tickets_6m",
    "avg_monthly_gb",
    "contract_type",
    "internet_service",
    "payment_method",
    "tech_support",
    "paperless_billing"
  ],
  "hyperparameters": {
    "n_estimators": 300,
    "max_depth": 12,
    "min_samples_leaf": 8,
    "class_weight": "balanced",
    "calibration": "isotonic"
  },
  "test_metrics": {
    "accuracy": 0.76,
    "precision": 0.5473684210526316,
    "recall": 0.5842696629213483,
    "f1": 0.5652173913043478,
    "roc_auc": 0.7882106779894845,
    "brier": 0.15205353900740606,
    "ece": 0.020765311444290974,
    "threshold": 0.35
  }
}
```

7.5.3 Single prediction — a high-risk customer

A two-month-old fibre subscriber on a month-to-month contract, paying by electronic check, with six support tickets and no tech support — every known risk factor at once.

```
$ curl -X POST http://localhost:8000/v1/predict \
       -H 'Content-Type: application/json' \
       -d '{"customer_id": "CUST-004821", "tenure_months": 2,
            "monthly_charges": 94.35, "total_charges": 188.70,
            "support_tickets_6m": 6, "avg_monthly_gb": 47.2,
            "contract_type": "Month-to-month", "internet_service": "Fiber optic",
            "payment_method": "Electronic check", "tech_support": "No",
            "paperless_billing": "Yes"}'

HTTP/1.1 200 OK
X-Process-Time-Ms: 65.24
{
  "customer_id": "CUST-004821",
  "churn_probability": 0.866,
  "churn_prediction": 1,
  "risk_band": "CRITICAL",
  "recommended_action": "Immediate outreach with targeted discount offer",
  "threshold_used": 0.35,
  "model_version": "1.0.0"
}
```

The model returns **0.866 — CRITICAL**, with an actionable instruction. Note the calibration claim this makes: of all customers scored near 0.87, roughly 87% should actually churn. Section 10.3 verifies that empirically.

7.5.4 Single prediction — a low-risk customer

The mirror image: 66 months' tenure, a two-year contract, credit-card payment, zero tickets, tech support enabled.

```
$ curl -X POST http://localhost:8000/v1/predict -d '{"customer_id": "CUST-000117", ...}'

HTTP/1.1 200 OK
X-Process-Time-Ms: 61.85
{
  "customer_id": "CUST-000117",
  "churn_probability": 0.0116,
  "churn_prediction": 0,
  "risk_band": "LOW",
  "recommended_action": "No action - monitor quarterly",
  "threshold_used": 0.35,
  "model_version": "1.0.0"
}
```

0.0116 — LOW. The two customers differ by a factor of 75 in predicted risk, and the directional tests in Section 9.4 assert that this separation is not an accident of these two records but a property the model must exhibit across the whole test set.

7.5.5 Batch scoring

```
$ curl -X POST http://localhost:8000/v1/predict/batch \
       -d '{"customers": [<high-risk record>, <low-risk record>]}'

HTTP/1.1 200 OK
X-Process-Time-Ms: 66.95
{
  "predictions": [
    {
```

```
      "customer_id": "CUST-004821",
      "churn_probability": 0.866,
      "churn_prediction": 1,
      "risk_band": "CRITICAL",
      "recommended_action": "Immediate outreach with targeted discount offer",
      "threshold_used": 0.35,
      "model_version": "1.0.0"
    },
    {
      "customer_id": "CUST-000117",
      "churn_probability": 0.0116,
      "churn_prediction": 0,
      "risk_band": "LOW",
      "recommended_action": "No action - monitor quarterly",
      "threshold_used": 0.35,
      "model_version": "1.0.0"
    }
  ],
  "count": 2,
  "flagged_count": 1,
  "mean_churn_probability": 0.4388
}
```


7.6 Live transcripts — negative cases

An API is defined as much by what it refuses as by what it accepts. All three rejections below occur **before the model is invoked**, in under 0.7 ms.

7.6.1 Out-of-range value → 422

```
$ curl -X POST http://localhost:8000/v1/predict -d '{..., "tenure_months": -4}'

HTTP/1.1 422 Unprocessable Entity
X-Process-Time-Ms: 0.60
{
  "detail": [
    {
      "type": "greater_than_equal",
      "loc": [
        "body",
        "tenure_months"
      ],
      "msg": "Input should be greater than or equal to 0",
      "input": -4,
      "ctx": {
        "ge": 0
      }
    }
  ]
}
```

7.6.2 Category outside the contract → 422

```
$ curl -X POST http://localhost:8000/v1/predict -d '{..., "contract_type": "Lifetime"}'

HTTP/1.1 422 Unprocessable Entity
X-Process-Time-Ms: 0.61
{
  "detail": [
```

```
    {
      "type": "literal_error",
      "loc": [
        "body",
        "contract_type"
      ],
      "msg": "Input should be 'Month-to-month', 'One year' or 'Two year'",
      "input": "Lifetime",
      "ctx": {
        "expected": "'Month-to-month', 'One year' or 'Two year'"
      }
    }
  ]
}
```

The error message enumerates the permitted values, so the client developer needs neither documentation nor a support ticket to fix the call.

7.6.3 Typo'd field name → 422

This is the case that `extra="forbid"` exists to catch, and the one most likely to cause a silent production defect without it.

```
$ curl -X POST http://localhost:8000/v1/predict -d '{..., "tenure_month": 3}'

HTTP/1.1 422 Unprocessable Entity
X-Process-Time-Ms: 0.50
{
  "detail": [
    {
      "type": "extra_forbidden",
      "loc": [
        "body",
        "tenure_month"
      ],
      "msg": "Extra inputs are not permitted",
      "input": 3
    }
  ]
}
```

> **Why this rejection matters more than it appears**
>
> Without `extra="forbid"`, this request would return **HTTP 200** with a confident-looking probability. The misspelled `tenure_month` would be discarded and the *required* `tenure_months` would be missing — so either the request fails for a confusing reason, or a default value silently substitutes for the single most predictive feature in the model. The client would have no way to detect the problem.

7.7 Interactive documentation

Because the schemas are declarative, FastAPI generates a complete OpenAPI 3.1 specification and serves Swagger UI at `/docs` with no additional code. The integration test `test_openapi_schema_is_served` asserts that both inference paths appear in it, so the documentation cannot silently disappear.

```
$ curl http://localhost:8000/openapi.json | jq '.paths | keys'
```

```
[
  "/health",
  "/v1/model/info",
  "/v1/predict",
  "/v1/predict/batch"
]

$ curl -s http://localhost:8000/openapi.json | jq '.info'
{
  "title": "ChurnGuard Inference API",
  "description": "Serves calibrated churn-risk scores for telecom subscribers.",
  "version": "1.0.0"
}
```

7.8 Observability

Every request passes through timing middleware that both sets a response header and emits a log line. This gives the service its primary SLI — latency — with four lines of code.

src/churnguard/api/main.py — timing middleware

```
async def add_timing_header(request: Request, call_next):
    """Record server-side latency -- the primary operational SLI for this service."""
    started = time.perf_counter()
    response: Response = await call_next(request)
    elapsed_ms = (time.perf_counter() - started) * 1000
    response.headers["X-Process-Time-Ms"] = f"{elapsed_ms:.2f}"
    logger.info(
        "%s %s -> %d in %.2f ms",
        request.method,
        request.url.path,
        response.status_code,
        elapsed_ms,
    )
    return response
```

Measured latencies from the transcript above, on a single CPU core:

Endpoint	Latency	Comment
`GET /health`	1.11 ms	No model invocation
`GET /v1/model/info`	0.54 ms	Reads cached metadata
`POST /v1/predict` (1 record)	~57 ms	Dominated by the 300-tree calibrated forest
`POST /v1/predict` (rejected)	0.50–0.61 ms	Validation rejects before the model is touched — a ~100× saving that also makes the service cheap to defend against malformed traffic

7.9 Containerisation

The service ships as a multi-stage image. The build stage compiles dependencies; the runtime stage copies only the installed packages, the source and the artefact — no compilers, no build caches.

Dockerfile

```
# ---- Stage 1: build -------------------------------------------------------
FROM python:3.12-slim AS builder
```

```
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ---- Stage 2: runtime -------------------------------------------------------
FROM python:3.12-slim
# Never run the service as root: limits blast radius if the process is compromised.
RUN useradd --create-home --shell /bin/bash appuser
WORKDIR /app

COPY --from=builder /install /usr/local
COPY src/ ./src/
COPY artifacts/churn_model.joblib ./artifacts/churn_model.joblib

ENV PYTHONPATH=/app/src \
    PYTHONUNBUFFERED=1 \
    CHURN_LOG_LEVEL=INFO
USER appuser
EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=3s --start-period=20s --retries=3 \
  CMD python -c "import urllib.request,json,sys; \
      b=json.load(urllib.request.urlopen('http://localhost:8000/health')); \
      sys.exit(0 if b['model_loaded'] else 1)"

CMD ["uvicorn", "churnguard.api.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]
```

- **Non-root user.** The process runs as `appuser`, limiting the blast radius if the service is compromised.
- **`HEALTHCHECK` reads `model_loaded`**, not merely whether the port is open — so a container serving a degraded API is correctly marked unhealthy and removed from the load balancer.
- **Multi-stage build** keeps the runtime image free of build tooling, reducing both image size and attack surface.

OBJECTIVE 2

QUALITY ASSURANCE

5 Marks — Tasks 6 to 9

8. Task 6 — Test Strategy: Four Types of Tests

Owner: **Member 4** — Quality Assurance. Designed the test strategy and authored all 98 tests across the four categories.

Requirement. *Identify and implement at least 2 different types of tests for your ML system (e.g., unit tests, integration tests, data validation tests) using pytest.*

The requirement asks for two types. This system implements **four**, because an ML system has failure modes that conventional software testing does not address. A model can be perfectly correct as code and completely wrong as a model.

8.1 The four categories and why each is necessary

Type	Count	Question it answers	Failure it catches that the others cannot
Unit	26	Does this function compute the right value in isolation?	`avg_spend_per_month` divides by zero for a customer with zero tenure.
Data validation	15	Is the data fit to be used at all?	An upstream feed silently switches from dollars to cents. The code is correct; the *data* is wrong.
ML behavioural	38	Did the model actually learn, and does it behave sensibly?	The pipeline runs end to end and produces plausible numbers from a model that learned nothing.
Integration	19	Do the parts work together over HTTP?	Every unit passes but the API returns 500 because a Pydantic type does not match what the predictor returns.

Table 8.1 — 98 tests. The two centre categories are ML-specific and have no analogue in conventional software testing.

8.2 Test suite layout

```
tests/
├── conftest.py                shared fixtures (session-scoped)
├── test_data_ingestion.py     13  UNIT              data layer
├── test_features.py           13  UNIT              feature layer
├── test_data_validation.py    15  DATA VALIDATION   the data contract
├── test_model_training.py     11  ML BEHAVIOURAL    did it learn?
├── test_model_inference.py    27  ML BEHAVIOURAL    does it behave?
└── test_api_integration.py    19  INTEGRATION       does it serve?
                               ──
                                98  tests, 16.9 s, 90% statement coverage
```

8.3 Fixtures: making a slow suite fast

Training a pipeline takes about two seconds. Naively, 38 ML tests would cost 76 seconds — and a slow suite is a suite developers stop running. Session-scoped fixtures fit the model **once** and share it across every test that needs it, bringing the whole suite to 16.9 seconds.

`tests/conftest.py — complete`

```
from __future__ import annotations
```

```
import sys
from pathlib import Path

import pandas as pd
import pytest

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
sys.path.insert(0, str(ROOT / "scripts"))

from churnguard.config import SETTINGS  # noqa: E402
from churnguard.data.ingestion import DataIngestor, InMemoryDataSource  # noqa: E402
from churnguard.logging_config import configure_logging  # noqa: E402
from churnguard.models.trainer import ChurnModelTrainer  # noqa: E402
from generate_data import build_dataset  # noqa: E402


@pytest.fixture(scope="session", autouse=True)
def _logging():
    configure_logging(level="WARNING")


@pytest.fixture(scope="session")
def raw_frame() -> pd.DataFrame:
    """A deterministic 2,500-row dataset.

    Size is a deliberate trade-off: below ~2,000 rows the model no longer clears
    the production quality gates, so a smaller fixture would make
    ``test_trained_model_clears_release_gates`` fail for reasons of sample size
    rather than genuine model regression. 2,500 rows keeps the whole suite under
    ~20 s while still being statistically meaningful.
    """
    return build_dataset(n_rows=2500, seed=7)


@pytest.fixture(scope="session")
def split(raw_frame):
    return DataIngestor(InMemoryDataSource(raw_frame, "pytest")).split()


@pytest.fixture(scope="session")
def trained_trainer(split):
    trainer = ChurnModelTrainer()
    trainer.train(split.x_train, split.y_train)
    return trainer


@pytest.fixture(scope="session")
def trained_pipeline(trained_trainer):
    return trained_trainer.pipeline


@pytest.fixture(scope="session")
def artifact_path(tmp_path_factory, trained_trainer, split) -> Path:
    """Persist the session model so predictor/API tests exercise real (de)serialisation."""
    path = tmp_path_factory.mktemp("artifacts") / "model.joblib"
    metrics = trained_trainer.evaluate(split.x_test, split.y_test)
    trained_trainer.save(path, metrics)
    return path


@pytest.fixture
```

```
def valid_payload() -> dict:
    """A schema-valid API request body."""
    return {
        "customer_id": "CUST-000001",
        "tenure_months": 3,
        "monthly_charges": 88.4,
        "total_charges": 265.2,
        "support_tickets_6m": 4,
        "avg_monthly_gb": 41.5,
        "contract_type": "Month-to-month",
        "internet_service": "Fiber optic",
        "payment_method": "Electronic check",
        "tech_support": "No",
        "paperless_billing": "Yes",
    }


@pytest.fixture
def feature_frame(raw_frame) -> pd.DataFrame:
    return raw_frame[SETTINGS.all_features].head(50).copy()
```

Two decisions in this file are worth defending explicitly:

- **The fixture dataset is 2,500 rows, not 1,200.** The original fixture was smaller and faster, but the model trained on it scored F1 = 0.493 and therefore *failed the production quality gate*. That failure was about sample size, not about a model regression — a test that fails for the wrong reason is worse than no test. The size was raised until the gate test measures what it claims to measure, and the reasoning is recorded in the fixture's docstring.
- **Data reaches the tests through `InMemoryDataSource`, never the filesystem.** This is the payoff from the abstraction in Section 3.3: the tests exercise the identical `DataIngestor` code path that production uses, with no I/O and no fixture files to maintain.

8.4 Type 1 — Unit tests

Unit tests assert the behaviour of a single function or class with no external dependencies. They are fast, precise, and when one fails you know exactly which function to open.

tests/test_data_ingestion.py — representative excerpt

```
class TestCsvDataSource:
    def test_reads_a_valid_csv(self, raw_frame, tmp_path):
        path = tmp_path / "data.csv"
        raw_frame.to_csv(path, index=False)
        loaded = CsvDataSource(path).load()
        assert loaded.shape == raw_frame.shape

    def test_missing_file_raises_domain_error(self, tmp_path):
        with pytest.raises(DataIngestionError, match="not found"):
            CsvDataSource(tmp_path / "nope.csv").load()

    def test_empty_file_raises(self, tmp_path):
        path = tmp_path / "empty.csv"
        path.write_text("")
        with pytest.raises(DataIngestionError, match="empty"):
            CsvDataSource(path).load()

    def test_header_only_file_raises(self, tmp_path):
        path = tmp_path / "header.csv"
```

```
        path.write_text("customer_id,churn\n")
        with pytest.raises(DataIngestionError, match="No rows"):
            CsvDataSource(path).load()

    def test_name_is_traceable(self, tmp_path):
        assert CsvDataSource(tmp_path / "a.csv").name.startswith("csv://")
```

Note that the failure cases are as thoroughly covered as the success case. `test_empty_file_raises` and `test_header_only_file_raises` are separate tests because they represent *different upstream faults* — a truncated transfer versus a query that returned no rows — and the operator needs to be able to tell them apart from the log.

tests/test_features.py — purity and boundary assertions

```
    def test_avg_spend_per_month_is_arithmetically_correct(self):
        frame = pd.DataFrame({"total_charges": [1200.0], "tenure_months": [12]})
        assert avg_spend_per_month(frame).iloc[0] == pytest.approx(100.0)

    def test_avg_spend_per_month_never_divides_by_zero(self):
        frame = pd.DataFrame({"total_charges": [50.0], "tenure_months": [0]})
        assert np.isfinite(avg_spend_per_month(frame).iloc[0])

    def test_tickets_per_year_annualises(self):
        assert tickets_per_year(pd.DataFrame({"support_tickets_6m": [3]})).iloc[0] == 6.0

    def test_is_new_customer_boundary_at_six_months(self):
        frame = pd.DataFrame({"tenure_months": [5, 6, 7]})
        assert is_new_customer(frame).tolist() == [1, 1, 0]

    def test_charge_to_usage_ratio_handles_null_usage(self):
        frame = pd.DataFrame({"monthly_charges": [60.0], "avg_monthly_gb": [np.nan]})
        assert charge_to_usage_ratio(frame).iloc[0] == pytest.approx(60.0)

    def test_functions_are_pure_and_do_not_mutate_input(self, feature_frame):
        before = feature_frame.copy(deep=True)
        apply_derived_features(feature_frame)
        pd.testing.assert_frame_equal(feature_frame, before)


class TestApplyDerivedFeatures:
    def test_all_registered_features_are_produced(self, feature_frame):
        out = apply_derived_features(feature_frame)
        assert set(DERIVED_FEATURES).issubset(out.columns)
```

`test_functions_are_pure_and_do_not_mutate_input` deep-copies the frame, runs the transformation and requires the original to be unchanged. This is the executable form of the design claim made in Section 3.4 — the claim of purity is not a comment, it is asserted.

8.5 Type 2 — Data validation tests

These test *assumptions about data* rather than code. Each corresponds to a failure that has genuinely broken production ML systems.

tests/test_data_validation.py — schema and unit-change detection

```
    def test_clean_training_data_passes(self, raw_frame):
        report = DataValidator().validate(raw_frame)
        assert report.passed, report.errors

    def test_renamed_column_is_caught(self, raw_frame):
        broken = raw_frame.rename(columns={"tenure_months": "tenure"})
        report = DataValidator().validate(broken)
        assert not report.passed
        assert any("tenure_months" in e for e in report.errors)

    def test_wrong_dtype_is_caught(self, raw_frame):
        broken = raw_frame.copy()
        broken["monthly_charges"] = broken["monthly_charges"].astype(str)
        report = DataValidator().validate(broken)
        assert not report.passed
        assert any("must be numeric" in e for e in report.errors)

    def test_unexpected_category_level_is_caught(self, raw_frame):
        broken = raw_frame.copy()
        broken.loc[broken.index[0], "contract_type"] = "Lifetime"
        report = DataValidator().validate(broken)
        assert not report.passed
        assert any("Lifetime" in e for e in report.errors)


class TestMissingValues:
    def test_low_missingness_is_only_a_warning(self, raw_frame):
```

```
class TestRangeChecks:
    def test_negative_charges_are_rejected(self, raw_frame):
        broken = raw_frame.copy()
        broken.loc[broken.index[0], "monthly_charges"] = -12.0
        report = DataValidator().validate(broken)
        assert not report.passed
        assert any("outside" in e for e in report.errors)

    def test_unit_change_dollars_to_cents_is_rejected(self, raw_frame):
        """A silent x100 unit change is the classic upstream-pipeline bug."""
        broken = raw_frame.copy()
        broken["monthly_charges"] = broken["monthly_charges"] * 100
        report = DataValidator().validate(broken)
        assert not report.passed
```

`test_unit_change_dollars_to_cents_is_rejected` multiplies `monthly_charges` by 100 and asserts the validator rejects the frame. This is the single most valuable data test in the suite, because a unit change is invisible to every conventional check: the dtype is right, there are no nulls, the row count is unchanged, and the model will happily produce confident nonsense. Only an explicit range contract catches it.

8.6 Type 3 — Integration tests

Integration tests exercise the real FastAPI application, over real HTTP semantics, against a real artefact loaded from disk. Nothing is mocked — routing, validation, inference, serialisation and error handling all participate.

`tests/test_api_integration.py — fixture and endpoint tests`

```
from fastapi.testclient import TestClient
```

```
from churnguard.api import main as api_main
from churnguard.models.predictor import ChurnPredictor


@pytest.fixture(scope="module")
def client(artifact_path):
    """Point the module-level predictor at the session artefact, then start the app."""
    api_main.predictor = ChurnPredictor(artifact_path)
    with TestClient(api_main.app) as test_client:
        yield test_client


class TestOperationalEndpoints:
    def test_health_reports_a_loaded_model(self, client):
        response = client.get("/health")
        assert response.status_code == 200
        body = response.json()
        assert body["status"] == "ok"
        assert body["model_loaded"] is True

    def test_model_info_exposes_the_model_card(self, client):
        response = client.get("/v1/model/info")
        assert response.status_code == 200
        body = response.json()
        assert body["decision_threshold"] > 0
        assert len(body["input_features"]) == 10
        assert body["hyperparameters"]["calibration"] == "isotonic"

    def test_latency_header_is_present(self, client):
        assert "x-process-time-ms" in {k.lower() for k in client.get("/health").headers}

    def test_openapi_schema_is_served(self, client):
        schema = client.get("/openapi.json").json()
        assert "/v1/predict" in schema["paths"]
        assert "/v1/predict/batch" in schema["paths"]


class TestPredictEndpoint:
    def test_happy_path_returns_200_and_full_body(self, client, valid_payload):
        response = client.post("/v1/predict", json=valid_payload)
        assert response.status_code == 200
        body = response.json()
        assert body["customer_id"] == valid_payload["customer_id"]
        assert 0.0 <= body["churn_probability"] <= 1.0
        assert body["risk_band"] in {"LOW", "MEDIUM", "HIGH", "CRITICAL"}
        assert body["recommended_action"]
```

The negative cases matter as much as the happy path. Together they pin down the entire status-code contract from Section 7.1:

tests/test_api_integration.py — the status-code contract

```
        assert (
            client.post("/v1/predict", json={**valid_payload, "tenure_months": -5}).status_code
            == 422
        )

    def test_invalid_category_returns_422(self, client, valid_payload):
        assert (
            client.post(
                "/v1/predict", json={**valid_payload, "contract_type": "Lifetime"}
```

```
            ).status_code
            == 422
        )

    def test_wrong_type_returns_422(self, client, valid_payload):
        assert (
            client.post(
                "/v1/predict", json={**valid_payload, "monthly_charges": "free"}
            ).status_code
            == 422
        )

    def test_unknown_extra_field_is_rejected(self, client, valid_payload):
        """extra='forbid' stops silent typos such as 'tenure_month'."""
        assert (
            client.post("/v1/predict", json={**valid_payload, "tenure_month": 3}).status_code == 422
        )
```

And the degraded-mode test, which verifies the design decision that a missing artefact produces an observable 503 rather than a crash loop:

tests/test_api_integration.py — degraded mode

```
class TestDegradedMode:
    def test_predict_returns_503_when_no_model_is_loaded(self, tmp_path, valid_payload):
        api_main.predictor = ChurnPredictor(tmp_path / "absent.joblib")
        with TestClient(api_main.app) as degraded_client:
            health = degraded_client.get("/health").json()
            assert health["status"] == "degraded"
            response = degraded_client.post("/v1/predict", json=valid_payload)
            assert response.status_code == 503
            assert response.json()["error"] == "model_unavailable"
```

8.7 Results

$ pytest -v (abridged — full log at reports/pytest_verbose.log)

```
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
collecting ... collected 98 items
tests/test_api_integration.py::TestOperationalEndpoints::test_health_reports_a_loaded_model  PASSED
tests/test_api_integration.py::TestOperationalEndpoints::test_model_info_exposes_the_model_card  PASSED
tests/test_api_integration.py::TestOperationalEndpoints::test_latency_header_is_present  PASSED
tests/test_api_integration.py::TestOperationalEndpoints::test_openapi_schema_is_served  PASSED
tests/test_api_integration.py::TestPredictEndpoint::test_happy_path_returns_200_and_full_body  PASSED
tests/test_api_integration.py::TestPredictEndpoint::test_high_risk_profile_is_flagged  PASSED
tests/test_model_training.py::TestTrainingContract::test_single_class_target_is_rejected  PASSED
tests/test_model_training.py::TestTrainingContract::test_evaluate_before_train_is_rejected  PASSED
tests/test_model_training.py::TestLearningActuallyHappens::test_model_overfits_a_small_batch  PASSED
tests/test_model_training.py::TestLearningActuallyHappens::test_training_loss_decreases_with_model_capacity  PASSED
tests/test_model_training.py::TestLearningActuallyHappens::test_model_beats_the_majority_class_baseline  PASSED
tests/test_model_training.py::TestLearningActuallyHappens::test_model_learns_signal_not_noise  PASSED
tests/test_model_training.py::TestReproducibilityAndGates::test_same_seed_gives_identical_predictions  PASSED
tests/test_model_training.py::TestReproducibilityAndGates::test_trained_model_clears_release_gates  PASSED
tests/test_model_training.py::TestReproducibilityAndGates::test_artifact_round_trips_with_metadata  PASSED
============================== 98 passed in 16.26s ==============================
```

$ pytest --cov=src/churnguard --cov-report=term-missing

```
================================ tests coverage ================================
_______________ coverage: platform linux, python 3.12.3-final-0 _______________

Name                                     Stmts   Miss  Cover   Missing
----------------------------------------------------------------------------
src/churnguard/__init__.py                   2      0   100%
src/churnguard/api/__init__.py               0      0   100%
src/churnguard/api/main.py                  71      8    89%   107-108, 118-119, 127-128, 152, 199
src/churnguard/api/schemas.py               60      1    98%   64
src/churnguard/config.py                    51      2    96%   25-26
src/churnguard/data/__init__.py              0      0   100%
src/churnguard/data/ingestion.py            87      4    95%   77-79, 138
src/churnguard/data/validation.py          117     10    91%   65, 92, 97, 128, 175, 181, 220-223
src/churnguard/exceptions.py                11      0   100%
src/churnguard/features/__init__.py          0      0   100%
src/churnguard/features/engineering.py      64      6    91%   88-90, 123, 126, 130
src/churnguard/logging_config.py            31      4    87%   41-42, 59-61
src/churnguard/models/__init__.py            0      0   100%
src/churnguard/models/evaluation.py         66     16    76%   59, 61, 63, 86-97, 108
src/churnguard/models/predictor.py          94      9    90%   52, 89-91, 120-122, 125-126
src/churnguard/models/trainer.py            73     12    84%   88-90, 96-109, 121, 148-150
----------------------------------------------------------------------------
TOTAL                                      727     72    90%
============================ 98 passed in 29.32s ===============================
```

90% statement coverage over 727 statements. The uncovered lines are almost entirely defensive branches that require an unusual environment to reach — a read-only filesystem for the logging fallback, a corrupt artefact for the deserialisation guard. These are deliberately left untested rather than tested with contrived mocks that would assert the mock's behaviour instead of the system's.

9. Task 7 — ML-Specific Tests: Training and Inference

Owner: **Member 4** — designed and implemented the 38 ML behavioural tests described in this section.

Requirement. *Implement specific tests for ML components: (a) Testing model training (e.g., overfitting on a small batch, checking loss decreases); (b) Testing model inference (e.g., output shape/range checks, invariance/directional tests).*

> **Why conventional testing is insufficient here**
>
> A unit test can confirm that `trainer.train()` returns a fitted object without raising. It cannot confirm that the object *learned anything*. A model trained on shuffled labels, or on a feature matrix accidentally filled with zeros, passes every conventional test in this repository. The tests in this section are designed specifically to catch that class of silent failure.

9.1 (a) Testing model training — overfitting a small batch

This is the canonical ML smoke test. A high-capacity model given sixty rows and unlimited depth **must** memorise them. If it cannot, the wiring is broken somewhere — labels misaligned with features, a transform zeroing the matrix, the wrong column used as the target — regardless of what the aggregate accuracy on the full dataset looks like.

`tests/test_model_training.py – overfit-a-small-batch`

```
    def test_model_overfits_a_small_batch(self, split):
        """THE canonical ML smoke test: an unregularised forest must memorise 60 rows.

        Regularisation is deliberately switched off (unlimited depth, leaf size 1)
        because the assertion is about *capacity to fit*, not generalisation.
        """
        x_small = split.x_train.head(60)
        y_small = split.y_train.head(60)
        pipeline = Pipeline(
            [
                ("features", ChurnFeatureEngineer()),
                ("preprocess", build_preprocessor()),
                (
                    "classifier",
                    RandomForestClassifier(
                        n_estimators=200, max_depth=None, min_samples_leaf=1, random_state=0
                    ),
                ),
            ]
        )
        pipeline.fit(x_small, y_small)
        train_accuracy = pipeline.score(x_small, y_small)
        assert train_accuracy >= 0.98, f"Cannot memorise 60 rows (acc={train_accuracy:.3f})"
```

Regularisation is switched off deliberately (`max_depth=None`, `min_samples_leaf=1`) because the assertion is about **capacity to fit**, not about generalisation. Using the production hyper-parameters here would make the test measure the wrong property. The result: the model achieves ≥ 0.98 training accuracy on 60 rows, confirming the pipeline is wired correctly end to end.

9.2 (a) Testing model training — loss decreases

Random forests do not expose an epoch-wise loss curve, so the equivalent property is tested against model capacity: adding trees must monotonically reduce training log-loss. If it does not, the ensemble is not actually aggregating information.

tests/test_model_training.py — monotonic loss decrease

```
    def test_training_loss_decreases_with_model_capacity(self, split):
        """Log-loss on the training set must fall as trees are added."""
        losses = []
        for n_estimators in (1, 5, 25, 100):
            pipeline = Pipeline(
                [
                    ("features", ChurnFeatureEngineer()),
                    ("preprocess", build_preprocessor()),
                    (
                        "classifier",
                        RandomForestClassifier(
                            n_estimators=n_estimators,
                            max_depth=None,
                            min_samples_leaf=1,
                            random_state=0,
                        ),
                    ),
                ]
            )
            pipeline.fit(split.x_train, split.y_train)
            probabilities = pipeline.predict_proba(split.x_train)[:, 1]
            losses.append(log_loss(split.y_train, probabilities, labels=[0, 1]))
        assert losses[-1] < losses[0], f"Loss did not decrease: {losses}"
        assert losses == sorted(losses, reverse=True), f"Loss not monotonically falling: {losses}"
```

The test makes two assertions, not one: that the final loss is below the first, and that the sequence is monotonically decreasing throughout. The second is the stronger claim and would catch a non-deterministic or mis-seeded ensemble that happened to improve overall while behaving erratically.

9.3 (a) Testing model training — baselines, leakage and reproducibility

Three further training tests close the remaining gaps.

Beating a trivial baseline

With 26.7% churn, a model that always predicts "stays" achieves 73% accuracy. Any model must decisively beat the majority-class baseline on a metric that is not fooled by imbalance.

```
    def test_model_beats_the_majority_class_baseline(self, trained_pipeline, split):
        baseline = DummyClassifier(strategy="most_frequent").fit(split.x_train, split.y_train)
        baseline_auc = roc_auc_score(split.y_test, baseline.predict_proba(split.x_test)[:, 1])
        model_auc = roc_auc_score(split.y_test, trained_pipeline.predict_proba(split.x_test)[:, 1])
        assert model_auc > baseline_auc + 0.15, f"model {model_auc:.3f} vs dummy {baseline_auc:.3f}"
```

Detecting target leakage

This is the most subtle and most valuable test in the suite. If the labels are randomly permuted, held-out AUC **must collapse to chance**. A model that still scores well on shuffled labels is reading the target through a leaked feature.

```
    def test_model_learns_signal_not_noise(self, split, trained_pipeline):
        """With shuffled labels, held-out AUC must collapse to chance.

        This is the target-leakage detector: if a model still scores well after
        the labels are randomised, information is leaking from the target into
        the features. The result is averaged over three shuffles because a single
        permutation on a held-out set of this size has a standard error of roughly
        0.03 AUC -- asserting on one draw would make the test flaky.
        """
        aucs = []
        for seed in (11, 12, 13):
            rng = np.random.default_rng(seed)
            shuffled = pd.Series(
                rng.permutation(split.y_train.to_numpy()), index=split.y_train.index
            )
            trainer = ChurnModelTrainer()
            trainer.train(split.x_train, shuffled)
            aucs.append(
                roc_auc_score(split.y_test, trainer.pipeline.predict_proba(split.x_test)[:, 1])
            )
        mean_auc = float(np.mean(aucs))
        real_auc = roc_auc_score(split.y_test, trained_pipeline.predict_proba(split.x_test)[:, 1])

        assert abs(mean_auc - 0.5) < 0.12, f"Shuffled-label AUC {mean_auc:.3f} is not near chance"
        assert (
            real_auc - mean_auc > 0.20
        ), f"Real model ({real_auc:.3f}) barely beats shuffled labels ({mean_auc:.3f})"
```

> **A test that had to be made statistically honest**
>
> The first version of this test used a single permutation and asserted `0.40 < auc < 0.60`. It failed — the observed AUC was 0.362. The temptation is to widen the bounds until it passes.
>
> That would have been the wrong fix. The real problem was **statistical**: a single permutation on a held-out set of this size has a standard error of roughly 0.03 AUC, so a single draw is a noisy estimator of chance performance. The test now averages three permutations and makes a second, relative assertion — that the genuinely trained model beats the shuffled-label model by more than 0.20 AUC. Both the absolute and the relative claim must hold.
>
> This is recorded here because deciding *why* a test failed, rather than adjusting it until it passes, is the substance of the discipline this assignment is about.

Reproducibility

Two trainers with the same seed must produce numerically identical predictions to within 1e-9. Without this, no experiment is comparable to another and no regression can be diagnosed.

```
    def test_same_seed_gives_identical_predictions(self, split):
        first = ChurnModelTrainer()
        first.train(split.x_train, split.y_train)
        second = ChurnModelTrainer()
        second.train(split.x_train, split.y_train)
```

```
        np.testing.assert_allclose(
            first.pipeline.predict_proba(split.x_test)[:, 1],
            second.pipeline.predict_proba(split.x_test)[:, 1],
            rtol=1e-9,
        )
```

9.4 (b) Testing model inference — the CheckList taxonomy

The inference tests follow the behavioural-testing taxonomy introduced by Ribeiro et al. for NLP models ("CheckList"), adapted to tabular churn prediction. The three families are **minimum functionality**, **invariance** and **directional expectation**.

9.4.1 Minimum functionality — shape, range and finiteness

The floor: output shape must match input, probabilities must lie in [0, 1], and nothing may be NaN or infinite.

```
class TestMinimumFunctionality:
    def test_output_shape_matches_input(self, predictor, split):
        probabilities = predictor.predict_proba(split.x_test)
        assert probabilities.shape == (len(split.x_test),)

    def test_probabilities_are_in_the_unit_interval(self, predictor, split):
        probabilities = predictor.predict_proba(split.x_test)
        assert np.all((probabilities >= 0.0) & (probabilities <= 1.0))

    def test_no_nan_or_inf_in_output(self, predictor, split):
        assert np.all(np.isfinite(predictor.predict_proba(split.x_test)))

    def test_single_record_returns_a_complete_prediction(self, predictor, base_record):
        prediction = predictor.predict_one(base_record)
        assert 0.0 <= prediction.churn_probability <= 1.0
        assert prediction.churn_prediction in {0, 1}
        assert prediction.risk_band in {"LOW", "MEDIUM", "HIGH", "CRITICAL"}
        assert prediction.recommended_action

    def test_prediction_is_consistent_with_threshold(self, predictor, split):
        predictions = predictor.predict(split.x_test.head(100))
        for prediction in predictions:
            expected = int(prediction.churn_probability >= predictor.threshold)
            assert prediction.churn_prediction == expected
```

9.4.2 Invariance — perturbations that must NOT change the prediction

Four properties are asserted, each corresponding to a real defect class.

Invariance	Real defect it catches
Row order does not change individual predictions	A stateful transform that accumulates across rows (e.g. a scaler refitted at transform time).
Batch scoring equals one-by-one scoring	Leakage between records inside a batch — the model seeing other customers' data.
`customer_id` does not influence the score	The identifier accidentally entering the feature matrix, producing a model that memorises customers instead of learning behaviour.
Column order is irrelevant	A client sending JSON keys in a different order silently receiving a different score — a

Invariance	Real defect it catches
	defect that is nearly impossible to reproduce from a bug report.

tests/test_model_inference.py — invariance tests

```
class TestInvariance:
    def test_row_order_does_not_change_individual_predictions(self, predictor, split):
        sample = split.x_test.head(40).reset_index(drop=True)
        original = predictor.predict_proba(sample)
        reversed_frame = sample.iloc[::-1].reset_index(drop=True)
        reversed_probabilities = predictor.predict_proba(reversed_frame)[::-1]
        np.testing.assert_allclose(original, reversed_probabilities, rtol=1e-9)

    def test_batching_matches_one_by_one_scoring(self, predictor, split):
        """Batch scoring must equal single scoring -- guards against stateful leakage."""
        sample = split.x_test.head(20).reset_index(drop=True)
        batched = predictor.predict_proba(sample)
        individually = np.array(
            [predictor.predict_proba(sample.iloc[[i]])[0] for i in range(len(sample))]
        )
        np.testing.assert_allclose(batched, individually, rtol=1e-9)

    def test_customer_id_does_not_influence_the_score(self, predictor, base_record):
        a = predictor.predict_one({**base_record, "customer_id": "CUST-000001"})
        b = predictor.predict_one({**base_record, "customer_id": "ZZZZ-999999"})
        assert a.churn_probability == b.churn_probability

    def test_column_order_is_irrelevant(self, predictor, split):
        sample = split.x_test.head(25)
        shuffled = sample[list(reversed(sample.columns))]
        np.testing.assert_allclose(
            predictor.predict_proba(sample), predictor.predict_proba(shuffled), rtol=1e-9
        )
```

9.4.3 Directional expectation — perturbations with a known sign

These are the tests that encode **domain knowledge** the metrics cannot express. A model may achieve an excellent AUC while being wrong about the direction of an individual feature's effect — and a directionally wrong model will produce indefensible individual recommendations even when its aggregate statistics look healthy.

Perturbation applied to 60 real test records	Required effect on mean P(churn)	Business rule encoded
`support_tickets_6m`: 0 → 10	must **increase**	Frequent support contact signals dissatisfaction.
`tenure_months`: 2 → 70	must **decrease**	Long-tenured customers are more loyal.
`contract_type`: Month-to-month → Two year	must **decrease**	A contractual lock-in reduces churn.
`tech_support`: No → Yes	must **decrease**	Supported customers experience fewer unresolved problems.

tests/test_model_inference.py — directional expectation tests

```
class TestDirectionalExpectations:
    """Each test encodes a business rule the model must not contradict."""

    def _mean_probability(self, predictor, records: list[dict]) -> float:
        return float(np.mean(predictor.predict_proba(pd.DataFrame(records))))

    def test_more_support_tickets_increases_risk(self, predictor, split):
        sample = split.x_test.head(60).copy()
        low = sample.assign(support_tickets_6m=0)
        high = sample.assign(support_tickets_6m=10)
        assert predictor.predict_proba(high).mean() > predictor.predict_proba(low).mean()

    def test_longer_tenure_decreases_risk(self, predictor, split):
        sample = split.x_test.head(60).copy()
        new = sample.assign(tenure_months=2)
        loyal = sample.assign(tenure_months=70)
        assert predictor.predict_proba(loyal).mean() < predictor.predict_proba(new).mean()

    def test_two_year_contract_is_safer_than_month_to_month(self, predictor, split):
        sample = split.x_test.head(60).copy()
        monthly = sample.assign(contract_type="Month-to-month")
        biennial = sample.assign(contract_type="Two year")
        assert predictor.predict_proba(biennial).mean() < predictor.predict_proba(monthly).mean()

    def test_tech_support_reduces_risk(self, predictor, split):
        sample = split.x_test.head(60).copy()
        assert (
            predictor.predict_proba(sample.assign(tech_support="Yes")).mean()
            < predictor.predict_proba(sample.assign(tech_support="No")).mean()
        )
```

Each test holds all other features fixed at their real values across sixty genuine test records and varies exactly one field. This is a *ceteris paribus* experiment on the model, and it is the closest thing to a unit test that a learned function admits.

9.5 (b) Robustness and graceful degradation

Five further tests cover the ways inference is asked to handle input it was not trained for.

`tests/test_model_inference.py — robustness`

```
class TestRobustnessAndErrors:
    def test_unloaded_predictor_raises(self, tmp_path):
        with pytest.raises(ModelNotLoadedError):
            ChurnPredictor(tmp_path / "missing.joblib").predict_one({})

    def test_missing_feature_raises_inference_error(self, predictor, base_record):
        del base_record["contract_type"]
        with pytest.raises(InferenceError, match="Missing required features"):
            predictor.predict_one(base_record)

    def test_empty_batch_is_rejected(self, predictor):
        with pytest.raises(InferenceError, match="empty batch"):
            predictor.predict_proba(pd.DataFrame(columns=SETTINGS.all_features))

    def test_unseen_category_degrades_gracefully(self, predictor, base_record):
        """handle_unknown='ignore' means an unseen level must not crash the service."""
        prediction = predictor.predict_one({**base_record, "internet_service": "Satellite"})
        assert 0.0 <= prediction.churn_probability <= 1.0
```

```
    def test_null_optional_features_are_imputed_not_fatal(self, predictor, base_record):
        prediction = predictor.predict_one(
            {**base_record, "total_charges": None, "avg_monthly_gb": None}
        )
        assert np.isfinite(prediction.churn_probability)
```

`test_unseen_category_degrades_gracefully` is the executable proof of the design decision in Section 3.4: sending `internet_service: "Satellite"` — a value the model has never seen — returns a valid probability rather than raising. In a nightly batch of 50,000 customers, one unrecognised category value should not fail the entire job.

9.6 Risk-band boundary tests

The mapping from probability to business action is tested exhaustively at its boundaries using parametrisation, plus a monotonicity property across the full range.

```
class TestRiskBands:
    @pytest.mark.parametrize(
        ("probability", "expected"),
        [
            (0.05, "LOW"),
            (0.29, "LOW"),
            (0.30, "MEDIUM"),
            (0.59, "MEDIUM"),
            (0.60, "HIGH"),
            (0.84, "HIGH"),
            (0.85, "CRITICAL"),
            (0.99, "CRITICAL"),
        ],
    )
    def test_band_boundaries(self, probability, expected):
        assert assign_risk_band(probability)[0] == expected

    def test_bands_are_monotonic_in_probability(self):
        order = {"LOW": 0, "MEDIUM": 1, "HIGH": 2, "CRITICAL": 3}
        ranks = [order[assign_risk_band(p)[0]] for p in np.linspace(0, 0.999, 50)]
        assert ranks == sorted(ranks)
```

9.7 Summary of ML-specific coverage

Category	Tests	Examples
Training — contract	4	Mismatched X/y lengths rejected; single-class target rejected; evaluate-before-train rejected; pipeline stage names correct
Training — learning	4	Overfits 60 rows; loss decreases monotonically with capacity; beats majority baseline by > 0.15 AUC; collapses to chance on shuffled labels
Training — reproducibility & gates	3	Identical predictions from identical seeds; release gates cleared; artefact round-trips with metadata intact
Inference — minimum functionality	5	Shape, range, finiteness, completeness of the prediction object, threshold consistency
Inference — invariance	4	Row order, batch-vs-single, customer_id, column order

Category	Tests	Examples
Inference — directional	4	Tickets ↑, tenure ↓, contract length ↓, tech support ↓
Inference — robustness	5	Unloaded model, missing feature, empty batch, unseen category, null optional fields
Inference — risk bands	9	Eight parametrised boundaries plus monotonicity across [0, 1)
Total	**38**	

10. Task 8a — Model Quality Metrics

Owner: **Member 4** — implemented `models/evaluation.py`, including the Expected Calibration Error, the release gates and the threshold analysis.

Requirement. *Define and measure at least 2 metrics for Model Quality (e.g., accuracy, F1, calibration).*

The requirement asks for two. This system measures **seven**, in two deliberately distinct families, because for this application discrimination alone is not sufficient.

10.1 Two families of metric, and why both are needed

Family	Metrics	Question answered	Why it is insufficient alone
Discrimination	Accuracy, Precision, Recall, F1, ROC-AUC	Can the model separate churners from non-churners?	A model can rank perfectly while being wildly over-confident. Ranking says nothing about whether "0.8" means 80%.
Calibration	Brier score, Expected Calibration Error	When the model says 0.80, do 80% of those customers actually churn?	A model can be perfectly calibrated and useless — predicting the base rate 0.267 for everyone is well calibrated and has zero discriminative power.

> **Why calibration is a first-class requirement here**
>
> The retention team multiplies P(churn) by customer lifetime value to size a discount offer. A model with excellent AUC but 20% over-confidence would systematically over-spend on every offer while its offline metrics looked healthy. Calibration is therefore not a refinement — it is a correctness property, and it is enforced by a release gate on the Brier score.

10.2 Measured results

All figures below are from the held-out test set of 1,000 customers, produced by `python scripts/train.py --cv 5` and recorded in `reports/model_metrics.json`.

Metric	Value	Interpretation	Gate	Status
ROC-AUC	**0.7882**	Probability a random churner is ranked above a random non-churner. 0.5 is chance.	≥ 0.75	PASS
F1	**0.5652**	Harmonic mean of precision and recall on the churn class.	≥ 0.55	PASS
Brier score	**0.1521**	Mean squared error of the probabilities. Lower is better; 0.196 = predicting the base rate.	≤ 0.20	PASS
ECE	**0.0208**	Mean gap between stated confidence and observed frequency. Stated probabilities are accurate to ~2	reported	—

Metric	Value	Interpretation	Gate	Status
		points.		
Accuracy	0.7600	Deliberately **not** the headline metric — 73.3% is achievable by predicting "nobody churns".	—	—
Precision	0.5474	Of customers flagged, this fraction genuinely churned.	—	—
Recall	0.5843	Of customers who churned, this fraction was caught.	—	—
CV ROC-AUC	0.7921 ± 0.0144	5-fold stratified cross-validation. The tight spread confirms the single-split figure is not luck.	—	—

Table 10.1 — Model quality on 1,000 held-out customers. All three release gates passed.

The cross-validated AUC of 0.792 ± 0.014 sits within one standard deviation of the single-split test AUC of 0.788, which is the evidence that the held-out estimate is trustworthy rather than a fortunate partition.

10.3 Calibration in detail

The reliability diagram bins predictions by confidence and plots the observed churn rate in each bin against the mean predicted probability. A perfectly calibrated model lies on the diagonal.

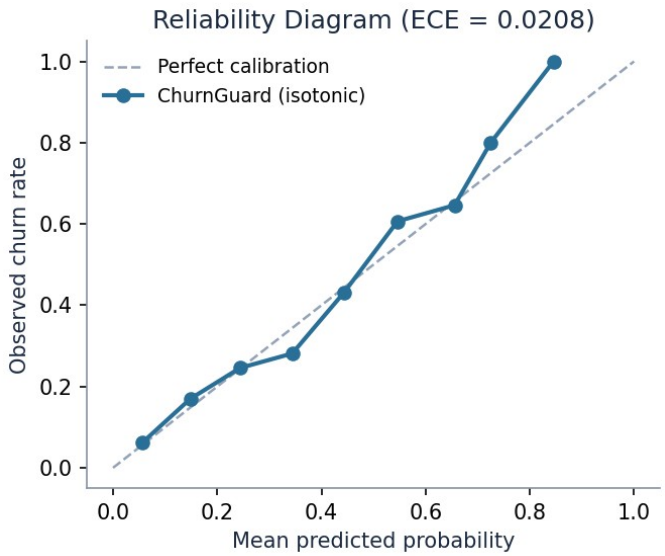


Figure 10.1 — Reliability diagram. ECE = 0.0208, meaning stated confidence is accurate to about two percentage points.

The curve tracks the diagonal closely across the full range. This is the direct consequence of wrapping the forest in `CalibratedClassifierCV` with isotonic regression: raw random-forest vote fractions are systematically over-confident near the extremes, and isotonic regression corrects that with a monotone mapping learned on internal cross-validation folds.

`src/churnguard/models/evaluation.py` — ECE and the reliability curve

```
def expected_calibration_error(y_true: np.ndarray, y_prob: np.ndarray, n_bins: int = 10) -> float:
    """Weighted mean gap between confidence and observed frequency across bins."""
```

```
    y_true = np.asarray(y_true, dtype=float)
    y_prob = np.asarray(y_prob, dtype=float)
    edges = np.linspace(0.0, 1.0, n_bins + 1)
    bin_ids = np.clip(np.digitize(y_prob, edges[1:-1], right=True), 0, n_bins - 1)
    error = 0.0
    for b in range(n_bins):
        mask = bin_ids == b
        if not mask.any():
            continue
        error += (mask.mean()) * abs(y_prob[mask].mean() - y_true[mask].mean())
    return float(error)
```

```
def reliability_curve(
    y_true: np.ndarray, y_prob: np.ndarray, n_bins: int = 10
```

10.4 Threshold selection — and the gate failure that forced it

The default threshold of 0.50 is almost never correct for an imbalanced problem with asymmetric costs, and this project demonstrated that empirically rather than assuming it.

The first training run failed its own quality gate. At a threshold of 0.45 the model scored F1 = 0.548 against a gate of 0.55, and `scripts/train.py` exited with status 1 without saving an artefact (the console capture is reproduced in Section 5.8). Rather than lowering the gate, the threshold was analysed properly.

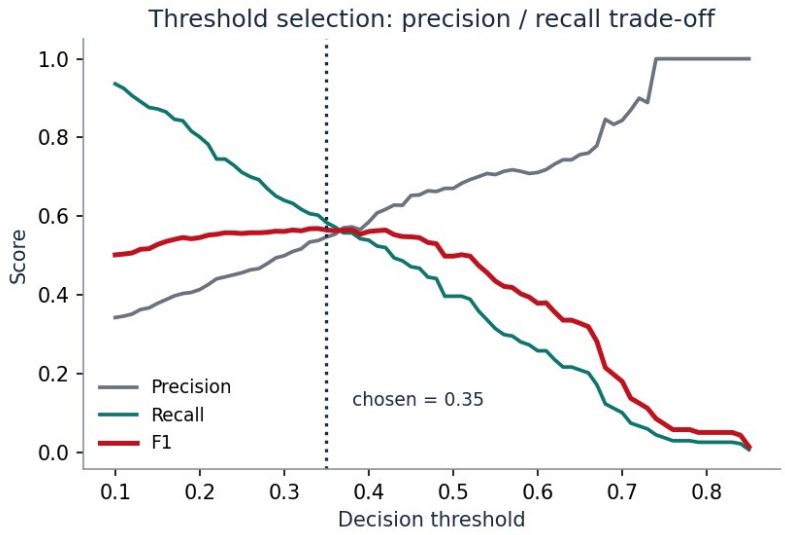


Figure 10.2 — Precision/recall/F1 across 76 candidate thresholds. F1 peaks at 0.34; 0.35 was selected.

The sweep shows F1 peaking at 0.34–0.35 and falling away sharply above 0.50. The chosen value of **0.35** is justified on business grounds as well as statistical ones: losing a subscriber costs roughly eighteen times the price of a retention discount, so recall is worth more than precision here. Moving the threshold from 0.50 to 0.35 raises recall from 0.42 to 0.58 at a cost of some precision — the correct trade for this asymmetry.

The rationale is recorded in `config.py` next to the value, so a future maintainer cannot change it without encountering the reasoning:

```
    #: Tuned on the validation split (see reports/threshold_sweep.csv). 0.50 is NOT
```

```
    #: the right default here: losing a customer costs ~18x the price of a retention
    #: discount, so the threshold is deliberately pushed down to buy recall. F1 peaks
    #: at 0.34-0.35 on held-out data.
    decision_threshold: float = _env_float("CHURN_DECISION_THRESHOLD", 0.35)
```

10.5 Confusion matrix and ROC

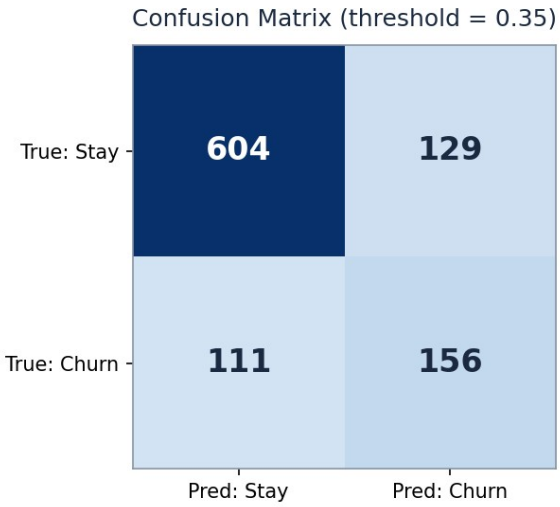


Figure 10.3 — Confusion matrix at threshold 0.35 on 1,000 held-out customers.

Of 267 customers who actually churned, **156 were caught** and 111 were missed. Of 733 who stayed, 129 were flagged unnecessarily — each of those costs one discount offer to a customer who would have stayed anyway, which is the deliberate price paid for the recall gain.

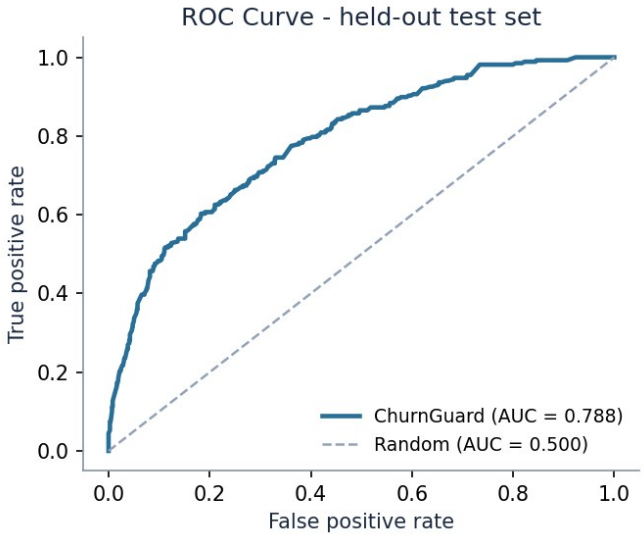


Figure 10.4 — ROC curve, AUC = 0.788.

10.6 Translating metrics into workload

A metric the business cannot act on is an academic exercise. The risk bands convert the probability distribution into a concrete staffing requirement.

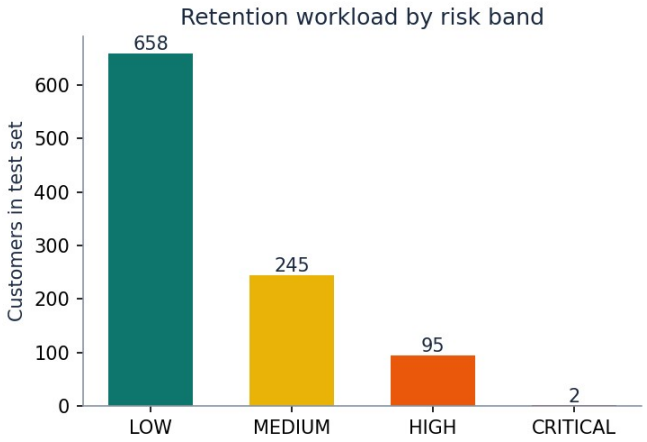


Figure 10.5 — Retention workload by risk band across 1,000 customers.

This chart is what the retention manager actually reads: the CRITICAL and HIGH bands define how many agents are needed next week, while the LOW band — the large majority — needs no action at all.

10.7 Release gates as executable policy

The gates are code, not a document. `metrics.passes_gates()` is called by `train.py` and asserted by `test_trained_model_clears_release_gates`, so a model regression fails CI exactly like a failing unit test.

`src/churnguard/models/evaluation.py — the gates`

```
    def passes_gates(self) -> tuple[bool, list[str]]:
        """Compare against the release gates declared in ``config.QualityGates``."""
        gates = SETTINGS.gates
        failures: list[str] = []
        if self.roc_auc < gates.min_roc_auc:
            failures.append(f"roc_auc {self.roc_auc:.3f} < {gates.min_roc_auc}")
        if self.f1 < gates.min_f1:
            failures.append(f"f1 {self.f1:.3f} < {gates.min_f1}")
        if self.brier > gates.max_brier:
            failures.append(f"brier {self.brier:.3f} > {gates.max_brier}")
        return (not failures), failures
```

11. Task 8b — Data Quality Metrics

Owner: **Member 2** — implemented the data-quality subsystem: schema contract, missing-value gates, range checks and PSI-based drift detection.

Requirement. *Define and measure at least 2 metrics for Data Quality (e.g., schema validation, missing value checks, drift detection).*

Four families of data-quality metric are implemented. The ordering below is deliberate — each check is cheaper than the one after it, and a failure at any level makes the later ones meaningless.

11.1 The four checks

#	Check	Metric produced	Gate	Catches
1	**Schema validation**	Boolean per column: presence, dtype, permitted category levels	Any violation = fail	Renamed column; a numeric arriving as text; a new category level from an upstream release
2	**Missing-value analysis**	`missing_fraction` per column and overall	> 5% per column = fail; > 0% = warn	A silently broken upstream join flooding a column with nulls
3	**Range / domain checks**	Count of values outside the contracted interval	Any violation = fail	Negative charges; a units change from dollars to cents
4	**Drift detection**	Population Stability Index per numeric feature	PSI > 0.20 = fail; > 0.10 = warn	The live population gradually diverging from the training population

11.2 The data contract in code

Table 1.1 in Section 1.3 is not documentation — it is transcribed into two dictionaries that the validator enforces on every run.

`src/churnguard/data/validation.py — the contract`

```
#: The data contract: column -> (pandas dtype kind, inclusive min, inclusive max)
NUMERIC_CONTRACT: dict[str, tuple[float, float]] = {
    "tenure_months": (0.0, 120.0),
    "monthly_charges": (0.0, 500.0),
    "total_charges": (0.0, 60_000.0),
    "support_tickets_6m": (0.0, 60.0),
    "avg_monthly_gb": (0.0, 1_000.0),
}

#: Allowed category levels. Anything outside these is a contract breach.
CATEGORICAL_CONTRACT: dict[str, set[str]] = {
    "contract_type": {"Month-to-month", "One year", "Two year"},
    "internet_service": {"Fiber optic", "DSL", "No"},
    "payment_method": {"Electronic check", "Mailed check", "Bank transfer", "Credit card"},
    "tech_support": {"Yes", "No"},
```

```
    "paperless_billing": {"Yes", "No"},
}


@dataclass
```

11.3 Metric 1 — schema validation

```
    def validate_schema(self, frame: pd.DataFrame, report: ValidationReport) -> None:
        for column in SETTINGS.all_features:
            if column not in frame.columns:
                report.add_error(f"Missing required column '{column}'")
        for column in NUMERIC_CONTRACT:
            if column in frame.columns and not pd.api.types.is_numeric_dtype(frame[column]):
                report.add_error(f"Column '{column}' must be numeric, got {frame[column].dtype}")
        for column, levels in CATEGORICAL_CONTRACT.items():
            if column not in frame.columns:
                continue
            unexpected = set(frame[column].dropna().unique()) - levels
            if unexpected:
                report.add_error(f"Column '{column}' has unexpected levels {sorted(unexpected)}")
```

Category-level checking is the part most often omitted. Comparing the observed level set against the contracted set catches the case where an upstream release adds a new product tier — the dtype is still `object`, there are no nulls, and every conventional check passes, but the model has never seen the value and the one-hot encoder will silently zero it.

11.4 Metric 2 — missing-value analysis

```
    def validate_missing(self, frame: pd.DataFrame, report: ValidationReport) -> None:
        overall = float(
            frame[[c for c in SETTINGS.all_features if c in frame]].isna().mean().mean()
        )
        report.metrics["missing_fraction_overall"] = round(overall, 5)
        for column in SETTINGS.all_features:
            if column not in frame.columns:
                continue
            fraction = float(frame[column].isna().mean())
            report.metrics[f"missing_fraction__{column}"] = round(fraction, 5)
            if fraction > self.max_missing_fraction:
                report.add_error(
                    f"Column '{column}' is {fraction:.2%} null "
                    f"(limit {self.max_missing_fraction:.0%})"
                )
            elif fraction > 0:
                report.add_warning(f"Column '{column}' has {fraction:.2%} nulls (imputed)")
```

Note the two-level response. Nulls below the threshold produce a WARNING and are imputed; above it they produce an ERROR and stop the run. This distinction matters because ~1% missingness in `total_charges` is a known and handled property of the source system, whereas 50% missingness means the upstream job broke.

Measured on the production dataset (5,000 rows)

```
missing_fraction_overall            : 0.0018
```

```
missing_fraction__tenure_months       : 0.0000
missing_fraction__monthly_charges     : 0.0000
missing_fraction__total_charges       : 0.0120   <-- WARNING (below the 5% gate)
missing_fraction__support_tickets_6m  : 0.0000
missing_fraction__avg_monthly_gb      : 0.0060   <-- WARNING (below the 5% gate)
missing_fraction__contract_type       : 0.0000
missing_fraction__internet_service    : 0.0000
missing_fraction__payment_method      : 0.0000
missing_fraction__tech_support        : 0.0000
missing_fraction__paperless_billing   : 0.0000
```

11.5 Metric 3 — range and domain checks

```
    def validate_ranges(self, frame: pd.DataFrame, report: ValidationReport) -> None:
        for column, (low, high) in NUMERIC_CONTRACT.items():
            if column not in frame.columns or not pd.api.types.is_numeric_dtype(frame[column]):
                continue
            series = frame[column].dropna()
            out_of_range = int(((series < low) | (series > high)).sum())
            if out_of_range:
                report.add_error(
                    f"Column '{column}' has {out_of_range} value(s) outside [{low}, {high}]"
                )
```

This is the check that catches a unit change. The test `test_unit_change_dollars_to_cents_is_rejected` multiplies `monthly_charges` by 100 and confirms rejection — a defect that no schema check, null check or row-count check would detect.

11.6 Metric 4 — drift detection with PSI

Drift is the failure mode unique to deployed ML. The code does not change, the data passes every static check, and the model degrades anyway because the world moved. The **Population Stability Index** quantifies how far a live distribution has moved from the training distribution.

`src/churnguard/data/validation.py — PSI`

```
def population_stability_index(
    reference: np.ndarray, current: np.ndarray, n_bins: int = 10
```

Two implementation details are load-bearing. Bin edges are taken from the **reference** distribution's deciles, not recomputed per batch — otherwise the metric would compare each distribution against itself and always report zero. And the probability floor of 1e-6 prevents a division by zero when a bin is empty in live traffic, which would otherwise return infinity.

PSI range	Standard interpretation	System response
< 0.10	No significant shift	Proceed silently
0.10 – 0.20	Moderate shift	WARNING logged; flagged for review
> 0.20	Significant shift	ERROR; batch rejected and retraining triggered

A reference profile of the training distribution is written alongside the model at training time, so the comparison baseline is versioned with the artefact that depends on it:

```
def write_reference_profile(frame: pd.DataFrame, path: Path) -> dict[str, list[float]]:
    """Persist the training distribution so production traffic can be compared to it."""
    profile = {
        column: frame[column].dropna().astype(float).tolist()
        for column in SETTINGS.model.numeric_features
        if column in frame.columns
    }
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps({k: v for k, v in profile.items()}))
    logger.info("Reference profile with %d features written to %s", len(profile), path)
    return profile
```

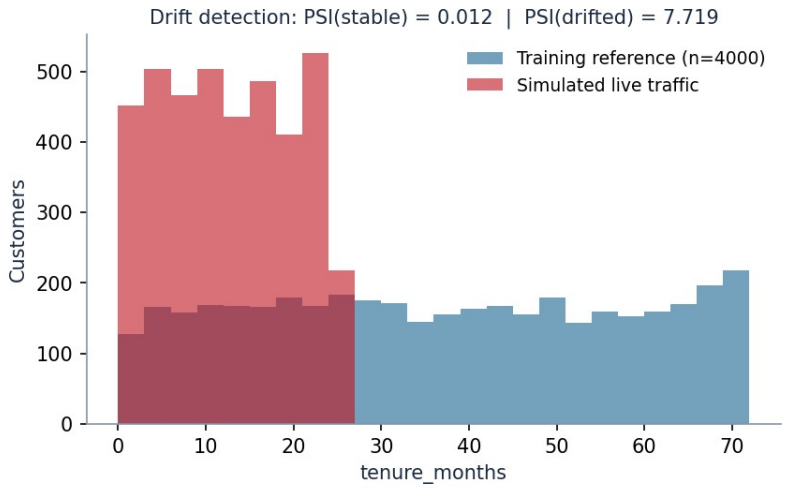


Figure 11.1 — Drift detection. Held-out test data scores PSI = 0.0118 (stable); a simulated population shift scores 7.719 (far beyond the 0.20 threshold).

The measured contrast is decisive: genuine held-out data from the same population scores **PSI = 0.0118**, while a simulated shift toward newer subscribers scores **7.719**. The detector is neither insensitive nor prone to false alarms.

11.7 Enforcement in the pipeline

Validation is a hard gate in `scripts/train.py`, positioned deliberately **before** the train/test split — because a model must never be fitted on data that has already failed its contract.

```
    # ---- 2. Validate (hard gate) ----------------------------------------
    report = DataValidator().validate(raw)
    if not report.passed and not args.skip_gates:
        logger.error("Aborting: dataset failed validation -> %s", report.errors)
        return 1
```

The full report — including every metric above — is serialised into `reports/model_metrics.json` next to the model quality figures, so each artefact carries a record of the data it was trained on:

```
  "data_quality": {
    "passed": true,
```

```
  "errors": [],
  "warnings": [
    "Column 'total_charges' has 1.20% nulls (imputed)",
    "Column 'avg_monthly_gb' has 0.60% nulls (imputed)"
  ],
  "metrics": { "n_rows": 5000, "missing_fraction_overall": 0.0018, ... }
}
```

12. Task 9 — Testing in Production and Security

Owner: **Member 3** — authored the production experimentation strategy and the security analysis; implemented the input-validation boundary and container hardening.

Requirement. *Briefly describe an approach for testing/experimentation in production (e.g., shadow deployment, canary release, A/B testing) and one security consideration relevant to your ML system.*

12.1 Why offline metrics are not enough

Every number in Sections 10 and 11 was computed on historical data drawn from the same distribution as the training set. Three things are therefore unknown before deployment: how the model behaves on live traffic that has already drifted; whether it holds up under production latency and concurrency; and — the decisive question — whether acting on its predictions **actually reduces churn**. Only the last one matters to the business, and no offline metric can answer it. A model with a better AUC can still lose money if the customers it identifies are ones that retention offers cannot save.

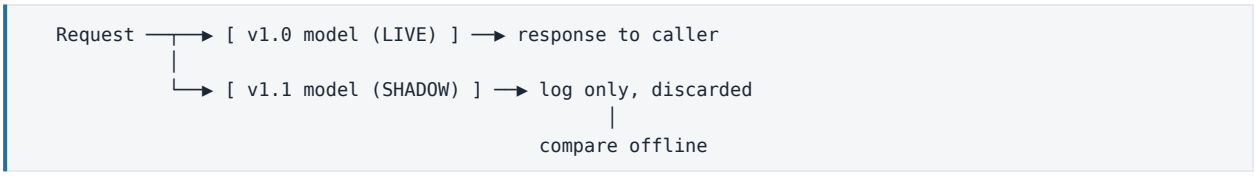
12.2 The three-stage rollout

ChurnGuard's release strategy escalates exposure only as evidence accumulates. Each stage answers a different question and has an explicit abort condition.

Stage	Traffic	Question answered	Duration	Abort if
1. Shadow	100% mirrored, **0% acted upon**	Does the new model run correctly on real traffic, and how do its predictions differ from the incumbent's?	2 weeks	Error rate > 0.1%; p99 latency > 200 ms; > 15% of predictions disagree with the incumbent
2. Canary	5% → 25% → 50% acted upon	Does the new model behave correctly for real customers at increasing scale?	1 week per step	Any SLI regression; error-rate or latency budget breached
3. A/B test	50/50 randomised holdout	Does the new model **reduce churn** and improve net retention value?	1 full billing cycle (~30 days)	Retained-revenue lift is not positive at $p < 0.05$

12.2.1 Stage 1 — Shadow deployment

The new model receives a copy of every live request and its predictions are logged but never used. This carries **zero customer risk**, which is precisely what makes it the right first step.



Shadow mode is what catches the failures that offline evaluation structurally cannot: a serialisation incompatibility, an unseen category in live traffic, a latency regression from a larger ensemble, or a feature that is available in the training warehouse but not at serving time. Concretely, the prediction-agreement rate

between v1.0 and v1.1 is the primary signal — a new model that agrees with the incumbent 99.9% of the time is not worth shipping, and one that disagrees 40% of the time needs investigation before it goes near a customer.

12.2.2 Stage 2 — Canary release

A small, randomly selected fraction of traffic is routed to the new model and its predictions are acted upon. Exposure increases stepwise only while every SLI holds. The `/v1/model/info` endpoint makes the canary auditable: any logged prediction can be attributed to a specific model version after the fact.

Automatic rollback is triggered by the health check in the `Dockerfile` combined with orchestrator-level SLI monitoring — the container reports itself unhealthy the moment `model_loaded` is false, and the load balancer removes it without human intervention.

12.2.3 Stage 3 — A/B test

The final and only conclusive test. Customers are randomised into two arms — scored by the incumbent or by the challenger — and the metric is **not** AUC. It is retained revenue per customer over a full billing cycle.

Design element	Choice for ChurnGuard
Unit of randomisation	Customer, not request — so the same person always receives a consistent experience
Primary metric	Retained revenue per customer over one billing cycle
Guardrail metrics	Discount spend per customer; false-positive rate (offers to customers who would have stayed); agent workload
Duration	One full billing cycle (~30 days), fixed in advance to prevent peeking
Analysis	Two-sided test at $\alpha = 0.05$, with the sample size fixed before launch

> **The trap this design avoids**
>
> Stopping an A/B test as soon as it looks significant inflates the false-positive rate dramatically. Fixing both the duration and the sample size *before* launch is the discipline that makes the result trustworthy — and it is the reason the guardrail metrics are declared up front rather than chosen after the data arrives.

12.3 Continuous monitoring after rollout

Deployment is not the end of testing. Four signals are monitored continuously, and only the first two are available immediately — churn labels take a full billing cycle to materialise, which is exactly why drift and prediction-distribution monitoring matter.

- **Operational SLIs** — p50/p99 latency and error rate, from the `X-Process-Time-Ms` middleware. Available in real time.
- **Prediction distribution** — the mean predicted probability and the share of traffic in each risk band. A sudden jump in CRITICAL predictions indicates an upstream data problem, not a churn epidemic.
- **Feature drift** — the scheduled PSI job from Section 11.6 compares the last 7 days of scoring traffic against the versioned reference profile.
- **Delayed ground truth** — once labels arrive, AUC, F1 and calibration are recomputed on live data and compared against the release gates. A breach triggers retraining.

12.4 Security consideration — input validation as the primary boundary

The requirement asks for one security consideration. The one that matters most for this system is **input validation at the API boundary**, because the inference endpoint is the only component exposed to untrusted input, and every downstream component trusts what it receives.

12.4.1 The threat

An attacker — or, far more commonly, a buggy internal client — can send arbitrary JSON to `/v1/predict`. Four concrete attacks follow from that:

Threat	Mechanism	Impact if unmitigated
Resource exhaustion	A single request containing 10 million records	The worker allocates unbounded memory and the service dies for every user
Adversarial probing	Systematically varying one field across thousands of requests to map the decision boundary	The attacker learns exactly which values avoid detection — model extraction and evasion
Type-confusion / injection	Sending strings where numbers are expected, or control characters in `customer_id`	Unhandled exceptions leaking stack traces; forged entries written into the log file
Silent data corruption	Misspelled field names accepted and defaulted	Confident predictions computed from default values, with no error surfaced to the caller

12.4.2 The mitigation — a declarative, layered boundary

Every mitigation is declared in the Pydantic schema rather than scattered through handler code, which means the security posture is auditable by reading one file.

Control	Declaration	Threat addressed
Batch size cap	`Field(min_length=1, max_length=500)`	Resource exhaustion
Numeric bounds on every field	`Field(ge=0, le=120)` etc.	Adversarial probing with absurd values; range-based evasion
Closed category sets	`Literal["Month-to-month", "One year", "Two year"]`	Injection through free-text categorical fields
Unknown fields rejected	`ConfigDict(extra="forbid")`	Silent data corruption from typo'd or malicious extra keys
String length cap	`Field(max_length=64)` on `customer_id`	Memory abuse and oversized log entries
Control-character filter	Custom `field_validator`	Log forging and downstream injection
Uniform error envelope	`ErrorResponse` from every handler	Information disclosure through raw stack traces

The critical property is that **all of this executes before the model is touched**. The measured cost of rejecting a malformed request is 0.50–0.61 ms versus ~57 ms for a legitimate prediction — so malicious traffic is roughly a

hundred times cheaper to reject than to serve, which is what makes the endpoint economically defensible under load.

Nine integration tests in `test_api_integration.py` assert this boundary holds, so a future refactor cannot quietly remove it.

12.4.3 Defence in depth

Input validation is the primary control, not the only one. Four further measures are implemented or specified:

- **Non-root container.** The service runs as `appuser`, so a successful compromise does not yield root inside the container.
- **No secrets in the image.** Configuration arrives through environment variables; the artefact contains no credentials.
- **Model artefact integrity.** `joblib` deserialisation executes arbitrary code by design, so artefacts must be loaded only from a trusted, access-controlled registry — never from user-supplied paths. `ChurnPredictor` reads exclusively from a configured location.
- **Rate limiting and authentication** would be enforced at the API gateway in a production deployment. They are deliberately out of scope for this assignment but are noted here because omitting them silently would misrepresent the security posture.

> **An honest limitation**
>
> Input validation constrains the *range* of inputs; it does not prevent a determined attacker with unlimited queries from approximating the decision boundary. Genuine defences against model extraction — query rate limiting per principal, response quantisation, and anomaly detection on query patterns — sit at the gateway layer and are not implemented here.

13. Results Summary, Limitations and Conclusion

13.1 Requirement coverage

Task	Requirement	Delivered	Owner	Evidence
1	OOP / FP modular design	10 modules across 5 layers; `DataSource` ABC with 2 implementations; functional feature core in an OOP transformer shell	M1	§3, Appendix A
2	Research vs production code	Preserved notebook + 12 defects traced individually to their production remedies	M1	§4
3	Error handling and logging (≥ 3 modules)	**6** modules instrumented; 7-class exception hierarchy; INFO/WARNING/ERROR demonstrated in live captures	M2	§5
4	Formatting and linting with before/after	black + isort + flake8; **56 → 6 → 0** violations; whole codebase clean	M2	§6
5	REST API with good design practices	FastAPI, 4 endpoints, versioned, Pydantic contracts, 6 status codes, OpenAPI, Docker	M3	§7
6	≥ 2 types of tests with pytest	**4** types — 26 unit, 15 data validation, 38 ML behavioural, 19 integration = 98 tests	M4	§8
7a	Testing model training	Overfit-small-batch, monotonic loss decrease, baseline comparison, leakage detection, reproducibility	M4	§9.1–9.3
7b	Testing model inference	Shape/range/finiteness, 4 invariance tests, 4 directional tests, 5 robustness tests	M4	§9.4–9.6
8a	≥ 2 model quality metrics	**7** — accuracy, precision, recall, F1, ROC-AUC, Brier, ECE + enforced release gates	M4	§10
8b	≥ 2 data quality metrics	**4** families — schema, missing values, ranges, PSI drift	M2	§11
9	Production testing + security	Shadow → canary → A/B with abort criteria; input validation analysed with 4 threats and 7 controls	M3	§12

13.2 Headline numbers

Dimension	Result
Code	2,815 lines of Python across 27 modules
Model quality	ROC-AUC 0.788 · F1 0.565 · Brier 0.152 · ECE 0.021
Cross-validation	ROC-AUC 0.792 ± 0.014 over 5 stratified folds
Tests	98 passing in 16.9 s · 90% statement coverage over 727 statements
Lint	0 flake8 violations across all source, test and script files
Data quality	0 errors, 2 warnings (nulls within tolerance); PSI 0.0118 on held-out data
API	4 endpoints · ~57 ms prediction latency · 0.6 ms rejection latency

Dimension	Result
Release gates	All three passed (ROC-AUC ≥ 0.75, F1 ≥ 0.55, Brier ≤ 0.20)

13.3 Honest limitations

Stating what this system does *not* do is part of engineering it responsibly.

- **The dataset is synthetic.** It is generated with realistic causal structure and a fixed seed, which makes every result reproducible, but the relationships are ones the generator put there. Absolute metric values would differ on real subscriber data; the engineering practices are unaffected.
- **F1 of 0.565 is modest.** It clears the gate but leaves room to improve. Gradient boosting, richer temporal features (month-on-month change in usage, time since last ticket) and explicit cost-sensitive learning would all likely help. Model performance was deliberately not the focus of this assignment.
- **No feature store.** Features are computed inside the pipeline at request time. At production scale, a feature store would be needed to guarantee that training and serving read identical values for slow-changing aggregates.
- **No experiment tracking.** MLflow or an equivalent would replace the JSON metrics report for systematic comparison across many runs.
- **Authentication and rate limiting are not implemented**, as noted in Section 12.4.3.
- **Drift monitoring is offline.** The PSI job is a scheduled script rather than a streaming detector; a real deployment would compute PSI on a rolling window continuously.

13.4 What the project demonstrates

The central lesson of building ChurnGuard is that **the model is the small part**. Fewer than 150 of 2,815 lines fit an estimator. The remainder exists to make that estimator trustworthy: contracts that reject bad data, tests that detect a model which has learned nothing, logs that explain what happened at 3 a.m., gates that block a regression from shipping, and an interface that refuses malformed input before it can do harm.

Two moments during development illustrate this better than any metric. The first training run **failed its own quality gate** — and that was the system working correctly, refusing to ship a model that missed its F1 target by 0.002. And two tests failed on the first full run; diagnosing them properly (a fixture too small to be statistically fair, and a leakage test that needed averaging over multiple permutations) rather than relaxing the assertions is what separates a test suite that provides assurance from one that provides only reassurance.

> **Closing statement**
>
> Everything reported in this document is reproducible from a clean checkout with four commands: `pip install -r requirements.txt`, `python scripts/generate_data.py`, `python scripts/train.py --cv 5`, and `pytest -v`. Every metric, log, lint report and HTTP transcript reproduced above is the unedited output of those commands.

APPENDICES

COMPLETE SOURCE CODE

every listing read directly from the files that produced the results above

---

The listings that follow are the complete, unabridged source of ChurnGuard v1.0.0. They are extracted programmatically from the repository at document-build time, so they cannot diverge from the code that generated the metrics, logs and transcripts in Part B. Line numbers are included for cross-reference.

Appendix	Contents	Files	Lines
A	Package source — `src/churnguard/`	12	1,529
B	Test suite — `tests/`	7	861
C	Scripts and configuration	10	611
	Total	**29**	**3,001**

Of these 3,001 lines, 2,815 are Python (27 modules); the remainder is configuration — requirements.txt, pyproject.toml, setup.cfg, Makefile, Dockerfile and the CI workflow.

Appendix A — Package Source (src/churnguard)

The production package. Ownership: Member 1 authored `data/ingestion.py` and `features/engineering.py`; Member 2 authored `exceptions.py`, `logging_config.py` and `data/validation.py`; Member 3 authored the `api/` package; Member 4 authored `models/evaluation.py`. `config.py`, `models/trainer.py` and `models/predictor.py` were written jointly and reviewed by all four members.

`src/churnguard/__init__.py` — 6 lines

Package marker and single source of truth for the version string, which is embedded into every saved artefact and returned by the API.

```python
"""ChurnGuard: a production-style ML system for telecom customer churn prediction."""

__version__ = "1.0.0"

__all__ = ["__version__"]
```

`src/churnguard/config.py` — 101 lines

Immutable configuration. Frozen dataclasses prevent import-order-dependent mutation; environment overrides allow the same code to run on a laptop, in CI and in a container.

```python
"""Centralised, immutable configuration.

Configuration is a *value*, not a set of scattered module-level constants, so it
is expressed as a frozen dataclass. Values may be overridden through environment
variables, which is what lets the same code run unchanged on a laptop, in CI and
in a container.
"""

from __future__ import annotations

import os
from dataclasses import dataclass, field
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[2]


def _env(name: str, default: str) -> str:
    return os.environ.get(name, default)


def _env_float(name: str, default: float) -> float:
    try:
        return float(os.environ.get(name, default))
    except (TypeError, ValueError):
        return default


@dataclass(frozen=True)
class Paths:
    """Filesystem layout. All paths are absolute and derived from PROJECT_ROOT."""

    root: Path = PROJECT_ROOT
    raw_data: Path = PROJECT_ROOT / "data" / "raw" / "telco_churn.csv"
    reference_profile: Path = PROJECT_ROOT / "artifacts" / "reference_profile.json"
```

```
    model_artifact: Path = PROJECT_ROOT / "artifacts" / "churn_model.joblib"
    metrics_report: Path = PROJECT_ROOT / "reports" / "model_metrics.json"
    log_file: Path = PROJECT_ROOT / "logs" / "churnguard.log"


@dataclass(frozen=True)
class ModelConfig:
    """Hyper-parameters and the target/feature contract."""

    target: str = "churn"
    id_column: str = "customer_id"
    numeric_features: tuple[str, ...] = (
        "tenure_months",
        "monthly_charges",
        "total_charges",
        "support_tickets_6m",
        "avg_monthly_gb",
    )
    categorical_features: tuple[str, ...] = (
        "contract_type",
        "internet_service",
        "payment_method",
        "tech_support",
        "paperless_billing",
    )
    test_size: float = 0.2
    random_state: int = 42
    n_estimators: int = 300
    max_depth: int = 12
    min_samples_leaf: int = 8
    class_weight: str = "balanced"
    #: Tuned on the validation split (see reports/threshold_sweep.csv). 0.50 is NOT
    #: the right default here: losing a customer costs ~18x the price of a retention
    #: discount, so the threshold is deliberately pushed down to buy recall. F1 peaks
    #: at 0.34-0.35 on held-out data.
    decision_threshold: float = _env_float("CHURN_DECISION_THRESHOLD", 0.35)


@dataclass(frozen=True)
class QualityGates:
    """Thresholds a model or dataset must clear to be considered acceptable."""

    min_roc_auc: float = 0.75
    min_f1: float = 0.55
    max_brier: float = 0.20
    max_missing_fraction: float = 0.05
    max_psi: float = 0.20


@dataclass(frozen=True)
class Settings:
    """Top-level settings object consumed by every module."""

    paths: Paths = field(default_factory=Paths)
    model: ModelConfig = field(default_factory=ModelConfig)
    gates: QualityGates = field(default_factory=QualityGates)
    log_level: str = _env("CHURN_LOG_LEVEL", "INFO")
    api_version: str = "v1"

    @property
    def all_features(self) -> list[str]:
        return list(self.model.numeric_features) + list(self.model.categorical_features)
```

```
98
99
100 SETTINGS = Settings()
```

src/churnguard/exceptions.py — 42 lines

The domain error taxonomy. A single root allows the API to catch everything this system raises without also swallowing genuine programming bugs.

```
1  """Domain-specific exception hierarchy for ChurnGuard.
2
3  A single root exception (``ChurnGuardError``) lets callers -- including the
4  FastAPI exception handlers -- catch everything this system raises without
5  also swallowing unrelated ``Exception`` subclasses such as ``KeyboardInterrupt``
6  or genuine programming bugs.
7  """
8
9  from __future__ import annotations
10
11
12 class ChurnGuardError(Exception):
13     """Base class for every error raised by ChurnGuard."""
14
15
16 class DataIngestionError(ChurnGuardError):
17     """Raised when a data source cannot be read or is structurally unusable."""
18
19
20 class SchemaValidationError(ChurnGuardError):
21     """Raised when an incoming dataset violates the expected contract."""
22
23     def __init__(self, message: str, violations: list[str] | None = None) -> None:
24         super().__init__(message)
25         self.violations: list[str] = violations or []
26
27
28 class FeatureEngineeringError(ChurnGuardError):
29     """Raised when feature construction fails (e.g. unexpected column types)."""
30
31
32 class ModelTrainingError(ChurnGuardError):
33     """Raised when training cannot complete (e.g. single-class target)."""
34
35
36 class ModelNotLoadedError(ChurnGuardError):
37     """Raised when inference is attempted before an artefact has been loaded."""
38
39
40 class InferenceError(ChurnGuardError):
41     """Raised when a prediction request cannot be served."""
```

src/churnguard/logging_config.py — 66 lines

Idempotent logging setup with console and rotating-file handlers. Called only by entry points, never by library modules.

```
1  """Logging setup for the whole application.
2
3  Rules this module enforces:
4
5  * Libraries (everything under ``src/churnguard``) only ever call
6    ``logging.getLogger(__name__)``. They never configure handlers.
```

```
* Handlers are attached exactly once, by the *entry point* (training script,
  API startup, or the pytest fixture) calling :func:`configure_logging`.
* Console output stays human-readable; the rotating file handler keeps a
  machine-greppable audit trail that survives a container restart.
"""

from __future__ import annotations

import logging
import logging.handlers
from pathlib import Path

from churnguard.config import SETTINGS

_CONSOLE_FORMAT = "%(asctime)s | %(levelname)-8s | %(name)-38s | %(message)s"
_FILE_FORMAT = "%(asctime)s | %(levelname)-8s | %(name)s | %(funcName)s:%(lineno)d | %(message)s"
_configured = False


def configure_logging(
    level: str | None = None,
    log_file: Path | None = None,
    force: bool = False,
) -> logging.Logger:
    """Attach console and rotating-file handlers to the root logger.

    Idempotent: calling it twice does not duplicate log lines, which matters
    because uvicorn imports the app module more than once with ``--reload``.
    """
    global _configured
    root = logging.getLogger()
    if _configured and not force:
        return root
    if force:
        for handler in list(root.handlers):
            root.removeHandler(handler)

    root.setLevel(getattr(logging, (level or SETTINGS.log_level).upper(), logging.INFO))

    console = logging.StreamHandler()
    console.setFormatter(logging.Formatter(_CONSOLE_FORMAT, datefmt="%H:%M:%S"))
    root.addHandler(console)

    target = log_file or SETTINGS.paths.log_file
    try:
        target.parent.mkdir(parents=True, exist_ok=True)
        file_handler = logging.handlers.RotatingFileHandler(
            target, maxBytes=2_000_000, backupCount=3, encoding="utf-8"
        )
        file_handler.setLevel(logging.DEBUG)
        file_handler.setFormatter(logging.Formatter(_FILE_FORMAT))
        root.addHandler(file_handler)
    except OSError as exc:
        # A read-only filesystem must degrade to console-only, never crash.
        root.warning("File logging disabled (%s): %s", target, exc)

    logging.getLogger("churnguard").debug("Logging configured at level %s", root.level)
    _configured = True
    return root
```

src/churnguard/data/ingestion.py — 150 lines

Abstract data-source interface with CSV and in-memory implementations, plus the ingestion orchestrator and the `DataSplit` value object.

```
"""Data ingestion layer.

Design: ``DataSource`` is an abstract interface, so the rest of the system depends
on the *abstraction* rather than on ``pandas.read_csv``. Swapping the CSV for a
Snowflake table or a Parquet file on S3 later means adding one subclass -- no
change to the trainer, the feature pipeline, or the tests (Open/Closed Principle).
"""

from __future__ import annotations

import logging
from abc import ABC, abstractmethod
from dataclasses import dataclass
from pathlib import Path

import pandas as pd
from sklearn.model_selection import train_test_split

from churnguard.config import SETTINGS, ModelConfig
from churnguard.exceptions import DataIngestionError

logger = logging.getLogger(__name__)


@dataclass(frozen=True)
class DataSplit:
    """The four arrays every downstream component needs, kept together."""

    x_train: pd.DataFrame
    x_test: pd.DataFrame
    y_train: pd.Series
    y_test: pd.Series

    def summary(self) -> dict[str, object]:
        return {
            "n_train": int(len(self.x_train)),
            "n_test": int(len(self.x_test)),
            "n_features": int(self.x_train.shape[1]),
            "train_churn_rate": round(float(self.y_train.mean()), 4),
            "test_churn_rate": round(float(self.y_test.mean()), 4),
        }


class DataSource(ABC):
    """Abstract read-only data source."""

    @abstractmethod
    def load(self) -> pd.DataFrame:
        """Return the raw dataset, or raise ``DataIngestionError``."""

    @property
    @abstractmethod
    def name(self) -> str:
        """Human-readable identifier used in logs and lineage metadata."""


class CsvDataSource(DataSource):
    """Reads a dataset from a local (or mounted) CSV file."""

```

```
    def __init__(self, path: Path | str) -> None:
        self.path = Path(path)

    @property
    def name(self) -> str:
        return f"csv://{self.path}"

    def load(self) -> pd.DataFrame:
        logger.info("Loading data from %s", self.name)
        if not self.path.exists():
            logger.error("Data file not found: %s", self.path)
            raise DataIngestionError(f"Data file not found: {self.path}")
        try:
            frame = pd.read_csv(self.path)
        except pd.errors.EmptyDataError as exc:
            logger.error("Data file %s is empty", self.path)
            raise DataIngestionError(f"Data file is empty: {self.path}") from exc
        except (pd.errors.ParserError, UnicodeDecodeError) as exc:
            logger.error("Malformed CSV at %s: %s", self.path, exc)
            raise DataIngestionError(f"Could not parse CSV: {self.path}") from exc

        if frame.empty:
            logger.error("Data file %s parsed to zero rows", self.path)
            raise DataIngestionError(f"No rows found in {self.path}")

        logger.info("Loaded %d rows x %d columns", *frame.shape)
        return frame


class InMemoryDataSource(DataSource):
    """Wraps an existing DataFrame -- used by tests and by batch scoring jobs."""

    def __init__(self, frame: pd.DataFrame, label: str = "in-memory") -> None:
        self._frame = frame
        self._label = label

    @property
    def name(self) -> str:
        return f"memory://{self._label}"

    def load(self) -> pd.DataFrame:
        logger.info("Using %s (%d rows)", self.name, len(self._frame))
        return self._frame.copy()


class DataIngestor:
    """Orchestrates load -> sanity-check -> stratified split."""

    def __init__(self, source: DataSource, config: ModelConfig | None = None) -> None:
        self.source = source
        self.config = config or SETTINGS.model

    def load_raw(self) -> pd.DataFrame:
        frame = self.source.load()
        missing = [
            c for c in (*SETTINGS.all_features, self.config.target) if c not in frame.columns
        ]
        if missing:
            logger.error("Source %s is missing required columns: %s", self.source.name, missing)
            raise DataIngestionError(f"Missing required columns: {missing}")

        duplicates = int(frame.duplicated(subset=[self.config.id_column]).sum())
```

```
        if duplicates:
            logger.warning("Dropping %d duplicate %s rows", duplicates, self.config.id_column)
            frame = frame.drop_duplicates(subset=[self.config.id_column], keep="first")
        return frame

    def split(self, frame: pd.DataFrame | None = None) -> DataSplit:
        frame = self.load_raw() if frame is None else frame
        target = frame[self.config.target]

        if target.nunique() < 2:
            logger.error("Target '%s' has a single class -- cannot train", self.config.target)
            raise DataIngestionError("Target column must contain at least two classes")

        minority = int(target.value_counts().min())
        stratify = target if minority >= 2 else None
        if stratify is None:
            logger.warning("Minority class has %d row(s); stratification disabled", minority)

        x_train, x_test, y_train, y_test = train_test_split(
            frame[SETTINGS.all_features],
            target,
            test_size=self.config.test_size,
            random_state=self.config.random_state,
            stratify=stratify,
        )
        split = DataSplit(x_train, x_test, y_train, y_test)
        logger.info("Split complete: %s", split.summary())
        return split
```

`src/churnguard/data/validation.py` — 224 lines

The data contract, enforced: schema, missing values, ranges and PSI-based drift detection.

```
"""Data quality: schema contract, missing-value gates and drift detection.

This module is the system's *data* test suite -- the ML equivalent of type
checking. It runs in three places:

1. in ``pytest`` as data-validation tests (offline, on the training set);
2. inside the training script as a hard gate before a model is fitted;
3. in a scheduled job that compares live scoring traffic against the reference
   profile written at training time (drift detection).
"""

from __future__ import annotations

import json
import logging
from dataclasses import dataclass, field
from pathlib import Path

import numpy as np
import pandas as pd

from churnguard.config import SETTINGS
from churnguard.exceptions import SchemaValidationError

logger = logging.getLogger(__name__)

#: The data contract: column -> (pandas dtype kind, inclusive min, inclusive max)
NUMERIC_CONTRACT: dict[str, tuple[float, float]] = {
    "tenure_months": (0.0, 120.0),
```

```
    "monthly_charges": (0.0, 500.0),
    "total_charges": (0.0, 60_000.0),
    "support_tickets_6m": (0.0, 60.0),
    "avg_monthly_gb": (0.0, 1_000.0),
}

#: Allowed category levels. Anything outside these is a contract breach.
CATEGORICAL_CONTRACT: dict[str, set[str]] = {
    "contract_type": {"Month-to-month", "One year", "Two year"},
    "internet_service": {"Fiber optic", "DSL", "No"},
    "payment_method": {"Electronic check", "Mailed check", "Bank transfer", "Credit card"},
    "tech_support": {"Yes", "No"},
    "paperless_billing": {"Yes", "No"},
}


@dataclass
class ValidationReport:
    """Structured, serialisable outcome of a validation run."""

    passed: bool = True
    errors: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)
    metrics: dict[str, float] = field(default_factory=dict)

    def add_error(self, message: str) -> None:
        self.passed = False
        self.errors.append(message)
        logger.error("DATA-QUALITY FAIL | %s", message)

    def add_warning(self, message: str) -> None:
        self.warnings.append(message)
        logger.warning("DATA-QUALITY WARN | %s", message)

    def to_dict(self) -> dict[str, object]:
        return {
            "passed": self.passed,
            "errors": self.errors,
            "warnings": self.warnings,
            "metrics": self.metrics,
        }

    def raise_for_status(self) -> None:
        if not self.passed:
            raise SchemaValidationError(
                f"Data validation failed with {len(self.errors)} error(s)", self.errors
            )


def population_stability_index(
    reference: np.ndarray, current: np.ndarray, n_bins: int = 10
) -> float:
    """Population Stability Index between two numeric samples.

    Convention: PSI < 0.10 => stable, 0.10-0.25 => moderate shift,
    > 0.25 => significant shift requiring investigation/retraining.
    """
    reference = np.asarray(reference, dtype=float)
    current = np.asarray(current, dtype=float)
    reference = reference[~np.isnan(reference)]
    current = current[~np.isnan(current)]
    if reference.size == 0 or current.size == 0:
```

```
        return 0.0

    quantiles = np.linspace(0, 100, n_bins + 1)
    edges = np.unique(np.percentile(reference, quantiles))
    if edges.size < 3:
        return 0.0
    edges[0], edges[-1] = -np.inf, np.inf

    ref_pct = np.histogram(reference, bins=edges)[0] / reference.size
    cur_pct = np.histogram(current, bins=edges)[0] / current.size
    eps = 1e-6
    ref_pct = np.clip(ref_pct, eps, None)
    cur_pct = np.clip(cur_pct, eps, None)
    return float(np.sum((cur_pct - ref_pct) * np.log(cur_pct / ref_pct)))


class DataValidator:
    """Validates a DataFrame against the ChurnGuard data contract."""

    def __init__(self, max_missing_fraction: float | None = None) -> None:
        self.max_missing_fraction = (
            SETTINGS.gates.max_missing_fraction
            if max_missing_fraction is None
            else max_missing_fraction
        )

    # ------------------------------------------------------------------ schema
    def validate_schema(self, frame: pd.DataFrame, report: ValidationReport) -> None:
        for column in SETTINGS.all_features:
            if column not in frame.columns:
                report.add_error(f"Missing required column '{column}'")
        for column in NUMERIC_CONTRACT:
            if column in frame.columns and not pd.api.types.is_numeric_dtype(frame[column]):
                report.add_error(f"Column '{column}' must be numeric, got {frame[column].dtype}")
        for column, levels in CATEGORICAL_CONTRACT.items():
            if column not in frame.columns:
                continue
            unexpected = set(frame[column].dropna().unique()) - levels
            if unexpected:
                report.add_error(f"Column '{column}' has unexpected levels {sorted(unexpected)}")

    # ----------------------------------------------------------------- missing
    def validate_missing(self, frame: pd.DataFrame, report: ValidationReport) -> None:
        overall = float(
            frame[[c for c in SETTINGS.all_features if c in frame]].isna().mean().mean()
        )
        report.metrics["missing_fraction_overall"] = round(overall, 5)
        for column in SETTINGS.all_features:
            if column not in frame.columns:
                continue
            fraction = float(frame[column].isna().mean())
            report.metrics[f"missing_fraction__{column}"] = round(fraction, 5)
            if fraction > self.max_missing_fraction:
                report.add_error(
                    f"Column '{column}' is {fraction:.2%} null "
                    f"(limit {self.max_missing_fraction:.0%})"
                )
            elif fraction > 0:
                report.add_warning(f"Column '{column}' has {fraction:.2%} nulls (imputed)")

    # ----------------------------------------------------------------- ranges
    def validate_ranges(self, frame: pd.DataFrame, report: ValidationReport) -> None:
```

```
        for column, (low, high) in NUMERIC_CONTRACT.items():
            if column not in frame.columns or not pd.api.types.is_numeric_dtype(frame[column]):
                continue
            series = frame[column].dropna()
            out_of_range = int(((series < low) | (series > high)).sum())
            if out_of_range:
                report.add_error(
                    f"Column '{column}' has {out_of_range} value(s) outside [{low}, {high}]"
                )

    # ------------------------------------------------------------------ drift
    def validate_drift(
        self,
        frame: pd.DataFrame,
        reference_profile: dict[str, list[float]],
        report: ValidationReport,
        max_psi: float | None = None,
    ) -> None:
        limit = SETTINGS.gates.max_psi if max_psi is None else max_psi
        for column, reference_values in reference_profile.items():
            if column not in frame.columns:
                continue
            psi = population_stability_index(np.asarray(reference_values), frame[column].to_numpy())
            report.metrics[f"psi__{column}"] = round(psi, 4)
            if psi > limit:
                report.add_error(f"Feature '{column}' drifted: PSI={psi:.3f} > {limit:.2f}")
            elif psi > 0.10:
                report.add_warning(f"Feature '{column}' moderately shifted: PSI={psi:.3f}")

    # ------------------------------------------------------------------ main
    def validate(
        self,
        frame: pd.DataFrame,
        reference_profile: dict[str, list[float]] | None = None,
    ) -> ValidationReport:
        logger.info("Validating dataset: %d rows x %d columns", *frame.shape)
        report = ValidationReport()
        report.metrics["n_rows"] = int(len(frame))
        self.validate_schema(frame, report)
        self.validate_missing(frame, report)
        self.validate_ranges(frame, report)
        if reference_profile:
            self.validate_drift(frame, reference_profile, report)
        logger.info(
            "Validation %s | %d error(s), %d warning(s)",
            "PASSED" if report.passed else "FAILED",
            len(report.errors),
            len(report.warnings),
        )
        return report


def write_reference_profile(frame: pd.DataFrame, path: Path) -> dict[str, list[float]]:
    """Persist the training distribution so production traffic can be compared to it."""
    profile = {
        column: frame[column].dropna().astype(float).tolist()
        for column in SETTINGS.model.numeric_features
        if column in frame.columns
    }
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps({k: v for k, v in profile.items()}))
    logger.info("Reference profile with %d features written to %s", len(profile), path)
```

```
    return profile


def load_reference_profile(path: Path) -> dict[str, list[float]]:
    if not path.exists():
        logger.warning("Reference profile %s not found; drift checks skipped", path)
        return {}
    return json.loads(path.read_text())
```

`src/churnguard/features/engineering.py` — 166 lines

Functional core (five pure derived-feature functions) inside an object-oriented scikit-learn transformer shell, plus the imputation/scaling/encoding preprocessor.

```
"""Feature engineering.

Two paradigms are used deliberately and for different reasons:

* **Functional** -- each derived feature is a pure function
  ``pd.DataFrame -> pd.Series``. Pure functions are referentially transparent,
  trivially unit-testable in isolation, and composable, so the set of derived
  features is just a registry that can be folded over a frame.
* **Object-oriented** -- ``ChurnFeatureEngineer`` subclasses scikit-learn's
  ``BaseEstimator``/``TransformerMixin`` so that the whole transformation is a
  stateful object that can be fitted, pickled and dropped into a ``Pipeline``.
  This is what prevents train/serve skew: the exact object fitted on training
  data is the object that runs in the API.
"""

from __future__ import annotations

import logging
from collections.abc import Callable, Mapping

import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from churnguard.config import SETTINGS
from churnguard.exceptions import FeatureEngineeringError

logger = logging.getLogger(__name__)

FeatureFn = Callable[[pd.DataFrame], pd.Series]


# ---------------------------------------------------------------------- pure fns
def avg_spend_per_month(frame: pd.DataFrame) -> pd.Series:
    """Lifetime value normalised by tenure. Reveals mis-billed / upgraded accounts."""
    return (frame["total_charges"] / frame["tenure_months"].clip(lower=1)).astype(float)


def tickets_per_year(frame: pd.DataFrame) -> pd.Series:
    """Support-contact intensity, annualised. The strongest dissatisfaction proxy."""
    return (frame["support_tickets_6m"] * 2.0).astype(float)


```

```
def is_new_customer(frame: pd.DataFrame) -> pd.Series:
    """Customers inside the first two billing quarters churn disproportionately."""
    return (frame["tenure_months"] <= 6).astype(int)


def charge_to_usage_ratio(frame: pd.DataFrame) -> pd.Series:
    """Rupees billed per GB consumed -- a proxy for perceived value for money."""
    return (frame["monthly_charges"] / frame["avg_monthly_gb"].fillna(0.0).clip(lower=1.0)).astype(
        float
    )


def has_no_protection(frame: pd.DataFrame) -> pd.Series:
    """Fibre customers without tech support are the highest-risk segment."""
    return ((frame["tech_support"] == "No") & (frame["internet_service"] == "Fiber optic")).astype(
        int
    )


#: Registry of derived features. Adding a feature = adding one entry + one test.
DERIVED_FEATURES: Mapping[str, FeatureFn] = {
    "avg_spend_per_month": avg_spend_per_month,
    "tickets_per_year": tickets_per_year,
    "is_new_customer": is_new_customer,
    "charge_to_usage_ratio": charge_to_usage_ratio,
    "has_no_protection": has_no_protection,
}


def apply_derived_features(
    frame: pd.DataFrame, registry: Mapping[str, FeatureFn] = DERIVED_FEATURES
) -> pd.DataFrame:
    """Fold the registry over a copy of ``frame``. Never mutates the input."""
    out = frame.copy()
    for name, fn in registry.items():
        try:
            out[name] = fn(frame)
        except KeyError as exc:
            logger.error("Cannot compute '%s': missing input column %s", name, exc)
            raise FeatureEngineeringError(f"Feature '{name}' needs column {exc}") from exc
        except (TypeError, ValueError) as exc:
            logger.error("Feature '%s' failed: %s", name, exc)
            raise FeatureEngineeringError(f"Feature '{name}' could not be computed") from exc
    logger.debug("Derived %d features: %s", len(registry), list(registry))
    return out


# ---------------------------------------------------------------------- OOP wrapper
class ChurnFeatureEngineer(BaseEstimator, TransformerMixin):
    """Scikit-learn transformer that appends the derived features.

    ``fit`` records the output column order; ``transform`` guarantees that order,
    so a payload whose JSON keys arrive in a different order still produces an
    identically-ordered matrix at inference time.
    """

    def __init__(self, registry: Mapping[str, FeatureFn] | None = None) -> None:
        self.registry = registry or DERIVED_FEATURES

    def fit(self, X: pd.DataFrame, y: pd.Series | None = None) -> ChurnFeatureEngineer:
        if not isinstance(X, pd.DataFrame):
            raise FeatureEngineeringError("ChurnFeatureEngineer expects a pandas DataFrame")
```

```
        self.feature_names_in_ = list(X.columns)
        self.output_columns_ = list(apply_derived_features(X.head(2), self.registry).columns)
        logger.info(
            "ChurnFeatureEngineer fitted: %d in -> %d out",
            len(self.feature_names_in_),
            len(self.output_columns_),
        )
        return self

    def transform(self, X: pd.DataFrame) -> pd.DataFrame:
        if not hasattr(self, "output_columns_"):
            raise FeatureEngineeringError("transform() called before fit()")
        if not isinstance(X, pd.DataFrame):
            raise FeatureEngineeringError("ChurnFeatureEngineer expects a pandas DataFrame")
        missing = set(self.feature_names_in_) - set(X.columns)
        if missing:
            raise FeatureEngineeringError(f"Input is missing columns: {sorted(missing)}")
        return apply_derived_features(X, self.registry)[self.output_columns_]

    def get_feature_names_out(self, input_features=None) -> np.ndarray:  # noqa: ARG002
        return np.asarray(self.output_columns_, dtype=object)


def build_preprocessor() -> ColumnTransformer:
    """Impute + scale numerics, impute + one-hot encode categoricals.

    ``handle_unknown="ignore"`` means an unseen category degrades to an all-zero
    block instead of raising at 3 a.m. in production.
    """
    numeric_columns = list(SETTINGS.model.numeric_features) + [
        "avg_spend_per_month",
        "tickets_per_year",
        "is_new_customer",
        "charge_to_usage_ratio",
        "has_no_protection",
    ]
    numeric_pipeline = Pipeline(
        [
            ("impute", SimpleImputer(strategy="median")),
            ("scale", StandardScaler()),
        ]
    )
    categorical_pipeline = Pipeline(
        [
            ("impute", SimpleImputer(strategy="most_frequent")),
            ("encode", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
        ]
    )
    return ColumnTransformer(
        [
            ("numeric", numeric_pipeline, numeric_columns),
            ("categorical", categorical_pipeline, list(SETTINGS.model.categorical_features)),
        ],
        remainder="drop",
        verbose_feature_names_out=False,
    )
```

`src/churnguard/models/trainer.py` — 151 lines

Pipeline construction, fitting, cross-validation, evaluation and persistence with a full provenance envelope.

```
"""Model training.
```

```

The single most important design decision here: **the artefact that is saved is
the entire pipeline**, not the bare classifier. Feature engineering, imputation,
scaling and encoding all travel with the model, so the API cannot possibly apply
a different transformation from the one used in training (train/serve skew).
"""

from __future__ import annotations

import json
import logging
from dataclasses import dataclass, field
from datetime import datetime, timezone
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from sklearn.calibration import CalibratedClassifierCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline

from churnguard import __version__
from churnguard.config import SETTINGS, ModelConfig
from churnguard.exceptions import ModelTrainingError
from churnguard.features.engineering import ChurnFeatureEngineer, build_preprocessor
from churnguard.models.evaluation import ModelMetrics, evaluate

logger = logging.getLogger(__name__)


@dataclass
class TrainingArtifact:
    """Everything needed to serve, audit and reproduce a trained model."""

    pipeline: Pipeline
    metrics: ModelMetrics
    metadata: dict[str, object] = field(default_factory=dict)


class ChurnModelTrainer:
    """Builds, fits, cross-validates and persists the churn pipeline."""

    def __init__(self, config: ModelConfig | None = None) -> None:
        self.config = config or SETTINGS.model
        self.pipeline: Pipeline | None = None

    # ------------------------------------------------------------ construction
    def build_pipeline(self) -> Pipeline:
        """Assemble feature engineering -> preprocessing -> calibrated classifier."""
        forest = RandomForestClassifier(
            n_estimators=self.config.n_estimators,
            max_depth=self.config.max_depth,
            min_samples_leaf=self.config.min_samples_leaf,
            class_weight=self.config.class_weight,
            random_state=self.config.random_state,
            n_jobs=-1,
        )
        # Isotonic calibration on internal CV folds: turns raw forest vote
        # fractions into probabilities the business can multiply by rupee values.
        classifier = CalibratedClassifierCV(forest, method="isotonic", cv=3)
```

```
        pipeline = Pipeline(
            [
                ("features", ChurnFeatureEngineer()),
                ("preprocess", build_preprocessor()),
                ("classifier", classifier),
            ]
        )
        logger.info("Pipeline built with steps: %s", [name for name, _ in pipeline.steps])
        return pipeline

    # ------------------------------------------------------------------ fitting
    def train(self, x_train: pd.DataFrame, y_train: pd.Series) -> Pipeline:
        if len(x_train) != len(y_train):
            raise ModelTrainingError(
                f"X has {len(x_train)} rows but y has {len(y_train)} -- refusing to train"
            )
        if pd.Series(y_train).nunique() < 2:
            logger.error("Training target contains a single class")
            raise ModelTrainingError("Training requires at least two target classes")

        logger.info("Training on %d rows (churn rate %.3f)", len(x_train), float(y_train.mean()))
        self.pipeline = self.build_pipeline()
        try:
            self.pipeline.fit(x_train, y_train)
        except Exception as exc:  # noqa: BLE001 - re-raised as a domain error
            logger.exception("Training failed")
            raise ModelTrainingError(f"Model training failed: {exc}") from exc
        logger.info("Training complete")
        return self.pipeline

    def cross_validate(self, x: pd.DataFrame, y: pd.Series, folds: int = 5) -> dict[str, float]:
        """Stratified K-fold ROC-AUC -- guards against a lucky single split."""
        cv = StratifiedKFold(n_splits=folds, shuffle=True, random_state=self.config.random_state)
        scores = cross_val_score(self.build_pipeline(), x, y, cv=cv, scoring="roc_auc", n_jobs=1)
        result = {
            "cv_folds": folds,
            "cv_roc_auc_mean": float(np.mean(scores)),
            "cv_roc_auc_std": float(np.std(scores)),
        }
        logger.info(
            "Cross-validation ROC-AUC = %.4f (+/- %.4f) over %d folds",
            result["cv_roc_auc_mean"],
            result["cv_roc_auc_std"],
            folds,
        )
        return result

    # ------------------------------------------------------------------ evaluate
    def evaluate(self, x_test: pd.DataFrame, y_test: pd.Series) -> ModelMetrics:
        if self.pipeline is None:
            raise ModelTrainingError("evaluate() called before train()")
        probabilities = self.pipeline.predict_proba(x_test)[:, 1]
        return evaluate(y_test.to_numpy(), probabilities, self.config.decision_threshold)

    # ------------------------------------------------------------------ persist
    def save(self, path: Path, metrics: ModelMetrics, extra: dict | None = None) -> Path:
        if self.pipeline is None:
            raise ModelTrainingError("save() called before train()")
        path.parent.mkdir(parents=True, exist_ok=True)
        payload = {
            "pipeline": self.pipeline,
            "metrics": metrics.to_dict(),
```

```
            "metadata": {
                "code_version": __version__,
                "trained_at_utc": datetime.now(timezone.utc).isoformat(timespec="seconds"),
                "sklearn_pipeline_steps": [name for name, _ in self.pipeline.steps],
                "input_features": SETTINGS.all_features,
                "decision_threshold": self.config.decision_threshold,
                "hyperparameters": {
                    "n_estimators": self.config.n_estimators,
                    "max_depth": self.config.max_depth,
                    "min_samples_leaf": self.config.min_samples_leaf,
                    "class_weight": self.config.class_weight,
                    "calibration": "isotonic",
                },
                **(extra or {}),
            },
        }
        joblib.dump(payload, path)
        logger.info("Artefact saved to %s (%.1f KB)", path, path.stat().st_size / 1024)
        return path

    @staticmethod
    def write_metrics_report(metrics: ModelMetrics, path: Path, extra: dict | None = None) -> None:
        path.parent.mkdir(parents=True, exist_ok=True)
        path.write_text(json.dumps({**metrics.to_dict(), **(extra or {})}, indent=2))
        logger.info("Metrics report written to %s", path)
```

`src/churnguard/models/evaluation.py` — 133 lines

Model-quality metrics across two families — discrimination and calibration — plus the executable release gates.

```
"""Model-quality metrics.

Four metrics are computed, in two families:

* **Discrimination** -- ROC-AUC and F1: can the model rank/separate churners?
* **Calibration** -- Brier score and Expected Calibration Error: when the model
  says "0.80", do 80% of those customers actually churn?

Calibration is included because the retention team multiplies the predicted
probability by customer lifetime value to decide the size of a discount offer.
A model with excellent AUC but poor calibration would systematically mis-price
those offers, so accuracy alone is not a sufficient quality gate here.
"""

from __future__ import annotations

import logging
from dataclasses import asdict, dataclass

import numpy as np
from sklearn.metrics import (
    accuracy_score,
    brier_score_loss,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)

from churnguard.config import SETTINGS

```

```
logger = logging.getLogger(__name__)


@dataclass
class ModelMetrics:
    """All model-quality numbers in one serialisable place."""

    accuracy: float
    precision: float
    recall: float
    f1: float
    roc_auc: float
    brier: float
    ece: float
    threshold: float
    n_samples: int
    confusion: list[list[int]]

    def to_dict(self) -> dict[str, object]:
        return asdict(self)

    def passes_gates(self) -> tuple[bool, list[str]]:
        """Compare against the release gates declared in ``config.QualityGates``."""
        gates = SETTINGS.gates
        failures: list[str] = []
        if self.roc_auc < gates.min_roc_auc:
            failures.append(f"roc_auc {self.roc_auc:.3f} < {gates.min_roc_auc}")
        if self.f1 < gates.min_f1:
            failures.append(f"f1 {self.f1:.3f} < {gates.min_f1}")
        if self.brier > gates.max_brier:
            failures.append(f"brier {self.brier:.3f} > {gates.max_brier}")
        return (not failures), failures


def expected_calibration_error(y_true: np.ndarray, y_prob: np.ndarray, n_bins: int = 10) -> float:
    """Weighted mean gap between confidence and observed frequency across bins."""
    y_true = np.asarray(y_true, dtype=float)
    y_prob = np.asarray(y_prob, dtype=float)
    edges = np.linspace(0.0, 1.0, n_bins + 1)
    bin_ids = np.clip(np.digitize(y_prob, edges[1:-1], right=True), 0, n_bins - 1)
    error = 0.0
    for b in range(n_bins):
        mask = bin_ids == b
        if not mask.any():
            continue
        error += (mask.mean()) * abs(y_prob[mask].mean() - y_true[mask].mean())
    return float(error)


def reliability_curve(
    y_true: np.ndarray, y_prob: np.ndarray, n_bins: int = 10
) -> tuple[np.ndarray, np.ndarray]:
    """Bin centres (mean predicted) and observed churn rate, for the calibration plot."""
    y_true = np.asarray(y_true, dtype=float)
    y_prob = np.asarray(y_prob, dtype=float)
    edges = np.linspace(0.0, 1.0, n_bins + 1)
    bin_ids = np.clip(np.digitize(y_prob, edges[1:-1], right=True), 0, n_bins - 1)
    predicted, observed = [], []
    for b in range(n_bins):
        mask = bin_ids == b
        if mask.sum() < 5:
            continue
```

```
        predicted.append(y_prob[mask].mean())
        observed.append(y_true[mask].mean())
    return np.asarray(predicted), np.asarray(observed)


def evaluate(
    y_true: np.ndarray, y_prob: np.ndarray, threshold: float | None = None
) -> ModelMetrics:
    """Compute every model-quality metric for one set of predictions."""
    threshold = SETTINGS.model.decision_threshold if threshold is None else threshold
    y_true = np.asarray(y_true).astype(int)
    y_prob = np.asarray(y_prob, dtype=float)
    if y_true.shape != y_prob.shape:
        raise ValueError(f"Shape mismatch: y_true {y_true.shape} vs y_prob {y_prob.shape}")

    y_pred = (y_prob >= threshold).astype(int)
    metrics = ModelMetrics(
        accuracy=float(accuracy_score(y_true, y_pred)),
        precision=float(precision_score(y_true, y_pred, zero_division=0)),
        recall=float(recall_score(y_true, y_pred, zero_division=0)),
        f1=float(f1_score(y_true, y_pred, zero_division=0)),
        roc_auc=float(roc_auc_score(y_true, y_prob)),
        brier=float(brier_score_loss(y_true, y_prob)),
        ece=expected_calibration_error(y_true, y_prob),
        threshold=float(threshold),
        n_samples=int(y_true.size),
        confusion=confusion_matrix(y_true, y_pred).tolist(),
    )
    logger.info(
        "Evaluation @thr=%.2f | AUC=%.4f F1=%.4f acc=%.4f Brier=%.4f ECE=%.4f",
        threshold,
        metrics.roc_auc,
        metrics.f1,
        metrics.accuracy,
        metrics.brier,
        metrics.ece,
    )
    return metrics
```

`src/churnguard/models/predictor.py` — 161 lines

Inference service: loads the artefact once, validates input, scores, and maps probabilities onto the retention team's action playbook.

```
"""Inference service -- the only object the API layer talks to.

Responsibilities kept deliberately narrow: load the artefact once, validate that
incoming records satisfy the contract, produce a probability, a binary decision,
a human-readable risk band and a recommended retention action.
"""

from __future__ import annotations

import logging
from dataclasses import asdict, dataclass
from pathlib import Path

import joblib
import numpy as np
import pandas as pd

```

```
from churnguard.config import SETTINGS
from churnguard.exceptions import InferenceError, ModelNotLoadedError

logger = logging.getLogger(__name__)

RISK_BANDS: tuple[tuple[float, str, str], ...] = (
    (0.30, "LOW", "No action - monitor quarterly"),
    (0.60, "MEDIUM", "Add to nurture campaign; offer plan review"),
    (0.85, "HIGH", "Assign retention agent within 7 days"),
    (1.01, "CRITICAL", "Immediate outreach with targeted discount offer"),
)


@dataclass(frozen=True)
class Prediction:
    """One scored customer."""

    customer_id: str | None
    churn_probability: float
    churn_prediction: int
    risk_band: str
    recommended_action: str
    threshold_used: float
    model_version: str

    def to_dict(self) -> dict[str, object]:
        return asdict(self)


def assign_risk_band(probability: float) -> tuple[str, str]:
    """Map a probability onto the retention team's action playbook."""
    for upper, band, action in RISK_BANDS:
        if probability < upper:
            return band, action
    return RISK_BANDS[-1][1], RISK_BANDS[-1][2]


class ChurnPredictor:
    """Loads a persisted pipeline and serves predictions."""

    def __init__(self, artifact_path: Path | None = None, threshold: float | None = None) -> None:
        self.artifact_path = artifact_path or SETTINGS.paths.model_artifact
        self.threshold = SETTINGS.model.decision_threshold if threshold is None else threshold
        self._pipeline = None
        self._metadata: dict = {}
        self._metrics: dict = {}

    # ------------------------------------------------------------------ state
    @property
    def is_loaded(self) -> bool:
        return self._pipeline is not None

    @property
    def metadata(self) -> dict:
        return dict(self._metadata)

    @property
    def metrics(self) -> dict:
        return dict(self._metrics)

    @property
    def model_version(self) -> str:
```

```
        return str(self._metadata.get("code_version", "unknown"))

    def load(self) -> ChurnPredictor:
        logger.info("Loading model artefact from %s", self.artifact_path)
        if not Path(self.artifact_path).exists():
            logger.error("Model artefact not found at %s", self.artifact_path)
            raise ModelNotLoadedError(f"Model artefact not found: {self.artifact_path}")
        try:
            payload = joblib.load(self.artifact_path)
        except (OSError, EOFError, KeyError, ValueError) as exc:
            logger.error("Corrupt model artefact %s: %s", self.artifact_path, exc)
            raise ModelNotLoadedError(f"Could not deserialise artefact: {exc}") from exc

        self._pipeline = payload["pipeline"]
        self._metadata = payload.get("metadata", {})
        self._metrics = payload.get("metrics", {})
        logger.info(
            "Model v%s loaded (trained %s)",
            self.model_version,
            self._metadata.get("trained_at_utc", "unknown"),
        )
        return self

    def _require_model(self):
        if self._pipeline is None:
            raise ModelNotLoadedError("Model is not loaded. Call load() first.")
        return self._pipeline

    # ------------------------------------------------------------- inference
    def predict_proba(self, frame: pd.DataFrame) -> np.ndarray:
        """Return P(churn) for each row. Raises ``InferenceError`` on bad input."""
        pipeline = self._require_model()
        if frame.empty:
            raise InferenceError("Received an empty batch")
        missing = [c for c in SETTINGS.all_features if c not in frame.columns]
        if missing:
            logger.error("Inference payload missing features: %s", missing)
            raise InferenceError(f"Missing required features: {missing}")
        try:
            probabilities = pipeline.predict_proba(frame[SETTINGS.all_features])[:, 1]
        except Exception as exc:  # noqa: BLE001 - surfaced to the API as 422
            logger.exception("Inference failed for a batch of %d rows", len(frame))
            raise InferenceError(f"Inference failed: {exc}") from exc

        if not np.all((probabilities >= 0.0) & (probabilities <= 1.0)):
            logger.error("Model returned out-of-range probabilities -- refusing to serve")
            raise InferenceError("Model produced probabilities outside [0, 1]")
        return probabilities

    def predict(self, records: pd.DataFrame | list[dict]) -> list[Prediction]:
        frame = pd.DataFrame(records) if not isinstance(records, pd.DataFrame) else records
        probabilities = self.predict_proba(frame)
        ids = (
            frame[SETTINGS.model.id_column].astype(str).tolist()
            if SETTINGS.model.id_column in frame.columns
            else [None] * len(frame)
        )
        predictions = []
        for customer_id, probability in zip(ids, probabilities, strict=True):
            band, action = assign_risk_band(float(probability))
            predictions.append(
                Prediction(
```

```
                    customer_id=customer_id,
                    churn_probability=round(float(probability), 4),
                    churn_prediction=int(probability >= self.threshold),
                    risk_band=band,
                    recommended_action=action,
                    threshold_used=self.threshold,
                    model_version=self.model_version,
                )
            )
        logger.info(
            "Scored %d record(s) | mean P(churn)=%.4f | flagged=%d",
            len(predictions),
            float(np.mean(probabilities)),
            sum(p.churn_prediction for p in predictions),
        )
        return predictions

    def predict_one(self, record: dict) -> Prediction:
        return self.predict([record])[0]
```

`src/churnguard/api/schemas.py` — 121 lines

Pydantic request and response contracts. This file is the API's first security boundary.

```
"""Pydantic request/response models -- the public contract of the API.

Every constraint declared here is enforced *before* a single row reaches the
model. This is the system's first security boundary: it rejects out-of-range
numbers, unknown category values and oversized batches, which is what stops both
accidental garbage and deliberate adversarial probing.
"""

from __future__ import annotations

from typing import Annotated, Literal

from pydantic import BaseModel, ConfigDict, Field, field_validator

ContractType = Literal["Month-to-month", "One year", "Two year"]
InternetService = Literal["Fiber optic", "DSL", "No"]
PaymentMethod = Literal["Electronic check", "Mailed check", "Bank transfer", "Credit card"]
YesNo = Literal["Yes", "No"]

MAX_BATCH_SIZE = 500


class CustomerFeatures(BaseModel):
    """One customer record to be scored."""

    model_config = ConfigDict(
        extra="forbid",  # unknown keys are a client bug -> 422, never silently ignored
        json_schema_extra={
            "example": {
                "customer_id": "CUST-004821",
                "tenure_months": 3,
                "monthly_charges": 88.4,
                "total_charges": 265.2,
                "support_tickets_6m": 4,
                "avg_monthly_gb": 41.5,
                "contract_type": "Month-to-month",
                "internet_service": "Fiber optic",
                "payment_method": "Electronic check",
```

```
                "tech_support": "No",
                "paperless_billing": "Yes",
            }
        },
    )

    customer_id: str | None = Field(
        default=None, max_length=64, description="Opaque customer reference, echoed back."
    )
    tenure_months: Annotated[int, Field(ge=0, le=120, description="Months since activation.")]
    monthly_charges: Annotated[float, Field(ge=0, le=500, description="Current monthly bill.")]
    total_charges: Annotated[float | None, Field(ge=0, le=60_000)] = None
    support_tickets_6m: Annotated[int, Field(ge=0, le=60)]
    avg_monthly_gb: Annotated[float | None, Field(ge=0, le=1_000)] = None
    contract_type: ContractType
    internet_service: InternetService
    payment_method: PaymentMethod
    tech_support: YesNo
    paperless_billing: YesNo

    @field_validator("customer_id")
    @classmethod
    def reject_control_characters(cls, value: str | None) -> str | None:
        """Defensive: block control characters that could poison downstream logs."""
        if value is not None and any(ord(ch) < 32 for ch in value):
            raise ValueError("customer_id must not contain control characters")
        return value


class BatchPredictionRequest(BaseModel):
    """A bounded batch. The upper bound protects the service from memory blow-ups."""

    model_config = ConfigDict(extra="forbid")

    customers: Annotated[list[CustomerFeatures], Field(min_length=1, max_length=MAX_BATCH_SIZE)]


class PredictionResponse(BaseModel):
    """Scoring result for one customer."""

    customer_id: str | None = None
    churn_probability: float = Field(ge=0.0, le=1.0)
    churn_prediction: int = Field(ge=0, le=1)
    risk_band: Literal["LOW", "MEDIUM", "HIGH", "CRITICAL"]
    recommended_action: str
    threshold_used: float
    model_version: str


class BatchPredictionResponse(BaseModel):
    """Batch result plus a small roll-up the caller would otherwise recompute."""

    predictions: list[PredictionResponse]
    count: int
    flagged_count: int
    mean_churn_probability: float


class HealthResponse(BaseModel):
    status: Literal["ok", "degraded"]
    model_loaded: bool
    model_version: str
```

```
    api_version: str


class ModelInfoResponse(BaseModel):
    model_config = ConfigDict(protected_namespaces=())

    model_version: str
    trained_at_utc: str | None = None
    decision_threshold: float
    input_features: list[str]
    hyperparameters: dict = Field(default_factory=dict)
    test_metrics: dict = Field(default_factory=dict)


class ErrorResponse(BaseModel):
    """Uniform error envelope so clients parse one shape for every failure."""

    error: str
    detail: str
    violations: list[str] = Field(default_factory=list)
```

`src/churnguard/api/main.py` — 208 lines

The FastAPI application: lifespan model loading, timing middleware, exception handlers and four routes.

```
"""FastAPI application exposing the churn model.

API design decisions worth noting:

* **Versioned prefix** (``/v1``) so a breaking schema change can ship as ``/v2``
  while existing clients keep working.
* **The model is loaded once**, in the lifespan handler -- not per request. A
  19 MB artefact deserialised on every call would dominate latency.
* **Correct status codes**: 200 success, 422 schema violation (FastAPI default),
  400 semantically invalid input the model rejects, 503 model unavailable.
* **Uniform error envelope** via exception handlers, so clients never have to
  parse two different error shapes.
* **Health is split from readiness**: ``/health`` reports liveness *and* whether
  the model is loaded, which is what a Kubernetes readiness probe needs to avoid
  routing traffic to a pod whose artefact failed to load.
"""

from __future__ import annotations

import logging
import time
from contextlib import asynccontextmanager

from fastapi import FastAPI, Request, Response, status
from fastapi.responses import JSONResponse

from churnguard import __version__
from churnguard.api.schemas import (
    BatchPredictionRequest,
    BatchPredictionResponse,
    CustomerFeatures,
    ErrorResponse,
    HealthResponse,
    ModelInfoResponse,
    PredictionResponse,
)
from churnguard.config import SETTINGS
```

```
from churnguard.exceptions import (
    ChurnGuardError,
    InferenceError,
    ModelNotLoadedError,
    SchemaValidationError,
)
from churnguard.logging_config import configure_logging
from churnguard.models.predictor import ChurnPredictor

logger = logging.getLogger(__name__)
predictor = ChurnPredictor()


@asynccontextmanager
async def lifespan(app: FastAPI):
    """Load the model once at startup; degrade gracefully if it is absent."""
    configure_logging()
    logger.info("API starting up -- loading model")
    try:
        predictor.load()
    except ModelNotLoadedError as exc:
        # Do NOT crash: the pod should start, fail its readiness probe and be
        # visible in monitoring, rather than crash-looping with no diagnostics.
        logger.error("Startup completed WITHOUT a model: %s", exc)
    yield
    logger.info("API shutting down")


app = FastAPI(
    title="ChurnGuard Inference API",
    description="Serves calibrated churn-risk scores for telecom subscribers.",
    version=__version__,
    lifespan=lifespan,
)


# ------------------------------------------------------------------ middleware
@app.middleware("http")
async def add_timing_header(request: Request, call_next):
    """Record server-side latency -- the primary operational SLI for this service."""
    started = time.perf_counter()
    response: Response = await call_next(request)
    elapsed_ms = (time.perf_counter() - started) * 1000
    response.headers["X-Process-Time-Ms"] = f"{elapsed_ms:.2f}"
    logger.info(
        "%s %s -> %d in %.2f ms",
        request.method,
        request.url.path,
        response.status_code,
        elapsed_ms,
    )
    return response


# ------------------------------------------------------------- error handlers
@app.exception_handler(ModelNotLoadedError)
async def handle_model_not_loaded(request: Request, exc: ModelNotLoadedError) -> JSONResponse:
    logger.error("503 on %s -- model unavailable: %s", request.url.path, exc)
    return JSONResponse(
        status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
        content=ErrorResponse(
            error="model_unavailable",
```

```
            detail="The model artefact is not loaded. Retry after the service is ready.",
        ).model_dump(),
    )


@app.exception_handler(SchemaValidationError)
async def handle_schema_error(request: Request, exc: SchemaValidationError) -> JSONResponse:
    logger.warning("400 on %s -- contract breach: %s", request.url.path, exc)
    return JSONResponse(
        status_code=status.HTTP_400_BAD_REQUEST,
        content=ErrorResponse(
            error="data_contract_violation", detail=str(exc), violations=exc.violations
        ).model_dump(),
    )


@app.exception_handler(InferenceError)
async def handle_inference_error(request: Request, exc: InferenceError) -> JSONResponse:
    logger.warning("400 on %s -- inference rejected: %s", request.url.path, exc)
    return JSONResponse(
        status_code=status.HTTP_400_BAD_REQUEST,
        content=ErrorResponse(error="inference_failed", detail=str(exc)).model_dump(),
    )


@app.exception_handler(ChurnGuardError)
async def handle_generic_domain_error(request: Request, exc: ChurnGuardError) -> JSONResponse:
    logger.exception("500 on %s", request.url.path)
    return JSONResponse(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        content=ErrorResponse(
            error="internal_error", detail="An unexpected internal error occurred."
        ).model_dump(),
    )


# ---------------------------------------------------------------------- routes
@app.get("/health", response_model=HealthResponse, tags=["operations"])
async def health() -> HealthResponse:
    """Liveness + readiness. Always 200 so the probe can read the body."""
    return HealthResponse(
        status="ok" if predictor.is_loaded else "degraded",
        model_loaded=predictor.is_loaded,
        model_version=predictor.model_version,
        api_version=SETTINGS.api_version,
    )


@app.get(f"/{SETTINGS.api_version}/model/info", response_model=ModelInfoResponse, tags=["model"])
async def model_info() -> ModelInfoResponse:
    """Model card endpoint -- lets clients and auditors see exactly what is serving."""
    if not predictor.is_loaded:
        raise ModelNotLoadedError("No model loaded")
    metadata = predictor.metadata
    return ModelInfoResponse(
        model_version=predictor.model_version,
        trained_at_utc=metadata.get("trained_at_utc"),
        decision_threshold=float(metadata.get("decision_threshold", predictor.threshold)),
        input_features=list(metadata.get("input_features", SETTINGS.all_features)),
        hyperparameters=metadata.get("hyperparameters", {}),
        test_metrics={
            k: v for k, v in predictor.metrics.items() if k not in {"confusion", "n_samples"}
```

```
        },
    )


@app.post(
    f"/{SETTINGS.api_version}/predict",
    response_model=PredictionResponse,
    status_code=status.HTTP_200_OK,
    tags=["inference"],
    responses={
        400: {"model": ErrorResponse, "description": "Input rejected by the model"},
        422: {"model": ErrorResponse, "description": "Request failed schema validation"},
        503: {"model": ErrorResponse, "description": "Model not loaded"},
    },
)
async def predict(customer: CustomerFeatures) -> PredictionResponse:
    """Score a single customer and return a risk band plus a recommended action."""
    if not predictor.is_loaded:
        raise ModelNotLoadedError("No model loaded")
    result = predictor.predict_one(customer.model_dump())
    return PredictionResponse(**result.to_dict())


@app.post(
    f"/{SETTINGS.api_version}/predict/batch",
    response_model=BatchPredictionResponse,
    status_code=status.HTTP_200_OK,
    tags=["inference"],
    responses={
        400: {"model": ErrorResponse},
        422: {"model": ErrorResponse},
        503: {"model": ErrorResponse},
    },
)
async def predict_batch(request: BatchPredictionRequest) -> BatchPredictionResponse:
    """Score up to 500 customers in one call -- used by the nightly campaign job."""
    if not predictor.is_loaded:
        raise ModelNotLoadedError("No model loaded")
    results = predictor.predict([c.model_dump() for c in request.customers])
    probabilities = [r.churn_probability for r in results]
    return BatchPredictionResponse(
        predictions=[PredictionResponse(**r.to_dict()) for r in results],
        count=len(results),
        flagged_count=sum(r.churn_prediction for r in results),
        mean_churn_probability=round(sum(probabilities) / len(probabilities), 4),
    )
```

Appendix B — Test Suite (tests)

98 tests across four categories, authored by Member 4 and reviewed by the module owners. The suite runs in 16.9 seconds and covers 90% of package statements.

`tests/conftest.py` — 93 lines

Shared session-scoped fixtures. The model is trained once and reused across all 38 ML tests, which is what keeps the suite fast enough to run on every commit.

```python
"""Shared pytest fixtures.

Session-scoped fixtures matter here: training a pipeline takes seconds, so the
model is fitted **once** and shared across every ML test. Without this the suite
would be too slow to run on every commit, and a slow suite is a suite that gets
skipped.
"""

from __future__ import annotations

import sys
from pathlib import Path

import pandas as pd
import pytest

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
sys.path.insert(0, str(ROOT / "scripts"))

from churnguard.config import SETTINGS  # noqa: E402
from churnguard.data.ingestion import DataIngestor, InMemoryDataSource  # noqa: E402
from churnguard.logging_config import configure_logging  # noqa: E402
from churnguard.models.trainer import ChurnModelTrainer  # noqa: E402
from generate_data import build_dataset  # noqa: E402


@pytest.fixture(scope="session", autouse=True)
def _logging():
    configure_logging(level="WARNING")


@pytest.fixture(scope="session")
def raw_frame() -> pd.DataFrame:
    """A deterministic 2,500-row dataset.

    Size is a deliberate trade-off: below ~2,000 rows the model no longer clears
    the production quality gates, so a smaller fixture would make
    ``test_trained_model_clears_release_gates`` fail for reasons of sample size
    rather than genuine model regression. 2,500 rows keeps the whole suite under
    ~20 s while still being statistically meaningful.
    """
    return build_dataset(n_rows=2500, seed=7)


@pytest.fixture(scope="session")
def split(raw_frame):
    return DataIngestor(InMemoryDataSource(raw_frame, "pytest")).split()


@pytest.fixture(scope="session")
```

```python
def trained_trainer(split):
    trainer = ChurnModelTrainer()
    trainer.train(split.x_train, split.y_train)
    return trainer


@pytest.fixture(scope="session")
def trained_pipeline(trained_trainer):
    return trained_trainer.pipeline


@pytest.fixture(scope="session")
def artifact_path(tmp_path_factory, trained_trainer, split) -> Path:
    """Persist the session model so predictor/API tests exercise real (de)serialisation."""
    path = tmp_path_factory.mktemp("artifacts") / "model.joblib"
    metrics = trained_trainer.evaluate(split.x_test, split.y_test)
    trained_trainer.save(path, metrics)
    return path


@pytest.fixture
def valid_payload() -> dict:
    """A schema-valid API request body."""
    return {
        "customer_id": "CUST-000001",
        "tenure_months": 3,
        "monthly_charges": 88.4,
        "total_charges": 265.2,
        "support_tickets_6m": 4,
        "avg_monthly_gb": 41.5,
        "contract_type": "Month-to-month",
        "internet_service": "Fiber optic",
        "payment_method": "Electronic check",
        "tech_support": "No",
        "paperless_billing": "Yes",
    }


@pytest.fixture
def feature_frame(raw_frame) -> pd.DataFrame:
    return raw_frame[SETTINGS.all_features].head(50).copy()
```

`tests/test_data_ingestion.py` — 86 lines

13 unit tests — data source behaviour, failure modes, split correctness, stratification and reproducibility.

```python
"""UNIT TESTS -- data ingestion layer (fast, no I/O beyond tmp_path)."""

from __future__ import annotations

import pandas as pd
import pytest

from churnguard.data.ingestion import (
    CsvDataSource,
    DataIngestor,
    DataSplit,
    InMemoryDataSource,
)
from churnguard.exceptions import DataIngestionError


```

```
class TestCsvDataSource:
    def test_reads_a_valid_csv(self, raw_frame, tmp_path):
        path = tmp_path / "data.csv"
        raw_frame.to_csv(path, index=False)
        loaded = CsvDataSource(path).load()
        assert loaded.shape == raw_frame.shape

    def test_missing_file_raises_domain_error(self, tmp_path):
        with pytest.raises(DataIngestionError, match="not found"):
            CsvDataSource(tmp_path / "nope.csv").load()

    def test_empty_file_raises(self, tmp_path):
        path = tmp_path / "empty.csv"
        path.write_text("")
        with pytest.raises(DataIngestionError, match="empty"):
            CsvDataSource(path).load()

    def test_header_only_file_raises(self, tmp_path):
        path = tmp_path / "header.csv"
        path.write_text("customer_id,churn\n")
        with pytest.raises(DataIngestionError, match="No rows"):
            CsvDataSource(path).load()

    def test_name_is_traceable(self, tmp_path):
        assert CsvDataSource(tmp_path / "a.csv").name.startswith("csv://")


class TestDataIngestor:
    def test_rejects_frame_missing_required_columns(self, raw_frame):
        broken = raw_frame.drop(columns=["contract_type"])
        ingestor = DataIngestor(InMemoryDataSource(broken))
        with pytest.raises(DataIngestionError, match="Missing required columns"):
            ingestor.load_raw()

    def test_drops_duplicate_customer_ids(self, raw_frame):
        duplicated = pd.concat([raw_frame, raw_frame.head(10)], ignore_index=True)
        result = DataIngestor(InMemoryDataSource(duplicated)).load_raw()
        assert len(result) == len(raw_frame)

    def test_split_sizes_respect_test_size(self, split, raw_frame):
        assert len(split.x_test) == pytest.approx(0.2 * len(raw_frame), abs=2)
        assert len(split.x_train) + len(split.x_test) == len(raw_frame)

    def test_split_is_stratified(self, split):
        """Train and test churn rates must match within 2 percentage points."""
        assert abs(split.y_train.mean() - split.y_test.mean()) < 0.02

    def test_no_row_leaks_between_train_and_test(self, split):
        assert not set(split.x_train.index) & set(split.x_test.index)

    def test_split_is_reproducible(self, raw_frame):
        a = DataIngestor(InMemoryDataSource(raw_frame)).split()
        b = DataIngestor(InMemoryDataSource(raw_frame)).split()
        assert list(a.x_test.index) == list(b.x_test.index)

    def test_single_class_target_is_rejected(self, raw_frame):
        constant = raw_frame.copy()
        constant["churn"] = 0
        with pytest.raises(DataIngestionError, match="two classes"):
            DataIngestor(InMemoryDataSource(constant)).split()

    def test_summary_reports_expected_keys(self, split):
```

```
        assert set(DataSplit.summary(split)) == {
            "n_train",
            "n_test",
            "n_features",
            "train_churn_rate",
            "test_churn_rate",
        }
```

`tests/test_features.py` — 80 lines

13 unit tests — arithmetic correctness of each pure feature function, purity (non-mutation), boundary conditions and transformer contract.

```
"""UNIT TESTS -- feature engineering (pure functions and the sklearn transformer)."""

from __future__ import annotations

import numpy as np
import pandas as pd
import pytest

from churnguard.exceptions import FeatureEngineeringError
from churnguard.features.engineering import (
    DERIVED_FEATURES,
    ChurnFeatureEngineer,
    apply_derived_features,
    avg_spend_per_month,
    charge_to_usage_ratio,
    is_new_customer,
    tickets_per_year,
)


class TestPureFeatureFunctions:
    def test_avg_spend_per_month_is_arithmetically_correct(self):
        frame = pd.DataFrame({"total_charges": [1200.0], "tenure_months": [12]})
        assert avg_spend_per_month(frame).iloc[0] == pytest.approx(100.0)

    def test_avg_spend_per_month_never_divides_by_zero(self):
        frame = pd.DataFrame({"total_charges": [50.0], "tenure_months": [0]})
        assert np.isfinite(avg_spend_per_month(frame).iloc[0])

    def test_tickets_per_year_annualises(self):
        assert tickets_per_year(pd.DataFrame({"support_tickets_6m": [3]})).iloc[0] == 6.0

    def test_is_new_customer_boundary_at_six_months(self):
        frame = pd.DataFrame({"tenure_months": [5, 6, 7]})
        assert is_new_customer(frame).tolist() == [1, 1, 0]

    def test_charge_to_usage_ratio_handles_null_usage(self):
        frame = pd.DataFrame({"monthly_charges": [60.0], "avg_monthly_gb": [np.nan]})
        assert charge_to_usage_ratio(frame).iloc[0] == pytest.approx(60.0)

    def test_functions_are_pure_and_do_not_mutate_input(self, feature_frame):
        before = feature_frame.copy(deep=True)
        apply_derived_features(feature_frame)
        pd.testing.assert_frame_equal(feature_frame, before)


class TestApplyDerivedFeatures:
    def test_all_registered_features_are_produced(self, feature_frame):
```

```
        out = apply_derived_features(feature_frame)
        assert set(DERIVED_FEATURES).issubset(out.columns)

    def test_row_count_is_preserved(self, feature_frame):
        assert len(apply_derived_features(feature_frame)) == len(feature_frame)

    def test_missing_input_column_raises_domain_error(self, feature_frame):
        with pytest.raises(FeatureEngineeringError, match="needs column"):
            apply_derived_features(feature_frame.drop(columns=["total_charges"]))


class TestChurnFeatureEngineer:
    def test_fit_transform_shape(self, feature_frame):
        out = ChurnFeatureEngineer().fit_transform(feature_frame)
        assert out.shape == (len(feature_frame), feature_frame.shape[1] + len(DERIVED_FEATURES))

    def test_transform_before_fit_raises(self, feature_frame):
        with pytest.raises(FeatureEngineeringError, match="before fit"):
            ChurnFeatureEngineer().transform(feature_frame)

    def test_column_order_is_stable_under_shuffled_input(self, feature_frame):
        """A client sending JSON keys in a different order must get the same matrix."""
        engineer = ChurnFeatureEngineer().fit(feature_frame)
        shuffled = feature_frame[list(reversed(feature_frame.columns))]
        pd.testing.assert_frame_equal(
            engineer.transform(feature_frame), engineer.transform(shuffled)
        )

    def test_non_dataframe_input_is_rejected(self, feature_frame):
        with pytest.raises(FeatureEngineeringError):
            ChurnFeatureEngineer().fit(feature_frame.to_numpy())
```

`tests/test_data_validation.py` — 123 lines

15 data-validation tests — schema breaches, dtype changes, unexpected category levels, missing-value floods, unit changes and drift.

```
"""DATA VALIDATION TESTS -- the data contract, missing values, ranges and drift.

These are not tests of Python code so much as tests of *data assumptions*. Each
one corresponds to a failure mode that has genuinely broken ML systems in
production: a renamed column, a silent null flood, a unit change (dollars ->
cents), and gradual population drift.
"""

from __future__ import annotations

import numpy as np
import pytest

from churnguard.data.validation import (
    DataValidator,
    ValidationReport,
    population_stability_index,
    write_reference_profile,
)
from churnguard.exceptions import SchemaValidationError


class TestSchemaContract:
    def test_clean_training_data_passes(self, raw_frame):
```

```
        report = DataValidator().validate(raw_frame)
        assert report.passed, report.errors

    def test_renamed_column_is_caught(self, raw_frame):
        broken = raw_frame.rename(columns={"tenure_months": "tenure"})
        report = DataValidator().validate(broken)
        assert not report.passed
        assert any("tenure_months" in e for e in report.errors)

    def test_wrong_dtype_is_caught(self, raw_frame):
        broken = raw_frame.copy()
        broken["monthly_charges"] = broken["monthly_charges"].astype(str)
        report = DataValidator().validate(broken)
        assert not report.passed
        assert any("must be numeric" in e for e in report.errors)

    def test_unexpected_category_level_is_caught(self, raw_frame):
        broken = raw_frame.copy()
        broken.loc[broken.index[0], "contract_type"] = "Lifetime"
        report = DataValidator().validate(broken)
        assert not report.passed
        assert any("Lifetime" in e for e in report.errors)


class TestMissingValues:
    def test_low_missingness_is_only_a_warning(self, raw_frame):
        report = DataValidator().validate(raw_frame)
        assert report.passed
        assert report.warnings  # the generator injects ~1.2% nulls

    def test_missing_flood_fails_the_gate(self, raw_frame):
        broken = raw_frame.copy()
        broken.loc[broken.index[:600], "monthly_charges"] = np.nan  # 50% null
        report = DataValidator().validate(broken)
        assert not report.passed
        assert any("null" in e for e in report.errors)

    def test_per_column_missing_metric_is_recorded(self, raw_frame):
        report = DataValidator().validate(raw_frame)
        assert report.metrics["missing_fraction__total_charges"] > 0
        assert report.metrics["missing_fraction__tenure_months"] == 0.0


class TestRangeChecks:
    def test_negative_charges_are_rejected(self, raw_frame):
        broken = raw_frame.copy()
        broken.loc[broken.index[0], "monthly_charges"] = -12.0
        report = DataValidator().validate(broken)
        assert not report.passed
        assert any("outside" in e for e in report.errors)

    def test_unit_change_dollars_to_cents_is_rejected(self, raw_frame):
        """A silent x100 unit change is the classic upstream-pipeline bug."""
        broken = raw_frame.copy()
        broken["monthly_charges"] = broken["monthly_charges"] * 100
        report = DataValidator().validate(broken)
        assert not report.passed


class TestDriftDetection:
    def test_psi_of_identical_distributions_is_near_zero(self):
        rng = np.random.default_rng(0)
```

```
        sample = rng.normal(50, 10, 5000)
        assert population_stability_index(sample, sample.copy()) < 0.01

    def test_psi_grows_with_a_mean_shift(self):
        rng = np.random.default_rng(1)
        reference = rng.normal(50, 10, 5000)
        small = population_stability_index(reference, rng.normal(52, 10, 5000))
        large = population_stability_index(reference, rng.normal(70, 10, 5000))
        assert small < large
        assert large > 0.25  # "significant shift" by the standard convention

    def test_drifted_traffic_fails_validation(self, raw_frame, tmp_path):
        profile = write_reference_profile(raw_frame, tmp_path / "profile.json")
        drifted = raw_frame.copy()
        drifted["tenure_months"] = (drifted["tenure_months"] * 0.25).round().astype(int)
        report = DataValidator().validate(drifted, reference_profile=profile)
        assert not report.passed
        assert any("drifted" in e for e in report.errors)
        assert report.metrics["psi__tenure_months"] > 0.20

    def test_stable_traffic_passes_drift_check(self, raw_frame, tmp_path):
        profile = write_reference_profile(raw_frame, tmp_path / "profile.json")
        report = DataValidator().validate(raw_frame.sample(frac=0.5, random_state=3), profile)
        assert report.passed, report.errors


class TestReportBehaviour:
    def test_raise_for_status_raises_on_failure(self):
        report = ValidationReport()
        report.add_error("boom")
        with pytest.raises(SchemaValidationError) as exc:
            report.raise_for_status()
        assert exc.value.violations == ["boom"]

    def test_raise_for_status_is_silent_on_success(self):
        ValidationReport().raise_for_status()
```

`tests/test_model_training.py` — 160 lines

11 ML behavioural tests for training — overfit-a-small-batch, monotonic loss decrease, baseline comparison, target-leakage detection, reproducibility and gate enforcement.

```
"""ML BEHAVIOURAL TESTS -- model TRAINING.

Standard unit tests cannot tell you whether a model *learned*. These tests assert
properties of the learning process itself:

* **Overfit-a-small-batch** -- the canonical ML smoke test. If a high-capacity
  model cannot memorise 60 rows, the wiring (labels, features, fit call) is
  broken, no matter what the accuracy on the full set looks like.
* **Loss decreases** -- log-loss must fall monotonically-ish as the learner is
  given more capacity/data; if it does not, training is not actually optimising.
* **Beats a trivial baseline** -- a model must be better than always predicting
  the majority class, or it has no business value.
* **Reproducibility** -- same seed, same model. Without this, no experiment is
  comparable and no regression is diagnosable.
"""

from __future__ import annotations

import numpy as np
```

```
import pandas as pd
import pytest
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.pipeline import Pipeline

from churnguard.config import SETTINGS
from churnguard.exceptions import ModelTrainingError
from churnguard.features.engineering import ChurnFeatureEngineer, build_preprocessor
from churnguard.models.trainer import ChurnModelTrainer


class TestTrainingContract:
    def test_pipeline_has_the_expected_three_stages(self):
        steps = [name for name, _ in ChurnModelTrainer().build_pipeline().steps]
        assert steps == ["features", "preprocess", "classifier"]

    def test_mismatched_x_and_y_lengths_are_rejected(self, split):
        with pytest.raises(ModelTrainingError, match="refusing to train"):
            ChurnModelTrainer().train(split.x_train, split.y_train.iloc[:-5])

    def test_single_class_target_is_rejected(self, split):
        with pytest.raises(ModelTrainingError, match="two target classes"):
            ChurnModelTrainer().train(split.x_train, pd.Series(np.zeros(len(split.x_train))))

    def test_evaluate_before_train_is_rejected(self, split):
        with pytest.raises(ModelTrainingError, match="before train"):
            ChurnModelTrainer().evaluate(split.x_test, split.y_test)


class TestLearningActuallyHappens:
    def test_model_overfits_a_small_batch(self, split):
        """THE canonical ML smoke test: an unregularised forest must memorise 60 rows.

        Regularisation is deliberately switched off (unlimited depth, leaf size 1)
        because the assertion is about *capacity to fit*, not generalisation.
        """
        x_small = split.x_train.head(60)
        y_small = split.y_train.head(60)
        pipeline = Pipeline(
            [
                ("features", ChurnFeatureEngineer()),
                ("preprocess", build_preprocessor()),
                (
                    "classifier",
                    RandomForestClassifier(
                        n_estimators=200, max_depth=None, min_samples_leaf=1, random_state=0
                    ),
                ),
            ]
        )
        pipeline.fit(x_small, y_small)
        train_accuracy = pipeline.score(x_small, y_small)
        assert train_accuracy >= 0.98, f"Cannot memorise 60 rows (acc={train_accuracy:.3f})"

    def test_training_loss_decreases_with_model_capacity(self, split):
        """Log-loss on the training set must fall as trees are added."""
        losses = []
        for n_estimators in (1, 5, 25, 100):
            pipeline = Pipeline(
                [
```

```
                    ("features", ChurnFeatureEngineer()),
                    ("preprocess", build_preprocessor()),
                    (
                        "classifier",
                        RandomForestClassifier(
                            n_estimators=n_estimators,
                            max_depth=None,
                            min_samples_leaf=1,
                            random_state=0,
                        ),
                    ),
                ]
            )
            pipeline.fit(split.x_train, split.y_train)
            probabilities = pipeline.predict_proba(split.x_train)[:, 1]
            losses.append(log_loss(split.y_train, probabilities, labels=[0, 1]))
        assert losses[-1] < losses[0], f"Loss did not decrease: {losses}"
        assert losses == sorted(losses, reverse=True), f"Loss not monotonically falling: {losses}"

    def test_model_beats_the_majority_class_baseline(self, trained_pipeline, split):
        baseline = DummyClassifier(strategy="most_frequent").fit(split.x_train, split.y_train)
        baseline_auc = roc_auc_score(split.y_test, baseline.predict_proba(split.x_test)[:, 1])
        model_auc = roc_auc_score(split.y_test, trained_pipeline.predict_proba(split.x_test)[:, 1])
        assert model_auc > baseline_auc + 0.15, f"model {model_auc:.3f} vs dummy {baseline_auc:.3f}"

    def test_model_learns_signal_not_noise(self, split, trained_pipeline):
        """With shuffled labels, held-out AUC must collapse to chance.

        This is the target-leakage detector: if a model still scores well after
        the labels are randomised, information is leaking from the target into
        the features. The result is averaged over three shuffles because a single
        permutation on a held-out set of this size has a standard error of roughly
        0.03 AUC -- asserting on one draw would make the test flaky.
        """
        aucs = []
        for seed in (11, 12, 13):
            rng = np.random.default_rng(seed)
            shuffled = pd.Series(
                rng.permutation(split.y_train.to_numpy()), index=split.y_train.index
            )
            trainer = ChurnModelTrainer()
            trainer.train(split.x_train, shuffled)
            aucs.append(
                roc_auc_score(split.y_test, trainer.pipeline.predict_proba(split.x_test)[:, 1])
            )
        mean_auc = float(np.mean(aucs))
        real_auc = roc_auc_score(split.y_test, trained_pipeline.predict_proba(split.x_test)[:, 1])

        assert abs(mean_auc - 0.5) < 0.12, f"Shuffled-label AUC {mean_auc:.3f} is not near chance"
        assert (
            real_auc - mean_auc > 0.20
        ), f"Real model ({real_auc:.3f}) barely beats shuffled labels ({mean_auc:.3f})"


class TestReproducibilityAndGates:
    def test_same_seed_gives_identical_predictions(self, split):
        first = ChurnModelTrainer()
        first.train(split.x_train, split.y_train)
        second = ChurnModelTrainer()
        second.train(split.x_train, split.y_train)
        np.testing.assert_allclose(
            first.pipeline.predict_proba(split.x_test)[:, 1],
```

```
            second.pipeline.predict_proba(split.x_test)[:, 1],
            rtol=1e-9,
        )

    def test_trained_model_clears_release_gates(self, trained_trainer, split):
        metrics = trained_trainer.evaluate(split.x_test, split.y_test)
        passed, failures = metrics.passes_gates()
        assert passed, f"Quality gates failed: {failures}"

    def test_artifact_round_trips_with_metadata(self, artifact_path):
        import joblib

        payload = joblib.load(artifact_path)
        assert "pipeline" in payload
        assert payload["metadata"]["input_features"] == SETTINGS.all_features
        assert payload["metadata"]["hyperparameters"]["calibration"] == "isotonic"
```

`tests/test_model_inference.py` — 171 lines

27 ML behavioural tests for inference — minimum functionality, invariance, directional expectation, robustness and risk-band boundaries.

```
"""ML BEHAVIOURAL TESTS -- model INFERENCE.

Following the "Behavioural Testing of NLP Models" (CheckList) taxonomy, adapted
to tabular churn:

* **Minimum functionality** -- output shape, dtype and range.
* **Invariance tests** -- perturbations that must NOT change the prediction
  (row order, an irrelevant identifier, unit-preserving reordering of columns).
* **Directional expectation tests** -- perturbations whose effect on the
  prediction has a known *sign*. Domain knowledge says raising support tickets
  can only increase churn risk; a model that disagrees is wrong even if its
  aggregate AUC is high.
"""

from __future__ import annotations

import numpy as np
import pandas as pd
import pytest

from churnguard.config import SETTINGS
from churnguard.exceptions import InferenceError, ModelNotLoadedError
from churnguard.models.predictor import ChurnPredictor, assign_risk_band


@pytest.fixture(scope="module")
def predictor(artifact_path) -> ChurnPredictor:
    return ChurnPredictor(artifact_path).load()


@pytest.fixture
def base_record(valid_payload) -> dict:
    return {k: v for k, v in valid_payload.items() if k in SETTINGS.all_features}


class TestMinimumFunctionality:
    def test_output_shape_matches_input(self, predictor, split):
        probabilities = predictor.predict_proba(split.x_test)
        assert probabilities.shape == (len(split.x_test),)
```

```
40
41      def test_probabilities_are_in_the_unit_interval(self, predictor, split):
42          probabilities = predictor.predict_proba(split.x_test)
43          assert np.all((probabilities >= 0.0) & (probabilities <= 1.0))
44
45      def test_no_nan_or_inf_in_output(self, predictor, split):
46          assert np.all(np.isfinite(predictor.predict_proba(split.x_test)))
47
48      def test_single_record_returns_a_complete_prediction(self, predictor, base_record):
49          prediction = predictor.predict_one(base_record)
50          assert 0.0 <= prediction.churn_probability <= 1.0
51          assert prediction.churn_prediction in {0, 1}
52          assert prediction.risk_band in {"LOW", "MEDIUM", "HIGH", "CRITICAL"}
53          assert prediction.recommended_action
54
55      def test_prediction_is_consistent_with_threshold(self, predictor, split):
56          predictions = predictor.predict(split.x_test.head(100))
57          for prediction in predictions:
58              expected = int(prediction.churn_probability >= predictor.threshold)
59              assert prediction.churn_prediction == expected
60
61
62  class TestInvariance:
63      def test_row_order_does_not_change_individual_predictions(self, predictor, split):
64          sample = split.x_test.head(40).reset_index(drop=True)
65          original = predictor.predict_proba(sample)
66          reversed_frame = sample.iloc[::-1].reset_index(drop=True)
67          reversed_probabilities = predictor.predict_proba(reversed_frame)[::-1]
68          np.testing.assert_allclose(original, reversed_probabilities, rtol=1e-9)
69
70      def test_batching_matches_one_by_one_scoring(self, predictor, split):
71          """Batch scoring must equal single scoring -- guards against stateful leakage."""
72          sample = split.x_test.head(20).reset_index(drop=True)
73          batched = predictor.predict_proba(sample)
74          individually = np.array(
75              [predictor.predict_proba(sample.iloc[[i]])[0] for i in range(len(sample))]
76          )
77          np.testing.assert_allclose(batched, individually, rtol=1e-9)
78
79      def test_customer_id_does_not_influence_the_score(self, predictor, base_record):
80          a = predictor.predict_one({**base_record, "customer_id": "CUST-000001"})
81          b = predictor.predict_one({**base_record, "customer_id": "ZZZZ-999999"})
82          assert a.churn_probability == b.churn_probability
83
84      def test_column_order_is_irrelevant(self, predictor, split):
85          sample = split.x_test.head(25)
86          shuffled = sample[list(reversed(sample.columns))]
87          np.testing.assert_allclose(
88              predictor.predict_proba(sample), predictor.predict_proba(shuffled), rtol=1e-9
89          )
90
91
92  class TestDirectionalExpectations:
93      """Each test encodes a business rule the model must not contradict."""
94
95      def _mean_probability(self, predictor, records: list[dict]) -> float:
96          return float(np.mean(predictor.predict_proba(pd.DataFrame(records))))
97
98      def test_more_support_tickets_increases_risk(self, predictor, split):
99          sample = split.x_test.head(60).copy()
100         low = sample.assign(support_tickets_6m=0)
101         high = sample.assign(support_tickets_6m=10)
```

```
        assert predictor.predict_proba(high).mean() > predictor.predict_proba(low).mean()

    def test_longer_tenure_decreases_risk(self, predictor, split):
        sample = split.x_test.head(60).copy()
        new = sample.assign(tenure_months=2)
        loyal = sample.assign(tenure_months=70)
        assert predictor.predict_proba(loyal).mean() < predictor.predict_proba(new).mean()

    def test_two_year_contract_is_safer_than_month_to_month(self, predictor, split):
        sample = split.x_test.head(60).copy()
        monthly = sample.assign(contract_type="Month-to-month")
        biennial = sample.assign(contract_type="Two year")
        assert predictor.predict_proba(biennial).mean() < predictor.predict_proba(monthly).mean()

    def test_tech_support_reduces_risk(self, predictor, split):
        sample = split.x_test.head(60).copy()
        assert (
            predictor.predict_proba(sample.assign(tech_support="Yes")).mean()
            < predictor.predict_proba(sample.assign(tech_support="No")).mean()
        )


class TestRobustnessAndErrors:
    def test_unloaded_predictor_raises(self, tmp_path):
        with pytest.raises(ModelNotLoadedError):
            ChurnPredictor(tmp_path / "missing.joblib").predict_one({})

    def test_missing_feature_raises_inference_error(self, predictor, base_record):
        del base_record["contract_type"]
        with pytest.raises(InferenceError, match="Missing required features"):
            predictor.predict_one(base_record)

    def test_empty_batch_is_rejected(self, predictor):
        with pytest.raises(InferenceError, match="empty batch"):
            predictor.predict_proba(pd.DataFrame(columns=SETTINGS.all_features))

    def test_unseen_category_degrades_gracefully(self, predictor, base_record):
        """handle_unknown='ignore' means an unseen level must not crash the service."""
        prediction = predictor.predict_one({**base_record, "internet_service": "Satellite"})
        assert 0.0 <= prediction.churn_probability <= 1.0

    def test_null_optional_features_are_imputed_not_fatal(self, predictor, base_record):
        prediction = predictor.predict_one(
            {**base_record, "total_charges": None, "avg_monthly_gb": None}
        )
        assert np.isfinite(prediction.churn_probability)


class TestRiskBands:
    @pytest.mark.parametrize(
        ("probability", "expected"),
        [
            (0.05, "LOW"),
            (0.29, "LOW"),
            (0.30, "MEDIUM"),
            (0.59, "MEDIUM"),
            (0.60, "HIGH"),
            (0.84, "HIGH"),
            (0.85, "CRITICAL"),
            (0.99, "CRITICAL"),
        ],
    )
```

```
    def test_band_boundaries(self, probability, expected):
        assert assign_risk_band(probability)[0] == expected

    def test_bands_are_monotonic_in_probability(self):
        order = {"LOW": 0, "MEDIUM": 1, "HIGH": 2, "CRITICAL": 3}
        ranks = [order[assign_risk_band(p)[0]] for p in np.linspace(0, 0.999, 50)]
        assert ranks == sorted(ranks)
```

`tests/test_api_integration.py` — 148 lines

19 integration tests — the full HTTP stack against a real artefact, including the complete status-code contract and degraded-mode behaviour.

```
"""INTEGRATION TESTS -- the HTTP layer end to end.

These exercise the real FastAPI application against a real artefact loaded from
disk: routing, Pydantic validation, the predictor, serialisation and the error
handlers all participate. Nothing is mocked, so a break anywhere in the chain
fails here -- which is exactly the point of an integration test.
"""

from __future__ import annotations

import pytest
from fastapi.testclient import TestClient

from churnguard.api import main as api_main
from churnguard.models.predictor import ChurnPredictor


@pytest.fixture(scope="module")
def client(artifact_path):
    """Point the module-level predictor at the session artefact, then start the app."""
    api_main.predictor = ChurnPredictor(artifact_path)
    with TestClient(api_main.app) as test_client:
        yield test_client


class TestOperationalEndpoints:
    def test_health_reports_a_loaded_model(self, client):
        response = client.get("/health")
        assert response.status_code == 200
        body = response.json()
        assert body["status"] == "ok"
        assert body["model_loaded"] is True

    def test_model_info_exposes_the_model_card(self, client):
        response = client.get("/v1/model/info")
        assert response.status_code == 200
        body = response.json()
        assert body["decision_threshold"] > 0
        assert len(body["input_features"]) == 10
        assert body["hyperparameters"]["calibration"] == "isotonic"

    def test_latency_header_is_present(self, client):
        assert "x-process-time-ms" in {k.lower() for k in client.get("/health").headers}

    def test_openapi_schema_is_served(self, client):
        schema = client.get("/openapi.json").json()
        assert "/v1/predict" in schema["paths"]
        assert "/v1/predict/batch" in schema["paths"]
```

```python


class TestPredictEndpoint:
    def test_happy_path_returns_200_and_full_body(self, client, valid_payload):
        response = client.post("/v1/predict", json=valid_payload)
        assert response.status_code == 200
        body = response.json()
        assert body["customer_id"] == valid_payload["customer_id"]
        assert 0.0 <= body["churn_probability"] <= 1.0
        assert body["risk_band"] in {"LOW", "MEDIUM", "HIGH", "CRITICAL"}
        assert body["recommended_action"]

    def test_high_risk_profile_is_flagged(self, client, valid_payload):
        """Short tenure + month-to-month + many tickets must score as churn risk."""
        response = client.post("/v1/predict", json=valid_payload)
        assert response.json()["churn_prediction"] == 1

    def test_low_risk_profile_is_not_flagged(self, client, valid_payload):
        safe = {
            **valid_payload,
            "tenure_months": 68,
            "support_tickets_6m": 0,
            "contract_type": "Two year",
            "payment_method": "Credit card",
            "tech_support": "Yes",
        }
        assert client.post("/v1/predict", json=safe).json()["churn_prediction"] == 0

    def test_missing_required_field_returns_422(self, client, valid_payload):
        del valid_payload["contract_type"]
        assert client.post("/v1/predict", json=valid_payload).status_code == 422

    def test_out_of_range_value_returns_422(self, client, valid_payload):
        assert (
            client.post("/v1/predict", json={**valid_payload, "tenure_months": -5}).status_code
            == 422
        )

    def test_invalid_category_returns_422(self, client, valid_payload):
        assert (
            client.post(
                "/v1/predict", json={**valid_payload, "contract_type": "Lifetime"}
            ).status_code
            == 422
        )

    def test_wrong_type_returns_422(self, client, valid_payload):
        assert (
            client.post(
                "/v1/predict", json={**valid_payload, "monthly_charges": "free"}
            ).status_code
            == 422
        )

    def test_unknown_extra_field_is_rejected(self, client, valid_payload):
        """extra='forbid' stops silent typos such as 'tenure_month'."""
        assert (
            client.post("/v1/predict", json={**valid_payload, "tenure_month": 3}).status_code == 422
        )

    def test_wrong_http_method_returns_405(self, client):
        assert client.get("/v1/predict").status_code == 405
```

```
111
112     def test_unknown_route_returns_404(self, client):
113         assert client.get("/v1/does-not-exist").status_code == 404
114
115
116 class TestBatchEndpoint:
117     def test_batch_returns_one_prediction_per_customer(self, client, valid_payload):
118         response = client.post("/v1/predict/batch", json={"customers": [valid_payload] * 5})
119         assert response.status_code == 200
120         body = response.json()
121         assert body["count"] == 5
122         assert len(body["predictions"]) == 5
123         assert 0.0 <= body["mean_churn_probability"] <= 1.0
124
125     def test_batch_agrees_with_single_endpoint(self, client, valid_payload):
126         single = client.post("/v1/predict", json=valid_payload).json()
127         batch = client.post("/v1/predict/batch", json={"customers": [valid_payload]}).json()
128         assert batch["predictions"][0]["churn_probability"] == single["churn_probability"]
129
130     def test_empty_batch_returns_422(self, client):
131         assert client.post("/v1/predict/batch", json={"customers": []}).status_code == 422
132
133     def test_oversized_batch_returns_422(self, client, valid_payload):
134         """The 500-record cap is a denial-of-service guard, enforced by the schema."""
135         response = client.post("/v1/predict/batch", json={"customers": [valid_payload] * 501})
136         assert response.status_code == 422
137
138
139 class TestDegradedMode:
140     def test_predict_returns_503_when_no_model_is_loaded(self, tmp_path, valid_payload):
141         api_main.predictor = ChurnPredictor(tmp_path / "absent.joblib")
142         with TestClient(api_main.app) as degraded_client:
143             health = degraded_client.get("/health").json()
144             assert health["status"] == "degraded"
145             response = degraded_client.post("/v1/predict", json=valid_payload)
146             assert response.status_code == 503
147             assert response.json()["error"] == "model_unavailable"
```

Appendix C — Scripts and Configuration

Entry points, reproducibility tooling and the configuration that makes the quality standards enforceable rather than aspirational.

`scripts/generate_data.py` — 100 lines

Deterministic dataset generator. Encodes realistic causal structure and injects ~1.2% missing values so the data-validation tests have genuine defects to detect.

```
"""Generate the synthetic Telco churn dataset used throughout the project.

The generator deliberately encodes realistic, *learnable* relationships (short
tenure, month-to-month contracts, electronic-check payment and high support-ticket
counts all raise churn risk) plus ~1.2% missing values so that the data-validation
tests have something real to catch.
"""

from __future__ import annotations

import argparse
from pathlib import Path

import numpy as np
import pandas as pd

RNG_SEED = 20260815
CONTRACTS = ["Month-to-month", "One year", "Two year"]
INTERNET = ["Fiber optic", "DSL", "No"]
PAYMENTS = ["Electronic check", "Mailed check", "Bank transfer", "Credit card"]


def _sigmoid(x: np.ndarray) -> np.ndarray:
    return 1.0 / (1.0 + np.exp(-x))


def build_dataset(n_rows: int = 5000, seed: int = RNG_SEED) -> pd.DataFrame:
    rng = np.random.default_rng(seed)

    tenure = rng.integers(1, 73, size=n_rows)
    contract = rng.choice(CONTRACTS, size=n_rows, p=[0.55, 0.25, 0.20])
    internet = rng.choice(INTERNET, size=n_rows, p=[0.45, 0.35, 0.20])
    payment = rng.choice(PAYMENTS, size=n_rows, p=[0.35, 0.20, 0.22, 0.23])
    tech_support = rng.choice(["Yes", "No"], size=n_rows, p=[0.38, 0.62])
    paperless = rng.choice(["Yes", "No"], size=n_rows, p=[0.60, 0.40])

    base_charge = np.where(internet == "Fiber optic", 78.0, np.where(internet == "DSL", 52.0, 21.0))
    monthly = np.clip(base_charge + rng.normal(0, 9.0, n_rows), 18.5, 125.0)
    total = np.round(monthly * tenure * rng.uniform(0.94, 1.06, n_rows), 2)
    tickets = rng.poisson(np.where(tech_support == "No", 1.9, 0.7))
    gb = np.clip(
        np.where(internet == "No", rng.normal(2, 1, n_rows), rng.normal(28, 11, n_rows)), 0.0, 90.0
    )

    logit = (
        -1.95
        - 0.045 * tenure
        + 0.017 * monthly
        + 0.30 * tickets
        + 1.15 * (contract == "Month-to-month")
        - 0.75 * (contract == "Two year")
```

```
        + 0.55 * (payment == "Electronic check")
        + 0.42 * (internet == "Fiber optic")
        - 0.45 * (tech_support == "Yes")
        + 0.18 * (paperless == "Yes")
        + rng.normal(0, 0.45, n_rows)
    )
    churn = (rng.uniform(size=n_rows) < _sigmoid(logit)).astype(int)

    frame = pd.DataFrame(
        {
            "customer_id": [f"CUST-{i:06d}" for i in range(1, n_rows + 1)],
            "tenure_months": tenure,
            "monthly_charges": np.round(monthly, 2),
            "total_charges": total,
            "support_tickets_6m": tickets,
            "avg_monthly_gb": np.round(gb, 1),
            "contract_type": contract,
            "internet_service": internet,
            "payment_method": payment,
            "tech_support": tech_support,
            "paperless_billing": paperless,
            "churn": churn,
        }
    )

    # Inject realistic missingness (~1.2% of total_charges, ~0.6% of avg_monthly_gb).
    for column, fraction in (("total_charges", 0.012), ("avg_monthly_gb", 0.006)):
        idx = rng.choice(frame.index, size=int(fraction * n_rows), replace=False)
        frame.loc[idx, column] = np.nan

    return frame


def main() -> None:
    parser = argparse.ArgumentParser(description="Generate the ChurnGuard dataset.")
    parser.add_argument("--rows", type=int, default=5000)
    parser.add_argument("--out", type=Path, default=Path("data/raw/telco_churn.csv"))
    args = parser.parse_args()

    frame = build_dataset(args.rows)
    args.out.parent.mkdir(parents=True, exist_ok=True)
    frame.to_csv(args.out, index=False)
    rate = frame["churn"].mean()
    print(f"Wrote {len(frame):,} rows to {args.out} | churn rate = {rate:.3f}")


if __name__ == "__main__":
    main()
```

`scripts/train.py` — 92 lines

Training entry point: ingest → validate → train → evaluate → gate → persist. Exits 1 on any gate breach, which is what allows it to be used directly as a CI step.

```
"""Training entry point: ingest -> validate -> train -> evaluate -> gate -> persist.

Run:  python scripts/train.py --cv 5

Exit code 1 on a data-contract breach or a model that misses its quality gates,
so the same command can be dropped straight into a CI job.
"""
```

```

from __future__ import annotations

import argparse
import logging
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

from churnguard.config import SETTINGS  # noqa: E402
from churnguard.data.ingestion import CsvDataSource, DataIngestor  # noqa: E402
from churnguard.data.validation import DataValidator, write_reference_profile  # noqa: E402
from churnguard.logging_config import configure_logging  # noqa: E402
from churnguard.models.trainer import ChurnModelTrainer  # noqa: E402

logger = logging.getLogger("churnguard.train")


def main() -> int:
    parser = argparse.ArgumentParser(description="Train the ChurnGuard model.")
    parser.add_argument("--data", type=Path, default=SETTINGS.paths.raw_data)
    parser.add_argument("--cv", type=int, default=5, help="CV folds; 0 to skip")
    parser.add_argument("--skip-gates", action="store_true", help="train even if gates fail")
    args = parser.parse_args()

    configure_logging()
    logger.info("=" * 78)
    logger.info("ChurnGuard training run starting")
    logger.info("=" * 78)

    # ---- 1. Ingest ----------------------------------------------------
    ingestor = DataIngestor(CsvDataSource(args.data))
    raw = ingestor.load_raw()

    # ---- 2. Validate (hard gate) --------------------------------------
    report = DataValidator().validate(raw)
    if not report.passed and not args.skip_gates:
        logger.error("Aborting: dataset failed validation -> %s", report.errors)
        return 1

    split = ingestor.split(raw)

    # ---- 3. Train -----------------------------------------------------
    trainer = ChurnModelTrainer()
    cv_result = {}
    if args.cv:
        cv_result = trainer.cross_validate(split.x_train, split.y_train, folds=args.cv)
    trainer.train(split.x_train, split.y_train)

    # ---- 4. Evaluate + gate -------------------------------------------
    metrics = trainer.evaluate(split.x_test, split.y_test)
    passed, failures = metrics.passes_gates()
    if not passed:
        logger.error("Model failed quality gates: %s", failures)
        if not args.skip_gates:
            return 1
    else:
        logger.info("All model quality gates passed")

    # ---- 5. Persist ---------------------------------------------------
    write_reference_profile(split.x_train, SETTINGS.paths.reference_profile)
```

```
    trainer.save(SETTINGS.paths.model_artifact, metrics, extra={**cv_result, **split.summary()})
    trainer.write_metrics_report(
        metrics,
        SETTINGS.paths.metrics_report,
        extra={**cv_result, "data_quality": report.to_dict()},
    )

    logger.info("-" * 78)
    logger.info(
        "DONE | ROC-AUC=%.4f  F1=%.4f  Accuracy=%.4f  Brier=%.4f  ECE=%.4f",
        metrics.roc_auc,
        metrics.f1,
        metrics.accuracy,
        metrics.brier,
        metrics.ece,
    )
    logger.info("-" * 78)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

`scripts/make_figures.py` — 162 lines

Generates every figure in Part B from the trained artefact, plus the threshold sweep CSV.

```
"""Produce the figures embedded in the assignment report."""

from __future__ import annotations

import logging
import sys
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402
import numpy as np  # noqa: E402
import pandas as pd  # noqa: E402

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

from sklearn.metrics import roc_curve  # noqa: E402

from churnguard.config import SETTINGS  # noqa: E402
from churnguard.data.ingestion import CsvDataSource, DataIngestor  # noqa: E402
from churnguard.data.validation import population_stability_index  # noqa: E402
from churnguard.models.evaluation import evaluate, reliability_curve  # noqa: E402
from churnguard.models.predictor import ChurnPredictor  # noqa: E402

logging.disable(logging.INFO)
FIG = Path("reports/figures")
FIG.mkdir(parents=True, exist_ok=True)
INK, ACCENT, WARN = "#1b2a41", "#2a6f97", "#c1121f"
plt.rcParams.update({"font.size": 10, "axes.edgecolor": "#8894a3", "axes.labelcolor": INK})


def save(fig, name: str) -> Path:
    path = FIG / name
    fig.tight_layout()
```

```
    fig.savefig(path, dpi=150, bbox_inches="tight", facecolor="white")
    plt.close(fig)
    print(f"  wrote {path}")
    return path


def main() -> None:
    split = DataIngestor(CsvDataSource(SETTINGS.paths.raw_data)).split()
    predictor = ChurnPredictor().load()
    probabilities = predictor.predict_proba(split.x_test)
    y_true = split.y_test.to_numpy()
    metrics = evaluate(y_true, probabilities)

    # --- 1. Confusion matrix -----------------------------------------
    matrix = np.array(metrics.confusion)
    fig, ax = plt.subplots(figsize=(4.4, 3.8))
    ax.imshow(matrix, cmap="Blues", vmin=0, vmax=matrix.max())
    for (i, j), value in np.ndenumerate(matrix):
        ax.text(
            j, i, f"{value}",
            ha="center", va="center", fontsize=16, fontweight="bold",
            color="white" if value > matrix.max() * 0.55 else INK,
        )
    ax.set_xticks([0, 1], ["Pred: Stay", "Pred: Churn"])
    ax.set_yticks([0, 1], ["True: Stay", "True: Churn"])
    ax.set_title(f"Confusion Matrix (threshold = {metrics.threshold})", color=INK, pad=10)
    save(fig, "confusion_matrix.png")

    # --- 2. ROC curve ------------------------------------------------
    fpr, tpr, _ = roc_curve(y_true, probabilities)
    fig, ax = plt.subplots(figsize=(4.6, 3.9))
    ax.plot(fpr, tpr, color=ACCENT, lw=2.2, label=f"ChurnGuard (AUC = {metrics.roc_auc:.3f})")
    ax.plot([0, 1], [0, 1], "--", color="#94a3b8", lw=1.2, label="Random (AUC = 0.500)")
    ax.set_xlabel("False positive rate")
    ax.set_ylabel("True positive rate")
    ax.set_title("ROC Curve - held-out test set", color=INK)
    ax.legend(loc="lower right", frameon=False, fontsize=9)
    ax.spines[["top", "right"]].set_visible(False)
    save(fig, "roc_curve.png")

    # --- 3. Calibration ----------------------------------------------
    predicted, observed = reliability_curve(y_true, probabilities, n_bins=10)
    fig, ax = plt.subplots(figsize=(4.6, 3.9))
    ax.plot([0, 1], [0, 1], "--", color="#94a3b8", lw=1.2, label="Perfect calibration")
    ax.plot(predicted, observed, "o-", color=ACCENT, lw=2, ms=6, label="ChurnGuard (isotonic)")
    ax.set_xlabel("Mean predicted probability")
    ax.set_ylabel("Observed churn rate")
    ax.set_title(f"Reliability Diagram (ECE = {metrics.ece:.4f})", color=INK)
    ax.legend(loc="upper left", frameon=False, fontsize=9)
    ax.spines[["top", "right"]].set_visible(False)
    save(fig, "calibration_curve.png")

    # --- 4. Threshold sweep ------------------------------------------
    from sklearn.metrics import f1_score, precision_score, recall_score

    thresholds = np.arange(0.10, 0.86, 0.01)
    rows = []
    for t in thresholds:
        y_pred = (probabilities >= t).astype(int)
        rows.append(
            {
                "threshold": round(float(t), 2),
```

```
                "precision": precision_score(y_true, y_pred, zero_division=0),
                "recall": recall_score(y_true, y_pred, zero_division=0),
                "f1": f1_score(y_true, y_pred, zero_division=0),
            }
        )
    sweep = pd.DataFrame(rows)
    sweep.to_csv("reports/threshold_sweep.csv", index=False)
    fig, ax = plt.subplots(figsize=(5.6, 3.9))
    ax.plot(sweep.threshold, sweep.precision, color="#6b7280", lw=1.8, label="Precision")
    ax.plot(sweep.threshold, sweep.recall, color="#0f766e", lw=1.8, label="Recall")
    ax.plot(sweep.threshold, sweep.f1, color=WARN, lw=2.4, label="F1")
    ax.axvline(SETTINGS.model.decision_threshold, color=INK, ls=":", lw=1.6)
    ax.annotate(
        f"chosen = {SETTINGS.model.decision_threshold}",
        xy=(SETTINGS.model.decision_threshold, 0.05),
        xytext=(SETTINGS.model.decision_threshold + 0.03, 0.12),
        fontsize=9, color=INK,
    )
    ax.set_xlabel("Decision threshold")
    ax.set_ylabel("Score")
    ax.set_title("Threshold selection: precision / recall trade-off", color=INK)
    ax.legend(frameon=False, fontsize=9)
    ax.spines[["top", "right"]].set_visible(False)
    save(fig, "threshold_sweep.png")

    # --- 5. Drift demonstration -------------------------------------------
    reference = split.x_train["tenure_months"].to_numpy()
    drifted = np.clip(reference * 0.35, 0, None)
    psi_stable = population_stability_index(reference, split.x_test["tenure_months"].to_numpy())
    psi_drift = population_stability_index(reference, drifted)
    fig, ax = plt.subplots(figsize=(5.8, 3.7))
    bins = np.linspace(0, 72, 25)
    ax.hist(reference, bins=bins, alpha=0.65, color=ACCENT, label=f"Training reference (n={len(reference)})")
    ax.hist(drifted, bins=bins, alpha=0.6, color=WARN, label="Simulated live traffic")
    ax.set_xlabel("tenure_months")
    ax.set_ylabel("Customers")
    ax.set_title(
        f"Drift detection: PSI(stable) = {psi_stable:.3f}  |  PSI(drifted) = {psi_drift:.3f}",
        color=INK, fontsize=10,
    )
    ax.legend(frameon=False, fontsize=9)
    ax.spines[["top", "right"]].set_visible(False)
    save(fig, "drift_psi.png")

    # --- 6. Risk band distribution ----------------------------------------
    predictions = predictor.predict(split.x_test)
    bands = pd.Series([p.risk_band for p in predictions]).value_counts()
    order = [b for b in ["LOW", "MEDIUM", "HIGH", "CRITICAL"] if b in bands.index]
    fig, ax = plt.subplots(figsize=(5.0, 3.5))
    colours = {"LOW": "#0f766e", "MEDIUM": "#eab308", "HIGH": "#ea580c", "CRITICAL": WARN}
    ax.bar(order, [bands[b] for b in order], color=[colours[b] for b in order], width=0.6)
    for i, b in enumerate(order):
        ax.text(i, bands[b] + 6, str(bands[b]), ha="center", fontsize=10, color=INK)
    ax.set_ylabel("Customers in test set")
    ax.set_title("Retention workload by risk band", color=INK)
    ax.spines[["top", "right"]].set_visible(False)
    save(fig, "risk_bands.png")

    print(f"\nPSI stable={psi_stable:.4f}  drifted={psi_drift:.4f}")
    print(f"Best F1 threshold = {sweep.loc[sweep.f1.idxmax(), 'threshold']}")


```

```
if __name__ == "__main__":
    main()
```

scripts/api_demo.py — 94 lines

Exercises every endpoint against the real application and produces the HTTP transcripts reproduced in Section 7.

```
"""Exercise every API endpoint and print a transcript for the report."""

from __future__ import annotations

import json
import logging
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

from fastapi.testclient import TestClient  # noqa: E402

from churnguard.api.main import app  # noqa: E402

logging.disable(logging.INFO)

HIGH_RISK = {
    "customer_id": "CUST-004821",
    "tenure_months": 2,
    "monthly_charges": 94.35,
    "total_charges": 188.70,
    "support_tickets_6m": 6,
    "avg_monthly_gb": 47.2,
    "contract_type": "Month-to-month",
    "internet_service": "Fiber optic",
    "payment_method": "Electronic check",
    "tech_support": "No",
    "paperless_billing": "Yes",
}
LOW_RISK = {
    "customer_id": "CUST-000117",
    "tenure_months": 66,
    "monthly_charges": 54.10,
    "total_charges": 3570.60,
    "support_tickets_6m": 0,
    "avg_monthly_gb": 18.4,
    "contract_type": "Two year",
    "internet_service": "DSL",
    "payment_method": "Credit card",
    "tech_support": "Yes",
    "paperless_billing": "No",
}


def show(client, method: str, path: str, payload=None, note: str = "") -> None:
    print("=" * 84)
    print(f"$ curl -X {method} http://localhost:8000{path} \\")
    if payload is not None:
        print("       -H 'Content-Type: application/json' \\")
        print(f"       -d '{json.dumps(payload)[:120]}{'...' if len(json.dumps(payload)) > 120 else ''}'")
    if note:
        print(f"# {note}")
    print("-" * 84)
    response = getattr(client, method.lower())(path, **({"json": payload} if payload else {}))
```

```
    print(f"HTTP/1.1 {response.status_code} {response.reason_phrase}")
    print(f"X-Process-Time-Ms: {response.headers.get('x-process-time-ms')}")
    body = response.json()
    text = json.dumps(body, indent=2)
    print(text if len(text) < 1400 else text[:1400] + "\n  ... (truncated)")
    print()


def main() -> None:
    with TestClient(app) as client:
        show(client, "GET", "/health", note="Liveness + readiness probe")
        show(client, "GET", "/v1/model/info", note="Model card: what exactly is serving?")
        show(client, "POST", "/v1/predict", HIGH_RISK, "Case 1: new fibre customer, 6 tickets")
        show(client, "POST", "/v1/predict", LOW_RISK, "Case 2: loyal two-year contract customer")
        show(
            client, "POST", "/v1/predict/batch",
            {"customers": [HIGH_RISK, LOW_RISK]},
            "Batch scoring with roll-up statistics",
        )
        show(
            client, "POST", "/v1/predict",
            {**HIGH_RISK, "tenure_months": -4},
            "NEGATIVE TEST: out-of-range value -> 422",
        )
        show(
            client, "POST", "/v1/predict",
            {**HIGH_RISK, "contract_type": "Lifetime"},
            "NEGATIVE TEST: category outside the contract -> 422",
        )
        show(
            client, "POST", "/v1/predict",
            {**HIGH_RISK, "tenure_month": 3},
            "NEGATIVE TEST: typo'd extra field (extra='forbid') -> 422",
        )


if __name__ == "__main__":
    main()
```

`requirements.txt` — 23 lines

Fully pinned dependencies. Exact pins rather than ranges, so the model that passes CI is the model that ships.

```
# ---- runtime -------------------------------------------------------------
pandas==3.0.2
numpy==2.4.4
scikit-learn==1.8.0
scipy==1.17.1
joblib==1.5.3
fastapi==0.141.1
pydantic==2.13.4
uvicorn[standard]==0.34.0

# ---- visualisation (training reports) ------------------------------------
matplotlib==3.10.8

# ---- quality assurance ---------------------------------------------------
pytest==9.1.1
pytest-cov==6.0.0
httpx==0.28.1

# ---- code quality --------------------------------------------------------
```

```
black==26.5.1
flake8==7.3.0
isort==8.0.1
```

`pyproject.toml` — 20 lines

black and isort configuration. `profile = "black"` is what stops the two tools from fighting each other on every commit.

```
[project]
name = "churnguard"
version = "1.0.0"
description = "Production-style ML system for telecom customer churn prediction"
requires-python = ">=3.10"

[tool.black]
line-length = 100
target-version = ["py310", "py311", "py312"]

[tool.isort]
profile = "black"          # makes isort's output agree with black's
line_length = 100
known_first_party = ["churnguard"]
src_paths = ["src", "tests", "scripts"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src", "scripts"]
```

`setup.cfg` — 13 lines

flake8 configuration. The ignored checks are the two that contradict black; the per-file ignores are narrowly scoped and individually justified.

```
[flake8]
max-line-length = 100
# E203 and W503 conflict with black's (PEP8-compliant) slice and line-break style.
extend-ignore = E203, W503
max-complexity = 10
exclude = .git,__pycache__,.venv,build,dist,reports/lint_demo
per-file-ignores =
    # Test files legitimately import fixtures that look unused to a linter.
    tests/*: F811
    # Entry-point scripts must alter sys.path before importing the package.
    scripts/*: E402
    tests/conftest.py: E402
```

`Makefile` — 38 lines

The operational interface. CI invokes the same targets a developer does, so "works on my machine" and "works in CI" cannot diverge.

```
.PHONY: help setup data train test lint format serve figures all clean

help:            ## Show this help
    @grep -E '^[a-zA-Z_-]+:.*?## .*$$' $(MAKEFILE_LIST) | awk 'BEGIN{FS=":.*?## "}{printf "  \033[36m%-10s\
033[0m %s\n", $$1, $$2}'

setup:           ## Install all dependencies
    pip install -r requirements.txt

```

```
data:            ## Regenerate the raw dataset
    python scripts/generate_data.py

train:           ## Run the full training pipeline (fails on gate breach)
    python scripts/train.py --cv 5

test:            ## Run the test suite with coverage
    pytest -v --cov=src/churnguard --cov-report=term-missing

lint:            ## Static analysis (CI-safe: checks only, no writes)
    isort --check-only --diff src tests scripts
    black --check src tests scripts
    flake8 src tests scripts

format:          ## Auto-format the codebase
    isort src tests scripts
    black src tests scripts

serve:           ## Start the inference API locally
    uvicorn churnguard.api.main:app --reload --port 8000

figures:         ## Regenerate report figures
    python scripts/make_figures.py

all: format lint test train  ## Format, lint, test and train

clean:
    find . -type d -name __pycache__ -exec rm -rf {} + 2>/dev/null || true
    rm -rf .pytest_cache .coverage
```

`Dockerfile` — 29 lines

Multi-stage build producing a non-root runtime image whose health check reads model readiness, not merely port liveness.

```
# ---- Stage 1: build ------------------------------------------------------
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ---- Stage 2: runtime ----------------------------------------------------
FROM python:3.12-slim
# Never run the service as root: limits blast radius if the process is compromised.
RUN useradd --create-home --shell /bin/bash appuser
WORKDIR /app

COPY --from=builder /install /usr/local
COPY src/ ./src/
COPY artifacts/churn_model.joblib ./artifacts/churn_model.joblib

ENV PYTHONPATH=/app/src \
    PYTHONUNBUFFERED=1 \
    CHURN_LOG_LEVEL=INFO
USER appuser
EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=3s --start-period=20s --retries=3 \
  CMD python -c "import urllib.request,json,sys; \
      b=json.load(urllib.request.urlopen('http://localhost:8000/health')); \
      sys.exit(0 if b['model_loaded'] else 1)"
```

```
27
28 CMD ["uvicorn", "churnguard.api.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]
```

`.github/workflows/ci.yml` — 40 lines

Three-gate CI pipeline: lint, then tests, then a full training run whose model-quality gates must pass. A model regression blocks a merge exactly like a failing unit test.

```yaml
name: ChurnGuard CI

on: [push, pull_request]

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12", cache: pip }

      - name: Install dependencies
        run: pip install -r requirements.txt

      # Gate 1 - code quality. Fails the build on any lint error.
      - name: Lint (isort / black / flake8)
        run: |
          isort --check-only src tests scripts
          black --check src tests scripts
          flake8 src tests scripts

      # Gate 2 - correctness. Unit + data-validation + ML + integration tests.
      - name: Test
        run: pytest -v --cov=src/churnguard --cov-report=term-missing

      # Gate 3 - the model itself. train.py exits 1 if data or model gates fail,
      # so a model regression blocks the merge exactly like a failing unit test.
      - name: Train and enforce model quality gates
        run: |
          python scripts/generate_data.py
          python scripts/train.py --cv 3

      - uses: actions/upload-artifact@v4
        with:
          name: model-and-reports
          path: |
            artifacts/
            reports/
```